AgentRoute: Code Appendix

Complete source code of the project (simulator, protocols, DQN, experiments, visualisation). Each module header names the responsible team member; files written with AI assistance say so in their header.

1. `agentroute/baselines/chord.py`
2. `agentroute/baselines/flooding.py`
3. `agentroute/baselines/registry.py`
4. `agentroute/config.py`
5. `agentroute/data/capabilities.py`
6. `agentroute/dqn/gnn.py`
7. `agentroute/dqn/model.py`
8. `agentroute/dqn/policy.py`
9. `agentroute/dqn/replay.py`
10. `agentroute/dqn/train.py`
11. `agentroute/embeddings.py`
12. `agentroute/forwarding.py`
13. `agentroute/sfib.py`
14. `agentroute/sim/events.py`
15. `agentroute/sim/mobility.py`
16. `agentroute/sim/simulator.py`
17. `agentroute/sim/topology.py`
18. `experiments/common.py`
19. `experiments/compare_encoders.py`
20. `experiments/compare_projection.py`
21. `experiments/run_ablation.py`
22. `experiments/run_all.py`
23. `experiments/run_demo.py`
24. `experiments/run_failure.py`
25. `experiments/run_handover_cost.py`
26. `experiments/run_load.py`
27. `experiments/run_matched.py`
28. `experiments/run_mobility.py`
29. `experiments/run_radius.py`
30. `experiments/run_rate.py`
31. `experiments/run_scale.py`
32. `experiments/run_tau.py`
33. `experiments/run_train.py`
34. `experiments/tune_tau.py`
35. `viz/animate.py`
36. `viz/app_streamlit.py`
37. `viz/export_web_data.py`
38. `viz/plots.py`
39. `tests/test_core.py`
40. `tests/test_sim.py`
41. `docs/build_pdfs.py`
42. `configs/default.yaml`
43. `configs/demo.yaml`

44. `viz/web/src/App.jsx`
45. `viz/web/src/components/EventFeed.jsx`
46. `viz/web/src/components/RunPicker.jsx`
47. `viz/web/src/components/StatsPanel.jsx`
48. `viz/web/src/components/Timeline.jsx`
49. `viz/web/src/components/TopologyView.jsx`
50. `viz/web/src/components/Waterfall.jsx`
51. `viz/web/src/data.js`
52. `viz/web/src/main.jsx`
53. `viz/web/src/styles.css`

agentroute/baselines/chord.py

```
"""Baseline: Chord DHT with exact-match keys.

Routers form a Chord ring (SHA-1 identifiers, finger tables, O(log N)
lookups). An agent stores its capability name under key = SHA-1(name). A
request is looked up under SHA-1(request text). This is how exact-match
discovery works today. Our requests never repeat a capability text word for
word, so this baseline finds nothing BY CONSTRUCTION: its 0% is a definition,
not a measurement. It only serves as a cost reference.

SemanticDHT (v2) is the serious application-layer rival. There is a public
catalogue of capability centroids. An agent registers under its nearest
catalogue entry, a requester looks up its nearest entries, and the node
responsible for an entry compares the intent with the agents stored there
(an inverted index on top of Chord). Routers stay plain IP routers.

Responsible: Ömer Faruk Gedikli (v2 revision written with AI assistance)
"""
import bisect
import hashlib
import re

import numpy as np

from agentroute.sfib import kmeans_summary

SMALL_MSG = 64


def normalize(text):
    return re.sub(r"\s+", " ", re.sub(r"[^a-z0-9 ]", " ", text.lower())).strip()


def chash(text, bits):
    return int(hashlib.sha1(text.encode()).hexdigest(), 16) % (1 << bits)


class Chord:
    def __init__(self, sim, bits):
        self.sim = sim
        self.bits = bits
        self.M = 1 << bits
        self.agent_key = [chash(normalize(sim.card_text[sim.agent_card[a]]), bits)
                          for a in range(sim.n_agents)]
        self.rebuild(count=False)

    # ---- ring ------------------------------------------------------------
    def rebuild(self, count=True):
        sim = self.sim
        nodes = [u for u in sim.G.nodes if u not in sim.known_dead]
        pairs = sorted((chash(f"router-{u}", self.bits), u) for u in nodes)
        self.ring = [i for i, _ in pairs]
        self.ring_nodes = [u for _, u in pairs]
        self.id_of = {u: i for i, u in pairs}
        self.fingers = {u: [self.successor((self.id_of[u] + (1 << k)) % self.M)
                            for k in range(self.bits)] for u in nodes}
        self._cost = {}
        self.store = {}
        self.loc = {}
        for a in range(sim.n_agents):
            if sim.attached[a] >= 0:
                self.put(a, count=count)

    def successor(self, key):
        idx = bisect.bisect_left(self.ring, key) % len(self.ring)
        return self.ring_nodes[idx]

    def _between(self, x, a, b, inclusive=True):
        """x in (a, b] on the ring (or (a, b) if not inclusive)."""
        if a < b:
            return a < x <= b if inclusive else a < x < b
        return (x > a or x <= b) if inclusive else (x > a or x < b)
```

```python
    def route(self, start, key):
        """Overlay path [start, ..., responsible node] of a Chord lookup."""
        n = start
        path = [n]
        for _ in range(2 * self.bits):
            succ = self.fingers[n][0]
            if self._between(key, self.id_of[n], self.id_of[succ]):
                if succ != n:
                    path.append(succ)
                return path
            nxt = n
            for f in reversed(self.fingers[n]):
                if self._between(self.id_of[f], self.id_of[n], key, inclusive=False):
                    nxt = f
                    break
            if nxt == n:
                path.append(succ)
                return path
            path.append(nxt)
            n = nxt
        return path

    def cost(self, start, key):
        """Underlay messages of one lookup (cached)."""
        if (start, key) not in self._cost:
            route = self.route(start, key)
            hops = 0
            for a, b in zip(route, route[1:]):
                p = self.sim.path(a, b)
                hops += len(p) - 1 if p else 0
            self._cost[(start, key)] = hops
        return self._cost[(start, key)]

    # ---- storage -----------------------------------------------------
    def put(self, a, count=True):
        u = int(self.sim.attached[a])
        if u < 0 or u not in self.id_of:
            return
        key = self.agent_key[a]
        self.store.setdefault(self.successor(key), {}).setdefault(key, set()).add(a)
        self.loc[a] = u
        if count:
            hops = self.cost(u, key)
            self.sim.count_ctrl(hops, hops * SMALL_MSG)

    def on_move(self, a):
        self.put(a)

    def refresh(self):
        sim = self.sim
        for a in range(sim.n_agents):
            u = int(sim.attached[a])
            if u >= 0 and u in self.id_of:
                hops = self.cost(u, self.agent_key[a])
                sim.count_ctrl(hops, hops * SMALL_MSG)

    # ---- lookup ------------------------------------------------------
    def query(self, pkt, start):
        sim = self.sim
        reason = yield from sim.radio()
        if reason:
            return sim.finish(pkt, False, reason, at=start)
        key = chash(normalize(pkt.text), self.bits)
        route = self.route(start, key)
        for a, b in zip(route, route[1:]):
            reason, at = yield from sim.traverse(pkt, sim.path(a, b))
            if reason:
                return sim.finish(pkt, False, reason, at=at)
        resp = route[-1]
        agents = [a for a in self.store.get(resp, {}).get(key, set()) if a != pkt.src_agent]
        back = sim.path(resp, start)
        if back:
            reason, at = yield from sim.traverse(pkt, back, size=SMALL_MSG)
            if reason:
                return sim.finish(pkt, False, reason, at=at)
        if not agents:
            return sim.finish(pkt, False, "not_found", at=resp)
```

```
        agent = min(agents)
        target = self.loc[agent]
        reason, at = yield from sim.traverse(pkt, sim.path(start, target), size=SMALL_MSG)
        if reason:
            return sim.finish(pkt, False, reason, at=at)
        reason = yield from sim.deliver_to(pkt, target, agent)
        if reason == "handover":
            reason = "stale_location"
        if reason:
            return sim.finish(pkt, False, reason, at=target)
        return sim.finish(pkt, True, None, agent=agent)


class SemanticDHT(Chord):
    """An inverted index over Chord: catalogue entry -> agents stored at one node."""

    def __init__(self, sim, bcfg):
        self.nprobe = bcfg["sdht_nprobe"]
        # the catalogue is computed from the public capability descriptions
        vecs = sim.card_vecs
        reps, _, _ = kmeans_summary(vecs, np.ones(len(vecs), np.float32), np.zeros(len(vecs), int),
                                    bcfg["sdht_buckets"])
        self.catalogue = reps
        self.entry_key = [chash(f"catalogue-{i}", bcfg["chord_bits"]) for i in range(len(reps))]
        # the registration key of every agent = its nearest catalogue entry
        self.agent_entry = np.argmax(sim.agent_vec @ reps.T, axis=1)
        super().__init__(sim, bcfg["chord_bits"])
        self.agent_key = [self.entry_key[e] for e in self.agent_entry]
        self.rebuild(count=False)

    def probe(self, pkt, start, key, out):
        """Look up one catalogue entry; append (similarity, agent) to out if a match is found."""
        sim = self.sim
        hops_before = pkt.hops
        route = self.route(start, key)
        for a, b in zip(route, route[1:]):
            reason, _ = yield from sim.traverse(pkt, sim.path(a, b))
            if reason:
                return
        resp = route[-1]
        ids = [a for a in self.store.get(resp, {}).get(key, set())
               if a != pkt.src_agent and sim.attached[a] >= 0]
        back = sim.path(resp, start)
        if back:
            reason, _ = yield from sim.traverse(pkt, back, size=SMALL_MSG)
            if reason:
                return
        pkt.hops = hops_before
        if ids:
            sims = sim.agent_vec[ids] @ pkt.intent
            j = int(sims.argmax())
            if sims[j] >= sim.tau:
                out.append((float(sims[j]), ids[j]))

    def query(self, pkt, start):
        sim = self.sim
        reason = yield from sim.radio()
        if reason:
            return sim.finish(pkt, False, reason, at=start)
        order = np.argsort(-(self.catalogue @ pkt.intent))[:self.nprobe]
        out = []
        probes = [sim.env.process(self.probe(pkt, start, self.entry_key[e], out)) for e in order]
        yield sim.env.all_of(probes)
        if not out:
            return sim.finish(pkt, False, "not_found", at=start)
        _, agent = max(out)
        target = self.loc[agent]
        pkt.hops = 0
        reason, at = yield from sim.traverse(pkt, sim.path(start, target), size=SMALL_MSG)
        if reason:
            return sim.finish(pkt, False, reason, at=at)
        reason = yield from sim.deliver_to(pkt, target, agent)
        if reason == "handover":
            reason = "stale_location"
        if reason:
            return sim.finish(pkt, False, reason, at=target)
        return sim.finish(pkt, True, None, agent=agent)
```

agentroute/baselines/flooding.py

```
"""Baseline: TTL-limited flooding.

The request is copied to every neighbour until its TTL runs out. Each router
handles a request only once (duplicate suppression by packet id) and
broadcasts it to its attached agents. A router whose best agent has
cos >= tau sends an answer back to the requester. The requester collects
answers for a fixed window, picks the most similar one and sends the task
there. The window, the answers and the task are part of the cost (v2).

Responsible: Ömer Faruk Gedikli (v2 revision written with AI assistance)
"""
SMALL_MSG = 64


class Flooding:
    def __init__(self, sim, ttl, window_s=0.2):
        self.sim = sim
        self.ttl = ttl
        self.window = window_s    # how long the requester collects answers
        self.copies = 0           # router-to-router copies of the request

    def query(self, pkt, start):
        sim = self.sim
        reason = yield from sim.radio()
        if reason:
            return sim.finish(pkt, False, reason, at=start)
        state = {"seen": set(), "answers": [], "closed": False}
        sim.env.process(self.copy(pkt, state, start, start, self.ttl, None))
        yield sim.env.timeout(self.window)
        state["closed"] = True
        if not state["answers"]:
            return sim.finish(pkt, False, "not_found", at=start)
        _, agent, u = max(state["answers"])
        pkt.hops = 0
        reason = yield from sim.radio()     # the task goes from the UE back to its router
        if reason:
            return sim.finish(pkt, False, reason, at=start)
        reason, at = yield from sim.traverse(pkt, sim.path(start, u), log_hops=False, size=SMALL_MSG)
        if reason:
            return sim.finish(pkt, False, reason, at=at)
        reason = yield from sim.deliver_to(pkt, u, agent)
        if reason == "handover":
            reason = "stale_location"
        if reason:
            return sim.finish(pkt, False, reason, at=u)
        return sim.finish(pkt, True, None, agent=agent)

    def copy(self, pkt, state, start, u, ttl, prev):
        sim = self.sim
        if u in state["seen"]:
            return
        state["seen"].add(u)
        reason = yield from sim.transit(u)
        if reason:
            return
        ids = [a for a in sim.at_router[u] if a != pkt.src_agent]
        if ids:
            sim.count_query()                       # one broadcast to the local agents
            sims = sim.agent_vec[ids] @ pkt.intent
            j = int(sims.argmax())
            if sims[j] >= sim.tau:
                sim.env.process(self.answer(pkt, state, start, u, ids[j], float(sims[j])))
        if ttl == 0:
            return
        for v in sim.G.adj[u]:
            if v == prev or v in sim.known_dead:
                continue
            sim.count_query()
            self.copies += 1
            sim.log.log(t=sim.env.now, type="hop", pkt=pkt.pid, method=sim.method,
                        **{"from": int(u), "to": int(v)})
            sim.env.process(self.forward(pkt, state, start, u, v, ttl - 1))
```

```
    def forward(self, pkt, state, start, u, v, ttl):
        reason = yield from self.sim.link(u, v)
        if reason:
            return
        yield from self.copy(pkt, state, start, v, ttl, u)

    def answer(self, pkt, state, start, u, agent, similarity):
        """The answer travels back to the requester and is real traffic."""
        sim = self.sim
        reason = yield from sim.radio()     # agent -> its router
        if reason:
            return
        back = sim.path(u, start)
        if not back:
            return
        hops_before = pkt.hops
        reason, _ = yield from sim.traverse(pkt, back, log_hops=False, size=SMALL_MSG)
        pkt.hops = hops_before
        if reason:
            return
        reason = yield from sim.radio()     # router -> requesting UE
        if reason or state["closed"]:
            return
        state["answers"].append((similarity, agent, u))
```

agentroute/baselines/registry.py

```
"""Baseline: capability registry (one central server, or several replicas).

The registry servers sit on core routers. Every agent registers its capability
vector and location at the nearest server, refreshes it periodically and
sends an update after every handover. A request travels to the nearest server,
the server returns the best matching agent, and the requester then sends the
task to that agent's last known cell.

With one server this is the DNS-like baseline of the proposal: its router is a
single point of failure and locations are stale for a while after a handover.
With three replicas it is what a real deployment would look like (v2): every
registration is copied to the other replicas, and a replica's failure only
sends its clients to the next one. A lookup (a vector search) and a
registration (a table write) have different service times.

Responsible: Ömer Faruk Gedikli (v2 revision written with AI assistance)
"""
import numpy as np
import simpy

SMALL_MSG = 64
TIE_BAND = 0.02     # agents this close to the best match count as equally good


class Registry:
    def __init__(self, sim, bcfg, n_replicas=1):
        self.sim = sim
        xy = np.array([[sim.G.nodes[c]["x"], sim.G.nodes[c]["y"]] for c in sim.cores])
        center = xy.mean(axis=0)
        # first server in the middle, then the core routers farthest from those chosen
        chosen = [int(np.argmin(np.linalg.norm(xy - center, axis=1)))]
        while len(chosen) < min(n_replicas, len(sim.cores)):
            dist = np.min(np.linalg.norm(xy[:, None] - xy[chosen][None], axis=2), axis=1)
            chosen.append(int(np.argmax(dist)))
        self.nodes = [sim.cores[i] for i in chosen]
        self.servers = {u: simpy.Resource(sim.env, capacity=1) for u in self.nodes}
        self.service = bcfg["registry_service_ms"] / 1000.0
        self.update_service = bcfg["registry_update_ms"] / 1000.0
        self.limit = bcfg["registry_queue_limit"]
        self.expiry = 3 * sim.cfg["sfib"]["refresh_s"]
        self.loc = np.full(sim.n_agents, -1)
        self.stamp = np.full(sim.n_agents, -1e9)
        self.size = sim.query_size

    def _alive_servers(self):
        return [u for u in self.nodes if u not in self.sim.known_dead]

    def nearest(self, u):
        """The closest server that is not known to be down, and the path to it."""
        best, best_path, best_delay = None, None, 1e9
        for s in self._alive_servers():
            path = self.sim.path(u, s)
            if path and self._path_delay(path) < best_delay:
                best, best_path, best_delay = s, path, self._path_delay(path)
        return best, best_path

    def _path_delay(self, path):
        return sum(self.sim.G.edges[a, b]["delay"] for a, b in zip(path, path[1:]))

    def _sync_cost(self, server):
        """A registration is copied from one replica to all the others."""
        msgs = 0
        for s in self._alive_servers():
            if s != server:
                path = self.sim.path(server, s)
                msgs += len(path) - 1 if path else 0
        return msgs

    def refresh(self, period, initial=False):
        """Soft-state registrations, spread uniformly over one refresh period."""
        sim = self.sim
        agents = [a for a in range(sim.n_agents) if sim.attached[a] >= 0]
```

```
        if initial:
            for a in agents:
                server, path = self.nearest(int(sim.attached[a]))
                if path:
                    sim.count_ctrl(len(path) - 1, (len(path) - 1) * self.size)
                    self.loc[a] = sim.attached[a]
                    self.stamp[a] = 0.0
            return
        gap = period / max(len(agents), 1)
        for a in agents:
            yield sim.env.timeout(gap)
            sim.env.process(self.update(a))

    def update(self, a):
        """Agent a tells the registry where it is now."""
        sim = self.sim
        u = int(sim.attached[a])
        if u < 0:
            return
        server, path = self.nearest(u)
        if not path:
            return
        msgs = len(path) - 1 + self._sync_cost(server)
        sim.count_ctrl(msgs, msgs * SMALL_MSG)
        yield sim.env.timeout(self._path_delay(path))
        if not sim.routers[server].alive or len(self.servers[server].queue) >= self.limit:
            return
        with self.servers[server].request() as req:
            yield req
            yield sim.env.timeout(self.update_service)
        self.loc[a] = u
        self.stamp[a] = sim.env.now

    def lookup(self, pkt, start):
        sim = self.sim
        valid = (self.loc >= 0) & (sim.env.now - self.stamp <= self.expiry)
        valid[pkt.src_agent] = False
        if not valid.any():
            return None
        sims = sim.agent_vec @ pkt.intent
        sims[~valid] = -2.0
        best = sims.max()
        if best < sim.tau:
            return None
        ties = np.nonzero(sims >= best - TIE_BAND)[0]
        dist = sim.hop_distance(start)
        return int(min(ties, key=lambda a: (dist.get(int(self.loc[a]), 1e9), -sims[a], a)))

    def query(self, pkt, start):
        sim = self.sim
        reason = yield from sim.radio()
        if reason:
            return sim.finish(pkt, False, reason, at=start)
        server, path = self.nearest(start)
        if not path:
            return sim.finish(pkt, False, "no_route", at=start)
        reason, at = yield from sim.traverse(pkt, path)
        if reason:
            return sim.finish(pkt, False, reason, at=at)
        if len(self.servers[server].queue) >= self.limit:
            return sim.finish(pkt, False, "registry_queue", at=server)
        with self.servers[server].request() as req:
            yield req
            yield sim.env.timeout(self.service)
        agent = self.lookup(pkt, start)
        # the answer goes back to the requester in any case
        reason, at = yield from sim.traverse(pkt, sim.path(server, start), size=SMALL_MSG)
        if reason:
            return sim.finish(pkt, False, reason, at=at)
        if agent is None:
            return sim.finish(pkt, False, "not_found", at=server)
        for _ in range(2):              # answer down to the UE, task back up
            reason = yield from sim.radio()
            if reason:
                return sim.finish(pkt, False, reason, at=start)
        target = int(self.loc[agent])
        task_path = sim.path(start, target)
```

```
        if not task_path:
            return sim.finish(pkt, False, "no_route", at=start)
        reason, at = yield from sim.traverse(pkt, task_path, size=SMALL_MSG)
        if reason:
            return sim.finish(pkt, False, reason, at=at)
        reason = yield from sim.deliver_to(pkt, target, agent)
        if reason == "handover":
            reason = "stale_location"
        if reason:
            return sim.finish(pkt, False, reason, at=target)
        return sim.finish(pkt, True, None, agent=agent)
```

agentroute/config.py

```
"""Config loading and seeding.

Every experiment starts from configs/default.yaml and overrides a few keys,
so all runs are reproducible from a YAML file plus a seed.

Responsible: Ömer Faruk Gedikli
"""
import copy
import random
from pathlib import Path

import numpy as np
import yaml

ROOT = Path(__file__).resolve().parent.parent
DEFAULT_CONFIG = ROOT / "configs" / "default.yaml"
RESULTS = ROOT / "results"


def deep_update(base, override):
    """Recursively merge `override` into `base` (in place) and return base."""
    for key, value in override.items():
        if isinstance(value, dict) and isinstance(base.get(key), dict):
            deep_update(base[key], value)
        else:
            base[key] = copy.deepcopy(value)
    return base


def load_config(path=None, overrides=None):
    """Load the default config, then an optional YAML file, then a dict of overrides."""
    with open(DEFAULT_CONFIG) as f:
        cfg = yaml.safe_load(f)
    if path is not None and Path(path).resolve() != DEFAULT_CONFIG:
        with open(path) as f:
            deep_update(cfg, yaml.safe_load(f) or {})
    if overrides:
        deep_update(cfg, overrides)
    return cfg


def set_by_path(cfg, dotted_key, value):
    """set_by_path(cfg, "sfib.radius", 3) sets cfg["sfib"]["radius"] = 3."""
    keys = dotted_key.split(".")
    node = cfg
    for key in keys[:-1]:
        node = node.setdefault(key, {})
    node[keys[-1]] = value


def seed_everything(seed):
    random.seed(seed)
    np.random.seed(seed)
    try:
        import torch
        torch.manual_seed(seed)
    except ImportError:
        pass
```

agentroute/data/capabilities.py

```
"""Synthetic capability dataset: 50 capability classes.

Each class has
  - two capability descriptions and three example requests, which form the
    capability cards that agents advertise, and
  - ten paraphrased task requests (what a requester sends as its intent).

The requests are written in everyday words on purpose: they never repeat a
capability description word for word. This is exactly the case where an
exact-match lookup (Chord baseline) fails and semantic matching is needed.

Responsible: Bekir Emre Sarıpınar
"""
import random

# (name, [descriptions], [intents])
CLASSES = [
    # ---------------- vision ----------------
    ("object_detection",
     ["object detection in images", "locate and label every object in a photo with bounding boxes"],
     ["find the cars in this photo",
      "draw boxes around all the people in this picture",
      "which things appear in this snapshot and where are they",
      "spot every bicycle in the street camera frame",
      "mark the position of each dog in my picture",
      "count and box the trucks visible in this parking lot image",
      "tell me where the bottles are on this shelf photo",
      "highlight all pedestrians in the dashcam still",
      "detect where the helmets are in this construction site shot",
      "point out the animals visible in this wildlife photograph"]),
    ("image_classification",
     ["image classification into categories", "assign a category label to a whole picture"],
     ["is this picture a cat or a dog",
      "what kind of scene is shown in this photo",
      "sort these product pictures into their categories",
      "tell me which flower species this photo shows",
      "label each of these photos as indoor or outdoor",
      "which type of food is on this plate picture",
      "decide whether this x-ray-free snapshot shows a beach or a forest",
      "group my holiday pictures by what they show",
      "what breed of bird is in this image",
      "classify the clothing item in this catalogue photo"]),
    ("face_recognition",
     ["face recognition and identity verification", "match a face against a gallery of known people"],
     ["check if the person at the door is one of our employees",
      "who is the man in this selfie",
      "unlock access only if the face matches the badge photo",
      "verify that this passport photo and this webcam image show the same person",
      "tag my friends automatically in these party pictures",
      "is this visitor on the authorised staff list based on the camera",
      "compare these two portraits and say whether it is the same individual",
      "identify the speaker on stage from the event photos",
      "confirm the driver is the registered owner using the cabin camera",
      "find all photos of my grandmother in this album"]),
    ("ocr",
     ["optical character recognition of scanned documents", "extract printed text from images"],
     ["read the words written on this receipt photo",
      "turn this scanned letter into editable text",
      "what does the street sign in this picture say",
      "copy the serial number off this label photo",
      "convert the pages of this old book scan into a text file",
      "get the licence plate characters from this camera frame",
      "pull the typed content out of this faxed form",
      "make the screenshot of this table searchable as text",
      "transcribe the menu board in this photo",
      "digitise the handwritten-free printed invoice I photographed"]),
    ("image_captioning",
     ["image captioning that describes a picture in words", "write a natural language description of a
photo"],
     ["describe what is happening in this picture for a blind user",
      "write one sentence about this photo for social media",
      "give me alt text for these website pictures",
```

```
      "tell me in plain words what this camera frame shows",
      "narrate the scene in this holiday snapshot",
      "produce a short caption for each image in my gallery",
      "explain the content of this photograph in a paragraph",
      "what is going on in this picture, in a sentence",
      "summarise this image as a short phrase for the archive",
      "generate accessibility descriptions for these product images"]),
    ("image_segmentation",
     ["semantic segmentation of images at pixel level", "label every pixel of a picture with its
class"],
     ["separate the road from the sidewalk in this street image pixel by pixel",
      "cut out the person from the background of this photo",
      "produce a mask of the sky region in these landscape pictures",
      "outline the exact shape of each organ in this scan slice",
      "split this satellite picture into water, forest and buildings",
      "give me a per-pixel map of the lanes in this driving frame",
      "remove the background around the product precisely",
      "colour each region of this indoor photo by what it is",
      "find the precise boundaries of the tumour area",
      "create a pixel mask for the crops in this drone photo"]),
    ("image_generation",
     ["text-to-image generation", "create new pictures from a written prompt"],
     ["draw me a castle on a floating island at sunset",
      "make an illustration of a robot reading a book",
      "produce a poster image of a futuristic city",
      "paint a watercolour style fox in the snow",
      "design a logo picture for my coffee shop",
      "render a realistic photo of a red sports car on a mountain road",
      "create artwork showing astronauts having a picnic",
      "sketch a cartoon cat wearing sunglasses",
      "generate a background wallpaper with abstract shapes",
      "make a fantasy landscape picture from my description"]),
    ("video_summarization",
     ["video summarization into short highlights", "condense a long video into its key moments"],
     ["give me the two minute highlight reel of this football match",
      "cut this one hour lecture recording down to the important parts",
      "what are the key scenes of this surveillance footage",
      "make a short trailer from my vacation clips",
      "pick the most interesting moments from this livestream",
      "shorten the meeting recording to the main points",
      "find the exciting parts of this race video",
      "create a quick recap clip of the conference talk",
      "skim through this dashcam video and keep only the events",
      "compress the story of this documentary into a few minutes"]),
    ("pose_estimation",
     ["human pose estimation from video", "track body joints and skeleton keypoints of people"],
     ["track how my arms and legs move during this workout clip",
      "find the knee and elbow positions of the dancer",
      "check the posture of the worker lifting boxes on camera",
      "get the skeleton of each player in this basketball video",
      "measure joint angles of the runner frame by frame",
      "detect if the elderly person has fallen using their body position",
      "follow the movement of the golfer's swing",
      "estimate where the shoulders and hips are in this yoga photo",
      "capture my body motion for an animated avatar",
      "analyse the gait of the patient walking in the corridor"]),
    ("medical_imaging",
     ["medical image analysis for X-ray and MRI scans", "detect findings in radiology images"],
     ["look at this chest radiograph for signs of pneumonia",
      "check the brain scan for any lesions",
      "is there a fracture visible in this wrist radiograph",
      "measure the size of the nodule in the lung CT",
      "screen these mammograms for suspicious areas",
      "flag abnormal retina photos from the eye clinic",
      "estimate bone age from a hand radiograph",
      "find signs of stroke in the head CT slices",
      "review this knee MRI for ligament damage",
      "highlight anything unusual in the dental radiograph"]),
    # ---------------- language ----------------
    ("translation",
     ["machine translation between languages", "translate text from one language to another"],
     ["put this paragraph into Turkish",
      "what does this German email say in English",
      "convert my French menu to Spanish",
      "render the user manual in Japanese",
      "I need this contract in Arabic",
      "change these subtitles from Italian to English",
```

```
      "how would you say this sentence in Portuguese",
      "make an English version of this Chinese news article",
      "localise the app strings for Korean users",
      "turn this Russian message into something I can read in English"]),
    ("summarization",
     ["text summarization of long documents", "write a short summary of a long text"],
     ["give me the main points of this 40 page report",
      "make this article into three bullet points",
      "what is the gist of this research paper",
      "boil down these meeting notes to a paragraph",
      "write a tl;dr for this long email thread",
      "condense the policy document for busy managers",
      "shorten this book chapter into a one page overview",
      "what are the key takeaways of this whitepaper",
      "make a brief abstract of my thesis",
      "reduce this legal text to a few sentences anyone can follow"]),
    ("sentiment_analysis",
     ["sentiment analysis of customer reviews", "decide whether an opinion is positive or negative"],
     ["are people happy with our new phone according to these comments",
      "tell me if this tweet sounds angry or pleased",
      "how do customers feel about the hotel based on the feedback",
      "score the mood of these survey answers",
      "find the unhappy comments under our product page",
      "is the tone of this email friendly or hostile",
      "measure public opinion about the election from these posts",
      "rate each restaurant comment as good, bad or neutral",
      "detect frustrated users in the support chat logs",
      "what is the overall feeling in these app store ratings"]),
    ("question_answering",
     ["question answering over a knowledge base", "answer factual questions from stored knowledge"],
     ["who was the first president of Turkey",
      "how tall is the Eiffel tower",
      "what is the capital of Australia",
      "when did the second world war end",
      "which element has the atomic number 8",
      "who wrote the novel Crime and Punishment",
      "how many moons does Mars have",
      "what is the boiling point of water at sea level",
      "in which year did the Berlin wall fall",
      "who discovered penicillin"]),
    ("code_generation",
     ["source code generation from natural language", "write a program from a plain description"],
     ["write a Python function that sorts a list of dictionaries by date",
      "make me a script that renames all photos in a folder",
      "create a REST endpoint in Flask for adding users",
      "build a small JavaScript snippet that validates an email field",
      "implement binary search in C",
      "give me a SQL-free Java class for a bank account",
      "program a bash script to back up my home directory",
      "write the HTML and CSS for a login page",
      "produce a Rust function that reverses a string",
      "make a unit test file for my calculator module"]),
    ("code_review",
     ["static code review and bug detection", "find bugs and security issues in source code"],
     ["check my pull request for mistakes",
      "is there a memory leak in this C++ file",
      "look over this function and tell me what could break",
      "find the off-by-one error in my loop",
      "audit this PHP code for SQL injection holes",
      "why does this Python script crash sometimes",
      "point out risky patterns in this JavaScript module",
      "review the diff before I merge it",
      "spot null pointer problems in this Java class",
      "tell me if this smart contract has vulnerabilities"]),
    ("grammar_correction",
     ["grammar and spelling correction", "fix grammar mistakes and typos in writing"],
     ["proofread my cover letter",
      "fix the typos in this paragraph",
      "make this sentence grammatically right",
      "correct the commas and verb tenses in my essay",
      "check my English email before I send it",
      "clean up the spelling errors in these product descriptions",
      "is 'their going home' written correctly",
      "polish the language of my thesis abstract",
      "find the misspelled words in this report",
      "edit this text so it has no grammar errors"]),
    ("named_entity_extraction",
```

```
     ["named entity extraction from text", "find names of people, places and companies in text"],
     ["list all the companies mentioned in this news article",
      "pull out every person's name from these interview notes",
      "which cities and countries appear in this travel blog",
      "extract the dates and amounts from this contract",
      "find the drug names in these clinical notes",
      "tag the organisations in these press releases",
      "get all the locations mentioned in the incident report",
      "highlight the product names in the customer emails",
      "which politicians are named in this speech transcript",
      "build a list of all brands mentioned in these tweets"]),
    ("legal_analysis",
     ["legal contract analysis", "review agreements for clauses, risks and obligations"],
     ["what are my obligations under this rental agreement",
      "find the termination clause in this employment contract",
      "is there anything risky in this NDA",
      "compare these two supplier agreements",
      "check whether this licence allows commercial use",
      "list the penalties described in this service level agreement",
      "explain the liability terms of this deal in simple words",
      "does this contract auto renew",
      "flag unusual terms in the partnership agreement",
      "summarise the payment conditions in this lease"]),
    ("customer_support_chat",
     ["customer support chatbot conversation", "answer customer questions in a live chat"],
     ["help my customer who cannot log into their account",
      "talk to users asking where their parcel is",
      "handle the refund questions on our website chat",
      "reply to shoppers asking about our return policy",
      "be the first line help desk for our internet subscribers",
      "chat with clients who want to change their booking",
      "answer the frequently asked questions of app users",
      "guide a customer through resetting their router",
      "respond to complaints about late delivery politely",
      "assist visitors who ask about opening hours and prices"]),
    # ---------------- speech and audio ----------------
    ("speech_to_text",
     ["speech-to-text transcription", "convert spoken audio into written text"],
     ["write down what was said in this voice memo",
      "make subtitles for this podcast episode",
      "type out the interview recording",
      "produce a written record of the phone call",
      "turn my dictated notes into text",
      "caption the live lecture as the professor talks",
      "get the words from this voicemail",
      "create a transcript of yesterday's meeting audio",
      "convert the court hearing recording into a document",
      "listen to this audio clip and give me the words"]),
    ("text_to_speech",
     ["text-to-speech voice synthesis", "read text aloud with a natural voice"],
     ["read this article out loud for me",
      "make an audio book from my novel",
      "give my navigation app a spoken voice",
      "produce a voice-over for this explainer video",
      "let the website speak its content to blind visitors",
      "say this announcement through the station speakers",
      "turn these lesson notes into an mp3 I can listen to",
      "create a natural sounding narration of the script",
      "voice the dialogue lines of my game characters",
      "convert the news headlines into spoken audio"]),
    ("speaker_identification",
     ["speaker identification from voice", "recognise who is talking from their voice"],
     ["who is talking in this audio recording",
      "tell apart the different people in the meeting audio",
      "verify the caller is really the account holder by their voice",
      "find every part of the podcast where the host speaks",
      "is this voice message from my boss",
      "label each speaker in the conference call",
      "check if the same person speaks in both clips",
      "identify the narrator of this documentary by voice",
      "detect which family member gave the smart speaker command",
      "match the voice in this ransom call to the suspects"]),
    ("music_generation",
     ["music composition and generation", "compose new songs and melodies"],
     ["make a calm piano tune for my study playlist",
      "create background music for my youtube video",
      "write a short jingle for our radio ad",
```

```
      "produce an upbeat electronic track",
      "come up with a melody for these lyrics",
      "generate a lullaby for my baby",
      "make an epic orchestral piece for a game trailer",
      "create a lo-fi beat I can loop",
      "write a guitar chord progression in a sad mood",
      "compose a theme song for my podcast"]),
    ("noise_suppression",
     ["audio noise suppression", "remove background noise from recordings"],
     ["clean the wind sound out of my outdoor video",
      "get rid of the hum in this podcast recording",
      "make my voice clear despite the traffic behind me",
      "remove keyboard clicks from the meeting audio",
      "filter the static from this old tape",
      "reduce the crowd noise in this interview",
      "cancel the fan sound in my voice call",
      "make the lecture recording less hissy",
      "isolate the speech from the restaurant chatter",
      "improve the sound of this phone recording"]),
    # ---------------- data and analytics ----------------
    ("time_series_forecasting",
     ["time series forecasting", "predict future values from historical data"],
     ["how many units will we sell next month",
      "predict tomorrow's electricity demand for the city",
      "what will the website traffic look like next week",
      "estimate next quarter's revenue from past numbers",
      "forecast the stock level of milk in our shop",
      "how busy will the call centre be on Monday",
      "project the number of passengers for the holiday season",
      "predict the water usage of the building tomorrow",
      "guess the bitcoin-free market index value for next week",
      "what will the server load be during the next hour"]),
    ("anomaly_detection",
     ["anomaly detection in sensor data", "find unusual readings in measurement streams"],
     ["warn me when the factory temperature readings look strange",
      "find the odd spikes in this vibration log",
      "is anything abnormal in the smart meter readings",
      "spot faulty sensors in our weather station network",
      "detect weird behaviour in the pump pressure values",
      "alert if the server metrics start to look unusual",
      "which machines show outlier readings this week",
      "flag strange patterns in the heart rate monitor data",
      "check the pipeline flow measurements for irregularities",
      "find the moments when the turbine sensor went off track"]),
    ("recommendation",
     ["product recommendation for users", "suggest items a person is likely to want"],
     ["what should I watch tonight based on my history",
      "suggest books similar to the ones I liked",
      "which products should we show this shopper next",
      "recommend songs for my morning run",
      "find restaurants this user would probably enjoy",
      "propose courses that fit this student's interests",
      "pick news articles this reader might click",
      "what else might someone who bought this tent want",
      "suggest new friends to follow for this account",
      "personalise the home page items for each visitor"]),
    ("fraud_detection",
     ["fraud detection in financial transactions", "flag suspicious payments and card misuse"],
     ["is this credit card purchase suspicious",
      "find money laundering patterns in these transfers",
      "check if this online order looks like a scam",
      "flag unusual withdrawals from this bank account",
      "detect fake insurance claims in this batch",
      "is someone using a stolen card on our shop",
      "spot risky wire transfers before they leave",
      "find accounts that look like they were taken over",
      "review today's payments for possible fraud",
      "detect fake refund requests in our marketplace"]),
    ("text_to_sql",
     ["database query generation (text to SQL)", "turn a question into a database query"],
     ["show me all customers from Ankara who ordered last month",
      "how many orders were cancelled in March, from our sales tables",
      "list the top ten products by revenue in the database",
      "get the average salary per department from the HR tables",
      "which students failed more than two courses according to the records",
      "count the users who signed up this week in our app database",
      "find suppliers with late deliveries in the purchase tables",
```

```
      "give me the total sales per region from the warehouse",
      "which employees have no manager assigned in the staff table",
      "fetch flights departing after 6 pm from the booking records"]),
    ("data_cleaning",
     ["data cleaning and deduplication of tables", "fix messy spreadsheets and remove duplicate rows"],
     ["remove the duplicate customers from this spreadsheet",
      "fix the inconsistent date formats in my csv",
      "fill in or drop the missing values in this dataset",
      "standardise the city names in the address list",
      "merge records that refer to the same company",
      "find typos in the product category column",
      "tidy up this exported table before analysis",
      "normalise the phone numbers in our contact list",
      "detect rows with impossible ages in the survey data",
      "combine these two lists without repeating anyone"]),
    ("data_visualization",
     ["chart and dashboard generation from data", "plot data as charts and dashboards"],
     ["draw a bar chart of sales per month",
      "make a dashboard of our app's daily users",
      "plot the temperature over the year as a line graph",
      "show this survey result as a pie chart",
      "create a map coloured by population",
      "visualise the network traffic per hour",
      "make a scatter plot of price against size",
      "build a report page with graphs of our KPIs",
      "turn this table into an easy-to-read figure",
      "show me a heatmap of website clicks"]),
    # ---------------- mobility, IoT and networks ----------------
    ("route_planning",
     ["route planning and navigation for vehicles", "find the best driving path between places"],
     ["what is the fastest way to drive from Kadikoy to Besiktas",
      "plan a delivery tour for my van with twelve stops",
      "how do I get to the airport avoiding tolls",
      "find a cycling path to the university",
      "give me turn by turn directions home",
      "plan the trip across Europe with charging stops for my electric car",
      "which way should the ambulance take right now",
      "optimise the bus route through these neighbourhoods",
      "find a walking route through the old town",
      "suggest the shortest path for the garbage trucks"]),
    ("traffic_prediction",
     ["road traffic congestion prediction", "predict traffic jams on roads"],
     ["will the bridge be jammed at 6 pm",
      "how bad will rush hour be on the highway tomorrow",
      "predict which streets will be busy after the match",
      "estimate the delay on the ring road this evening",
      "when is the best time to leave to avoid queues",
      "which intersections will be congested during the festival",
      "forecast car density around the mall on Saturday",
      "will there be a jam near the tunnel in an hour",
      "how crowded will the city centre roads be on Monday morning",
      "predict travel times on the coastal road for the weekend"]),
    ("indoor_localization",
     ["indoor positioning and localization", "find where a device is inside a building"],
     ["where exactly is my phone inside the shopping mall",
      "track the forklifts inside the warehouse",
      "which room is the patient's wristband in",
      "guide me to gate B12 inside the airport",
      "locate the lost tablet somewhere in the office",
      "find which floor the visitor is on using wifi signals",
      "follow the nurses' positions across the hospital wards",
      "show my location on the museum floor plan",
      "where did the robot vacuum stop in the house",
      "track equipment carts inside the factory hall"]),
    ("drone_control",
     ["drone flight path control", "fly unmanned aerial vehicles along a mission"],
     ["fly the quadcopter over the field and come back",
      "keep the UAV hovering above the accident site",
      "plan a survey flight over the solar farm",
      "steer the delivery copter to the rooftop pad",
      "make the aerial robot follow the cyclist",
      "avoid the trees while flying through the orchard",
      "send the UAV to inspect the power line",
      "land the quadcopter safely in the wind",
      "patrol the border fence with the flying camera",
      "coordinate three UAVs to map the forest fire"]),
    ("energy_optimization",
```

```
     ["energy consumption optimization for buildings", "reduce the power usage of heating and
cooling"],
     ["lower the electricity bill of our office block",
      "when should the heating turn on to save power",
      "schedule the air conditioning to waste less",
      "cut the energy use of the school at night",
      "balance the battery and solar panels at home",
      "find which floors consume too much power",
      "tune the thermostat settings for efficiency",
      "plan the washing machine runs for cheap tariff hours",
      "reduce the cooling cost of the server room",
      "save power in the shopping centre lighting"]),
    ("weather_forecasting",
     ["weather forecasting for a location", "predict rain, wind and temperature for a place"],
     ["will it rain in Istanbul tomorrow",
      "how hot will it be in Izmir this weekend",
      "do I need an umbrella this afternoon",
      "what is the wind going to be like at the beach on Sunday",
      "is snow expected in Erzurum next week",
      "should we postpone the outdoor wedding because of storms",
      "what will the sky look like during the concert tonight",
      "will there be frost on the farm tonight",
      "is it a good day for sailing on Friday",
      "how cold will the morning be for my run"]),
    ("intrusion_detection",
     ["network intrusion detection", "spot attacks and malicious traffic on a network"],
     ["is someone hacking into our servers",
      "find the port scans in the firewall logs",
      "alert me if the IoT cameras start talking to strange hosts",
      "detect a DDoS attack on the web service",
      "spot malware beacons in the office traffic",
      "check the VPN logs for brute force logins",
      "which machines are sending suspicious packets",
      "watch the 5G core for signalling attacks",
      "find data being sneaked out of the company",
      "flag attempts to break into the router admin page"]),
    ("predictive_maintenance",
     ["predictive maintenance for industrial machines", "estimate when equipment will fail"],
     ["when will this conveyor motor break down",
      "which pumps need service before they fail",
      "estimate the remaining life of the turbine bearings",
      "plan repairs for the production line before breakdowns",
      "will the elevator need fixing soon",
      "predict which trains need brake replacement",
      "tell me which compressors are wearing out",
      "schedule the factory robot service before it stops",
      "how long until the wind turbine gearbox fails",
      "find machines likely to stop next week"]),
    # ---------------- other services ----------------
    ("meeting_scheduling",
     ["meeting scheduling and calendar management", "find a common time slot and book meetings"],
     ["find a time next week when the whole team is free",
      "book a call with the client on Thursday afternoon",
      "move my dentist appointment to Monday",
      "set up a weekly sync with the project group",
      "when are both professors available for my defence",
      "arrange interviews for the five candidates",
      "put the planning session in everyone's calendar",
      "cancel tomorrow's standup and notify people",
      "reserve a room and time for the workshop",
      "organise a lunch slot that suits all three managers"]),
    ("email_writing",
     ["email drafting and reply writing", "compose professional emails"],
     ["write a polite reply declining the invitation",
      "draft a message asking my professor for an extension",
      "answer this customer complaint in a friendly way",
      "compose a follow-up note after the job interview",
      "write to the landlord about the broken heater",
      "prepare a newsletter message for our subscribers",
      "reply to the supplier asking for a discount",
      "write a thank you note to the conference organisers",
      "draft an out of office message for my holiday",
      "send a formal request for the meeting minutes"]),
    ("web_search",
     ["web search and information retrieval", "find relevant pages and documents online"],
     ["find recent articles about 6G networks",
      "look up reviews of this laptop model",
```

```
      "search for papers on semantic routing",
      "find the official website of the ministry",
      "get me sources about climate change in Turkey",
      "look for tutorials on SimPy",
      "find the opening hours of the museum online",
      "search the web for cheap flights to Rome",
      "find documents explaining 5G handover",
      "dig up news coverage of the earthquake"]),
    ("math_solving",
     ["mathematical equation solving", "solve algebra and calculus problems"],
     ["what is x if 3x plus 7 equals 22",
      "integrate x squared times sine of x",
      "find the derivative of e to the 2x",
      "solve this system of two linear equations",
      "what are the roots of x squared minus 5x plus 6",
      "compute the limit of sin x over x as x goes to zero",
      "simplify this fraction with polynomials",
      "find the eigenvalues of this 2 by 2 matrix",
      "how do I solve this differential equation",
      "calculate the area under the curve between 0 and 2"]),
    ("video_transcoding",
     ["video transcoding and compression", "convert video files to other formats and bitrates"],
     ["make this 4K clip small enough to send on WhatsApp",
      "convert my MOV file to MP4",
      "shrink the lecture recordings without losing much quality",
      "prepare the movie in several resolutions for streaming",
      "change the frame rate of this footage to 30",
      "re-encode the camera files in H.265",
      "turn this video into a lower bitrate version for mobile",
      "export the clip as a GIF",
      "reduce the size of the drone footage for upload",
      "package this film for adaptive streaming"]),
    ("encryption",
     ["data encryption and decryption", "protect files and messages with cryptography"],
     ["lock this file so only my colleague can open it",
      "secure the patient records before uploading them",
      "make this message unreadable to anyone without the key",
      "sign this document so people know it is from me",
      "protect the backup with AES",
      "unlock the encrypted archive with my key",
      "generate a key pair for secure email",
      "hide the credentials in the config file safely",
      "make the sensor data confidential before sending it",
      "verify that this file was not tampered with"]),
    ("invoice_processing",
     ["invoice processing and expense extraction", "read bills and record their amounts"],
     ["add these supplier bills to our accounting system",
      "how much did we pay for electricity according to these bills",
      "pull the totals and tax from these receipts",
      "check that the invoice matches the purchase order",
      "file my travel expenses from these taxi receipts",
      "find duplicate bills in this folder",
      "record the due dates of all unpaid invoices",
      "sort the vendor bills by cost centre",
      "extract the VAT numbers from these PDFs of bills",
      "prepare the monthly expense report from my receipts"]),
    ("symptom_checking",
     ["health symptom checking", "suggest possible causes of symptoms and next steps"],
     ["I have a headache and fever, what could it be",
      "should I see a doctor about this rash",
      "my child has been coughing for a week, is it serious",
      "what might cause pain in my lower back",
      "is chest tightness after running a worry",
      "I feel dizzy when I stand up, why",
      "could these stomach cramps be food poisoning",
      "what should I do about a sprained ankle",
      "my throat hurts and I cannot swallow well",
      "is it normal to feel tired all the time"]),
    ("tutoring",
     ["personalized tutoring and quiz generation", "teach a topic and create practice questions"],
     ["explain fractions to my 10 year old",
      "make a ten question quiz about the French revolution",
      "help me understand how TCP works",
      "give me practice problems on derivatives",
      "teach me the basics of Python step by step",
      "prepare flashcards for my biology exam",
      "check my answers and explain what I got wrong",
```

```
      "create a revision plan for the physics final",
      "quiz me on English irregular verbs",
      "walk me through the theory of relativity simply"]),
    ("robot_grasping",
     ["robot arm grasp planning", "plan how a robot hand picks up objects"],
     ["pick up the cup from the table with the robot hand",
      "how should the manipulator hold this fragile glass",
      "make the arm take boxes off the conveyor",
      "find a stable way to lift this oddly shaped part",
      "sort the parcels into bins with the robot",
      "let the arm grab the screws from the tray",
      "plan the gripper motion to pick the apple",
      "move the tools from the bench to the shelf with the arm",
      "pick fruit gently with the farm robot",
      "load the dishwasher using the kitchen robot"]),
]

# Example requests that each agent lists on its capability card (like the
# "examples" field of an A2A agent card). They are written separately from the
# intents above and never repeat an intent.
EXAMPLES = {
    "object_detection": ["box the vehicles in a traffic camera image", "show where each person stands in a crowd photo", "list the items visible on a desk picture with their positions"],
    "image_classification": ["say what category a photo belongs to", "recognise the animal species in a picture", "tag images as landscape, portrait or food"],
    "face_recognition": ["check that a person's face matches their ID card", "recognise known people in camera images", "grant door access by recognising staff faces"],
    "ocr": ["turn a photographed page into text", "read the numbers printed on a meter photo", "get text out of a scanned PDF"],
    "image_captioning": ["write a sentence describing a picture", "create alt text for photos", "explain in words what a camera frame shows"],
    "image_segmentation": ["separate foreground from background in a photo", "mask each region of a street scene", "outline objects with exact pixel boundaries"],
    "image_generation": ["paint a picture from a written idea", "create an illustration for a story", "make a poster image from a short prompt"],
    "video_summarization": ["make a highlight clip of a long game recording", "keep only the important scenes of a lecture video", "produce a short recap of a long stream"],
    "pose_estimation": ["find the joints of a person in a video", "track body posture of athletes", "estimate arm and leg positions from camera frames"],
    "medical_imaging": ["read a chest scan for disease signs", "find fractures on a bone radiograph", "spot tumours in CT or MRI slices"],
    "translation": ["convert an English text into German", "render a document in another language", "translate chat messages for a foreign customer"],
    "summarization": ["shorten a long report into key points", "write an abstract of an article", "condense meeting notes"],
    "sentiment_analysis": ["tell whether a review is happy or unhappy", "measure how customers feel from their comments", "score the tone of social media posts"],
    "question_answering": ["answer trivia and general knowledge questions", "tell facts about history, science and geography", "respond to who, what, when questions"],
    "code_generation": ["write a function from a description", "create a script that automates a task", "produce program code in Python or Java"],
    "code_review": ["look for defects in a code change", "check software for security holes", "explain why a program crashes"],
    "grammar_correction": ["proofread an essay", "fix typos and wrong verb forms", "make sentences grammatically correct"],
    "named_entity_extraction": ["list the people and organisations named in an article", "pull places and dates out of documents", "tag brand and product names in text"],
    "legal_analysis": ["explain the clauses of an agreement", "find risks in a contract", "check the obligations and penalties in a lease"],
    "customer_support_chat": ["chat with customers about orders and refunds", "answer help desk questions", "talk users through account problems"],
    "speech_to_text": ["write down what is said in a recording", "produce subtitles from audio", "transcribe phone calls and meetings"],
    "text_to_speech": ["read written text aloud", "create narration audio from a script", "give an app a spoken voice"],
    "speaker_identification": ["tell who is speaking in a recording", "separate the voices of different people", "check a caller's identity by voice"],
    "music_generation": ["create a new melody", "produce background tracks for videos", "write a song in a chosen mood"],
    "noise_suppression": ["clean background sounds out of audio", "make speech clearer in noisy recordings", "remove hum and hiss from a track"],
    "time_series_forecasting": ["predict next week's sales from history", "estimate future demand", "project numbers forward in time"],
    "anomaly_detection": ["find abnormal values in machine sensor logs", "alert on strange measurement patterns", "spot outliers in monitoring metrics"],
    "recommendation": ["suggest movies a viewer may like", "pick products to show a shopper", "personalise a list of songs"],
```

```
    "fraud_detection": ["spot suspicious card payments", "detect scams in online orders", "flag money laundering in transfers"],
    "text_to_sql": ["turn a question about a database into a query", "fetch records from tables using plain English", "answer questions from company databases"],
    "data_cleaning": ["remove repeated rows in a table", "fix inconsistent formats in a csv", "fill missing entries in a dataset"],
    "data_visualization": ["draw graphs from a table of numbers", "make a dashboard of metrics", "plot values as bar, line or pie charts"],
    "route_planning": ["find the quickest road between two addresses", "plan a multi-stop delivery trip", "give driving directions"],
    "traffic_prediction": ["predict jams on city roads", "estimate travel delay at rush hour", "forecast how busy a highway will be"],
    "indoor_localization": ["find a phone's position inside a building", "track assets inside a warehouse", "navigate people inside an airport or mall"],
    "drone_control": ["fly a UAV over an area", "steer a quadcopter to a target", "plan an inspection flight"],
    "energy_optimization": ["cut the electricity use of a building", "schedule heating and cooling to save power", "manage solar and battery at home"],
    "weather_forecasting": ["say whether it will rain tomorrow", "predict temperature and wind for a city", "warn about storms or snow"],
    "intrusion_detection": ["detect hackers attacking a network", "find malicious traffic in logs", "alert on scans and denial of service"],
    "predictive_maintenance": ["predict when a machine will break", "plan repairs before equipment fails", "estimate remaining life of parts"],
    "meeting_scheduling": ["find a free time for a group", "book appointments in a calendar", "reschedule or cancel meetings"],
    "email_writing": ["draft a reply to a message", "write a formal letter by email", "send a polite reminder after no answer"],
    "web_search": ["look up information on the internet", "find articles and documents on a topic", "search websites for an answer"],
    "math_solving": ["solve an equation step by step", "compute derivatives and integrals", "work out algebra homework"],
    "video_transcoding": ["convert a video to another format", "make video files smaller", "prepare videos for streaming at several qualities"],
    "encryption": ["encrypt a file with a key", "decrypt protected data", "digitally sign and verify documents"],
    "invoice_processing": ["read amounts and dates from bills", "enter receipts into accounting", "match invoices to orders"],
    "symptom_checking": ["suggest what might cause a symptom", "advise whether to see a doctor", "give first steps for common illnesses"],
    "tutoring": ["teach a school topic simply", "make practice questions and quizzes", "explain mistakes in homework"],
    "robot_grasping": ["pick objects with a robot gripper", "plan how a robot arm lifts items", "move parts between places with a manipulator"],
}


def class_names():
    return [name for name, _, _ in CLASSES]


def capability_cards():
    """Two capability cards per class, as (class_id, variant_id, sentences).

    A card is one description plus two example requests. Two variants model
    two different providers describing the same skill in different words.
    """
    rows = []
    for class_id, (name, descs, _) in enumerate(CLASSES):
        ex = EXAMPLES[name]
        rows.append((class_id, 0, [descs[0], ex[0], ex[1]]))
        rows.append((class_id, 1, [descs[1], ex[1], ex[2]]))
    return rows


def class_sentences():
    """All five capability sentences of each class: two descriptions + three examples.

    Every agent mixes these with its own random weights (v2), so that no two
    agents advertise the same capability vector.
    """
    return [list(descs) + list(EXAMPLES[name]) for name, descs, _ in CLASSES]


def split_intents(n_test=3, seed=0):
    """Split the ten intents of each class into train and test sets.
```

```
    Returns two lists of (class_id, text). The split is fixed by the seed so
    that the DQN is never trained on the requests used for evaluation.
    """
    rng = random.Random(seed)
    train, test = [], []
    for class_id, (_, _, intents) in enumerate(CLASSES):
        order = list(range(len(intents)))
        rng.shuffle(order)
        for i, idx in enumerate(order):
            if i < n_test:
                test.append((class_id, intents[idx]))
            else:
                train.append((class_id, intents[idx]))
    return train, test
```

agentroute/dqn/gnn.py

```python
"""GNN variant of the Q-network.

The plain QNet scores every neighbour in isolation. Here each neighbour is
first embedded, then the embeddings of all neighbours are pooled (mean and
max) into a context vector, i.e. one round of message passing on the local
star graph "router <-> its neighbours". The score of a neighbour can then
depend on the alternatives (e.g. "this path is congested, but so are all the
others").

Responsible: Bekir Göktepe
"""
import torch
from torch import nn

from agentroute.dqn.model import NEG


class GNNQNet(nn.Module):
    def __init__(self, n_features, hidden=64):
        super().__init__()
        self.encode = nn.Sequential(nn.Linear(n_features, hidden), nn.ReLU(),
                                    nn.Linear(hidden, hidden), nn.ReLU())
        self.score = nn.Sequential(nn.Linear(3 * hidden, hidden), nn.ReLU(),
                                   nn.Linear(hidden, 1))

    def forward(self, x, mask):
        h = self.encode(x)                                      # (B, D, H)
        m = mask.unsqueeze(-1).float()
        mean = (h * m).sum(dim=1) / m.sum(dim=1).clamp(min=1.0)
        mx = h.masked_fill(~mask.unsqueeze(-1), NEG).max(dim=1).values
        mx = torch.where(mask.any(dim=1, keepdim=True), mx, torch.zeros_like(mx))
        ctx = torch.cat([mean, mx], dim=-1).unsqueeze(1).expand(-1, h.shape[1], -1)
        q = self.score(torch.cat([h, ctx], dim=-1)).squeeze(-1)
        return q.masked_fill(~mask, NEG)
```

agentroute/dqn/model.py

```python
"""Q-network shared by all routers.

The number of neighbours differs from router to router, so the network does
not output one Q-value per action. Instead it scores each action on its own:
input = the feature vector of one action (a neighbour, or "deliver here"),
output = one Q-value. The action with the highest score is taken.

Batches are padded to a fixed maximum degree; `mask` marks real neighbours.

Responsible: Bekir Göktepe
"""
import torch
from torch import nn

MAX_DEG = 32
NEG = -1e9


class QNet(nn.Module):
    def __init__(self, n_features, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1),
        )

    def forward(self, x, mask):
        """x: (B, D, F) neighbour features, mask: (B, D) bool -> Q: (B, D)."""
        q = self.net(x).squeeze(-1)
        return q.masked_fill(~mask, NEG)
```

agentroute/dqn/policy.py

```python
"""DQN policy (AgentRoute): deliver here, or forward to which neighbour.

At every router the shared Q-network scores each possible action and takes the
best one. v2 adds "deliver to the best local agent" as an action, so the
network also decides whether a local match is good enough or a better one is
known further away. That decision is what the reward depends on most.

    reward per decision = -(hop_cost + delay / delay_scale)
    on correct delivery = + deliver_bonus
    on any drop         = - drop_penalty   (TTL, queue, link loss, wrong agent ...)

The next-state value is the max Q over the actions of the next router, which
is the Q-routing update (Boyan & Littman, 1994) with a neural network.

Responsible: Bekir Göktepe (v2 revision written with AI assistance)
"""
from pathlib import Path

import numpy as np
import torch

from agentroute.dqn.gnn import GNNQNet
from agentroute.dqn.model import MAX_DEG, QNet
from agentroute.forwarding import LOCAL, N_FEATURES, Policy, best_local_agent, neighbor_features


def build_model(kind, hidden=64):
    if kind == "gnn":
        return GNNQNet(N_FEATURES, hidden)
    return QNet(N_FEATURES, hidden)


class DQNPolicy(Policy):
    def __init__(self, model, kind="mlp", epsilon=0.0, learner=None, rng=None, reward=None):
        super().__init__(reward)
        self.model = model
        self.kind = kind
        self.name = "agentroute_gnn" if kind == "gnn" else "agentroute"
        self.epsilon = epsilon
        self.learner = learner
        self.rng = rng if rng is not None else np.random.default_rng(0)
        self._cache = None
        self._next = None

    @classmethod
    def load(cls, path, kind="mlp"):
        ckpt = torch.load(Path(path), map_location="cpu", weights_only=True)
        model = build_model(kind, ckpt["hidden"])
        model.load_state_dict(ckpt["state_dict"])
        model.eval()
        return cls(model, kind=kind)

    def save(self, path, hidden):
        Path(path).parent.mkdir(parents=True, exist_ok=True)
        torch.save({"state_dict": self.model.state_dict(), "hidden": hidden, "kind": self.kind}, path)

    def q_values(self, X):
        d = min(len(X), MAX_DEG)
        x = torch.zeros((1, MAX_DEG, X.shape[1]))
        x[0, :d] = torch.from_numpy(X[:d])
        mask = torch.zeros((1, MAX_DEG), dtype=torch.bool)
        mask[0, :d] = True
        with torch.no_grad():
            return self.model(x, mask)[0, :d].numpy()

    def _features(self, view, pkt):
        if self._cache is not None and self._cache[0] is view and self._cache[1] == pkt.pid:
            return self._cache[2], self._cache[3]
        return neighbor_features(view, pkt, with_local=True)

    def accept(self, view, pkt):
        """The one decision per router. Returns an agent, or None and remembers the next hop."""
```

```
        actions, X = self._features(view, pkt)
        self._cache = None
        self._next = None
        if not actions:
            return None
        if pkt.ttl <= 0:            # the hop budget is used up: delivering is the only option
            a = actions.index(LOCAL)
        elif self.epsilon > 0 and self.rng.random() < self.epsilon:
            a = int(self.rng.integers(len(actions)))
        else:
            a = int(np.argmax(self.q_values(X)))
        pkt.pending = (X, a, view.now)
        if actions[a] == LOCAL:
            return best_local_agent(view, pkt)
        self._next = actions[a]
        return None

    def choose(self, view, pkt):
        return self._next

    def learn(self, X, a, r, pkt, next_view, outcome):
        if self.learner is None:
            return
        if outcome is None:
            actions2, X2 = neighbor_features(next_view, pkt, with_local=True)
            self._cache = (next_view, pkt.pid, actions2, X2)
            self.learner.push(X, a, r, X2, False)
        else:
            self.learner.push(X, a, r, None, True)
```

agentroute/dqn/replay.py

```
"""Experience replay buffer for per-neighbour DQN transitions.

A transition is (state, action, reward, next state, done) where a state is
the padded feature matrix of the candidate neighbours of one router.

Responsible: Bekir Göktepe
"""
import numpy as np
import torch

from agentroute.dqn.model import MAX_DEG


class ReplayBuffer:
    def __init__(self, capacity, n_features, rng):
        self.capacity = capacity
        self.rng = rng
        self.s = np.zeros((capacity, MAX_DEG, n_features), np.float32)
        self.s_mask = np.zeros((capacity, MAX_DEG), bool)
        self.a = np.zeros(capacity, np.int64)
        self.r = np.zeros(capacity, np.float32)
        self.s2 = np.zeros((capacity, MAX_DEG, n_features), np.float32)
        self.s2_mask = np.zeros((capacity, MAX_DEG), bool)
        self.done = np.zeros(capacity, bool)
        self.size = 0
        self.pos = 0

    def push(self, x, a, r, x2, done):
        i = self.pos
        d = min(len(x), MAX_DEG)
        self.s[i] = 0
        self.s[i, :d] = x[:d]
        self.s_mask[i] = False
        self.s_mask[i, :d] = True
        self.a[i] = a
        self.r[i] = r
        self.s2[i] = 0
        self.s2_mask[i] = False
        if x2 is not None and len(x2):
            d2 = min(len(x2), MAX_DEG)
            self.s2[i, :d2] = x2[:d2]
            self.s2_mask[i, :d2] = True
        self.done[i] = done or x2 is None or len(x2) == 0
        self.pos = (self.pos + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def __len__(self):
        return self.size

    def sample(self, batch):
        idx = self.rng.integers(self.size, size=batch)
        return (torch.from_numpy(self.s[idx]), torch.from_numpy(self.s_mask[idx]),
                torch.from_numpy(self.a[idx]), torch.from_numpy(self.r[idx]),
                torch.from_numpy(self.s2[idx]), torch.from_numpy(self.s2_mask[idx]),
                torch.from_numpy(self.done[idx]))
```

agentroute/dqn/train.py

```
"""Offline Double-DQN training in the simulator.

Each episode is one simulation run in a randomly drawn scenario (agent count,
mobility speed, load, failures, radius), so the shared network sees many
different situations. Training uses the *train* paraphrases only; the test
paraphrases are kept for the evaluation experiments.

Every `eval_every` episodes the policy is evaluated without exploration on
fixed validation scenarios and the best weights are kept. v2 computes the same
return for greedy and for load-aware greedy on the same scenarios, so the
learning curve can be read against both.

Responsible: Bekir Göktepe (v2 revision written with AI assistance)
"""
import copy
import csv
import time
from pathlib import Path

import numpy as np
import torch
import torch.nn.functional as F

from agentroute.config import ROOT, deep_update, load_config, seed_everything
from agentroute.dqn.policy import DQNPolicy, build_model
from agentroute.dqn.replay import ReplayBuffer
from agentroute.forwarding import N_FEATURES, GreedyPolicy, LoadAwareGreedy
from agentroute.sim.simulator import Simulator


class Learner:
    def __init__(self, model, dqn_cfg, rng):
        self.online = model
        self.target = copy.deepcopy(model)
        self.cfg = dqn_cfg
        self.opt = torch.optim.Adam(model.parameters(), lr=dqn_cfg["lr"])
        self.buffer = ReplayBuffer(dqn_cfg["buffer_size"], N_FEATURES, rng)
        self.pushed = 0
        self.steps = 0
        self.losses = []

    def push(self, x, a, r, x2, done):
        self.buffer.push(x, a, r, x2, done)
        self.pushed += 1
        if self.pushed % self.cfg["train_every"] == 0 and len(self.buffer) >= 4 *
self.cfg["batch_size"]:
            self.train_step()

    def train_step(self):
        s, s_mask, a, r, s2, s2_mask, done = self.buffer.sample(self.cfg["batch_size"])
        q = self.online(s, s_mask).gather(1, a[:, None]).squeeze(1)
        with torch.no_grad():
            # Double DQN: the online net picks the next action, the target net values it
            a2 = self.online(s2, s2_mask).argmax(dim=1)
            q2 = self.target(s2, s2_mask).gather(1, a2[:, None]).squeeze(1)
            q2 = torch.where(done | ~s2_mask.any(dim=1), torch.zeros_like(q2), q2)
            y = r + self.cfg["gamma"] * q2
        loss = F.smooth_l1_loss(q, y)
        self.opt.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.online.parameters(), 1.0)
        self.opt.step()
        self.steps += 1
        self.losses.append(loss.item())
        if self.steps % self.cfg["target_update"] == 0:
            self.target.load_state_dict(self.online.state_dict())


def sample_scenario(rng):
    """A random training scenario, as config overrides (biased towards loaded networks)."""
    return {
        "agents": {"n_agents": int(rng.choice([300, 600, 1000, 2000])),
```

```
                    "speed_mps": float(rng.choice([0.0, 2.0, 5.0, 10.0, 20.0]))},
        "sim": {"duration_s": 20.0, "load": float(rng.choice([0.5, 1.0, 2.0, 3.0, 4.0])),
                "failure_fraction": float(rng.choice([0.0, 0.0, 0.1, 0.2]))},
        "sfib": {"radius": int(rng.choice([2, 3, 4]))},
    }


VALIDATION = [
    {"agents": {"n_agents": 1000, "speed_mps": 5.0}, "sim": {"duration_s": 20.0, "load": 2.0}},
    {"agents": {"n_agents": 1000, "speed_mps": 10.0}, "sim": {"duration_s": 20.0, "load": 3.0,
                                                              "failure_fraction": 0.1}},
    {"agents": {"n_agents": 2000, "speed_mps": 2.0}, "sim": {"duration_s": 20.0, "load": 1.0}},
]


def evaluate(policy, method, base_cfg, seeds=(9001, 9002, 9003)):
    """Validation on fixed scenarios (train intents).

    Returns (success rate, completion time in ms, return per request).
    """
    rates, times, returns = [], [], []
    for scen in VALIDATION:
        for seed in seeds:
            cfg = deep_update(copy.deepcopy(base_cfg), scen)
            policy.episode_return = 0.0
            s = Simulator(cfg, method, seed=seed, policy=policy, intents="train").run()
            rates.append(s["success_rate"])
            times.append(s["time_ms"])
            returns.append(policy.episode_return / max(s["n_queries"], 1))
    return float(np.mean(rates)), float(np.nanmean(times)), float(np.mean(returns))


def train(kind="mlp", episodes=None, config_path=None, out_csv=None, verbose=True):
    base = load_config(config_path)
    dqn = base["dqn"]
    seed_everything(base["seed"])
    torch.set_num_threads(2)
    rng = np.random.default_rng(base["seed"])
    episodes = episodes or dqn["episodes"]
    model = build_model(kind, dqn["hidden"])
    learner = Learner(model, dqn, rng)
    policy = DQNPolicy(model, kind=kind, learner=learner, rng=rng, reward=dqn["reward"])
    model_path = ROOT / (dqn["gnn_model_path"] if kind == "gnn" else dqn["model_path"])
    out_csv = Path(out_csv or ROOT / "results" / "raw" / f"train_{kind}.csv")
    out_csv.parent.mkdir(parents=True, exist_ok=True)

    greedy_val = evaluate(GreedyPolicy(dqn["reward"]), "greedy", base)
    load_val = evaluate(LoadAwareGreedy(base["forwarding"]["load_band"], dqn["reward"]), "greedy_load",
base)
    best_val = -1e9
    rows = []
    decay_eps = max(1, int(dqn["eps_decay_frac"] * episodes))
    t_start = time.time()
    for ep in range(episodes):
        eps = max(dqn["eps_end"], dqn["eps_start"] - (dqn["eps_start"] - dqn["eps_end"]) * ep /
decay_eps)
        cfg = deep_update(copy.deepcopy(base), sample_scenario(rng))
        policy.epsilon = eps
        policy.learner = learner
        policy.episode_return = 0.0
        n_losses = len(learner.losses)
        s = Simulator(cfg, "agentroute", seed=10_000 + ep, policy=policy, intents="train").run()
        row = {"episode": ep, "epsilon": round(eps, 3), "success_rate": s["success_rate"],
               "return_per_query": policy.episode_return / max(s["n_queries"], 1),
               "loss": float(np.mean(learner.losses[n_losses:])) if len(learner.losses) > n_losses else
float("nan"),
               "n_agents": cfg["agents"]["n_agents"], "val_success": float("nan"),
               "val_time_ms": float("nan"), "val_return": float("nan"),
               "greedy_val_success": greedy_val[0], "greedy_val_time_ms": greedy_val[1],
               "greedy_val_return": greedy_val[2], "load_val_success": load_val[0],
               "load_val_time_ms": load_val[1], "load_val_return": load_val[2]}
        if (ep + 1) % dqn["eval_every"] == 0 or ep == episodes - 1:
            policy.epsilon, policy.learner = 0.0, None
            val_rate, val_time, val_ret = evaluate(policy, "agentroute", base)
            row["val_success"], row["val_time_ms"], row["val_return"] = val_rate, val_time, val_ret
            if val_ret > best_val:      # keep the weights with the best validation return
                best_val = val_ret
```

```
                policy.save(model_path, dqn["hidden"])
        rows.append(row)
        if verbose:
            print(f"[{kind}] ep {ep:3d} eps={eps:.2f} succ={s['success_rate']:.3f} "
                  f"ret/q={row['return_per_query']:.3f} loss={row['loss']:.4f} "
                  f"val={row['val_success']:.3f}/{row['val_time_ms']:.1f}ms "
                  f"(greedy {greedy_val[0]:.3f}/{greedy_val[2]:.3f}, load
{load_val[0]:.3f}/{load_val[2]:.3f}) "
                  f"[{time.time() - t_start:.0f}s]", flush=True)
        with open(out_csv, "w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
            w.writeheader()
            w.writerows(rows)
    return best_val, greedy_val
```

agentroute/embeddings.py

```python
"""Sentence embeddings for capability addresses and intents.

We use the pre-trained all-MiniLM-L6-v2 encoder (384 dimensions). Vectors are
L2-normalised, so cosine similarity is a plain dot product. Embeddings are
computed once and cached as .npy files under results/cache/.

Responsible: Bekir Emre Sarıpınar
"""
import hashlib

import numpy as np

from agentroute.config import RESULTS
from agentroute.data.capabilities import capability_cards, class_sentences, split_intents

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
DIM = 384
CACHE_DIR = RESULTS / "cache"

_models = {}


def get_model(name=MODEL_NAME):
    if name not in _models:
        from sentence_transformers import SentenceTransformer
        _models[name] = SentenceTransformer(name, device="cpu")
    return _models[name]


def embed_texts(texts, name=MODEL_NAME):
    """Return an (n, dim) float32 array of normalised embeddings, using the cache."""
    key = hashlib.sha1((name + "\n" + "\n".join(texts)).encode()).hexdigest()[:16]
    path = CACHE_DIR / f"emb_{key}.npy"
    if path.exists():
        return np.load(path)
    vecs = get_model(name).encode(list(texts), batch_size=64, normalize_embeddings=True,
                                  show_progress_bar=False)
    vecs = np.asarray(vecs, dtype=np.float32)
    CACHE_DIR.mkdir(parents=True, exist_ok=True)
    np.save(path, vecs)
    return vecs


def card_vector(sentence_vecs):
    """Capability address of one agent: normalised centroid of its card sentences."""
    v = sentence_vecs.mean(axis=0)
    return (v / np.linalg.norm(v)).astype(np.float32)


def agent_vectors(class_vecs, cls, rng, alpha=2.0):
    """Capability address of each agent (v2): its own random mix of its class's sentences.

    class_vecs has shape (n_classes, n_sentences, dim). The weights are drawn
    from a Dirichlet(alpha) distribution, so two providers of the same skill
    are close to each other but never identical.
    """
    n_sent = class_vecs.shape[1]
    weights = rng.dirichlet(alpha * np.ones(n_sent), size=len(cls))
    v = (weights[:, :, None] * class_vecs[cls]).sum(axis=1)
    return (v / np.linalg.norm(v, axis=1, keepdims=True)).astype(np.float32)


def load_dataset_vectors(split_seed=0):
    """Embed every capability card and every train/test intent.

    desc_* arrays have one row per (class, variant) card. desc_text is the
    card's description sentence (used as the Chord key and in plots).
    """
    cards = capability_cards()
    train, test = split_intents(seed=split_seed)
    data = {
        "desc_text": [c[2][0] for c in cards],
```

```
        "desc_card": [c[2] for c in cards],
        "desc_class": np.array([c[0] for c in cards]),
        "desc_variant": np.array([c[1] for c in cards]),
        "train_text": [t for _, t in train],
        "train_class": np.array([c for c, _ in train]),
        "test_text": [t for _, t in test],
        "test_class": np.array([c for c, _ in test]),
    }
    flat = [s for c in cards for s in c[2]]
    flat_vecs = embed_texts(flat)
    sizes = [len(c[2]) for c in cards]
    starts = np.cumsum([0] + sizes[:-1])
    data["desc_vecs"] = np.stack([card_vector(flat_vecs[s:s + n]) for s, n in zip(starts, sizes)])
    sentences = class_sentences()
    flat_class = embed_texts([s for row in sentences for s in row])
    data["class_vecs"] = flat_class.reshape(len(sentences), len(sentences[0]), -1)
    data["train_vecs"] = embed_texts(data["train_text"])
    data["test_vecs"] = embed_texts(data["test_text"])
    return data


def quantize(vec, header="float32"):
    """Simulate a compressed intent header.

    float32: 384 x 4 B = 1536 B (unchanged)
    int8:    384 x 1 B + 4 B scale = 388 B
    binary:  384 bits = 48 B (only the sign of each dimension is kept)
    """
    if header == "float32":
        return vec
    if header == "int8":
        scale = 127.0 / max(np.abs(vec).max(), 1e-8)
        q = np.round(vec * scale) / scale
    elif header == "binary":
        q = np.where(vec >= 0, 1.0, -1.0)
    else:
        raise ValueError(f"unknown header type {header}")
    q = q.astype(np.float32)
    return q / np.linalg.norm(q)


def header_bytes(header="float32", dim=DIM):
    return {"float32": dim * 4, "int8": dim + 4, "binary": dim // 8}[header]
```

agentroute/forwarding.py

```python
"""Intent-based forwarding: the decision of a single node.

This module does not know about SimPy. A policy gets the local state of a
node (NodeView) and a packet, and decides whether to hand the task to a local
agent (accept) or which neighbour to forward it to (choose). The same code can
therefore be used by the SimPy simulator, the Streamlit app or, later, a real
overlay implementation.

v2 adds: an acceptance rule that compares the local agent with what the S-FIB
knows, a load-aware greedy heuristic and an SPFR-style potential policy, and
reward bookkeeping for every policy so that all of them can be compared on
the same return.

Responsible: Bekir Emre Sarıpınar (v2 revision written with AI assistance)
"""
from dataclasses import dataclass, field

import numpy as np

DEFAULT_REWARD = {"hop_cost": 0.05, "delay_scale_s": 0.1, "deliver_bonus": 1.0, "drop_penalty": 1.0}
LOCAL = -1      # pseudo neighbour: "hand the task to the best local agent"


@dataclass
class Packet:
    pid: int
    intent: np.ndarray          # normalised intent embedding carried in the header
    intent_class: int           # ground truth, only used for evaluation
    ttl: int
    ttl_max: int
    src_agent: int
    created: float
    method: str
    prev_hop: int = None
    visited: list = field(default_factory=list)
    hops: int = 0
    text: str = ""              # request text, only used by the Chord baseline (exact match)
    pending: object = None      # last decision waiting for its reward
    oracle_best: bool = False   # the globally best agent is correct and passes tau
    oracle_any: bool = False    # some correct agent passes tau


@dataclass
class NodeView:
    """Everything a router knows locally when it forwards a packet."""
    node: int
    neighbors: list             # alive neighbours (router ids)
    sfib: object                # agentroute.sfib.SFIB of this router
    local_agents: list          # agent ids attached to this router
    local_vecs: np.ndarray      # their capability vectors, shape (len(local_agents), dim)
    queue_occ: dict             # neighbour -> queue occupancy in [0, 1]
    link_quality: dict          # neighbour -> 1 - loss probability
    link_delay: dict            # neighbour -> one-way delay in seconds
    in_handover: dict           # neighbour -> 1 if that gNB had a recent handover
    kind: dict                  # neighbour -> "gnb" | "core" | "cloud"
    upstream: int               # default route (nearest core router), may be None
    tau: float
    radius: int
    now: float = 0.0            # local clock when the packet is processed
    accept_mode: str = "best_known"     # "first" | "best_known"
    omega: float = 0.08         # attenuation per hop of a remote candidate


def best_local_agent(view, pkt):
    """Best local agent with cos >= tau, or None."""
    if len(view.local_agents) == 0:
        return None
    sims = view.local_vecs @ pkt.intent
    j = int(np.argmax(sims))
    if sims[j] >= view.tau:
        return view.local_agents[j]
    return None
```

```
def accept_rule(view, pkt):
    """Hand the task to a local agent, or return None to keep forwarding.

    "first" takes the first agent on the path that passes tau (v1). "best_known"
    also requires that no unvisited neighbour advertises a better candidate,
    after attenuating that candidate by its distance.
    """
    agent = best_local_agent(view, pkt)
    if agent is None or view.accept_mode == "first" or pkt.ttl <= 0:
        return agent
    fresh = [n for n in view.neighbors if n != pkt.prev_hop and n not in pkt.visited]
    if not fresh:
        return agent
    local = float(view.local_vecs[view.local_agents.index(agent)] @ pkt.intent)
    max_sim, _, hops = view.sfib.scores(pkt.intent)
    idx = [view.sfib.index[n] for n in fresh]
    potential = max_sim[idx] * np.exp(-view.omega * hops[idx])
    return agent if local >= potential.max() else None


FEATURES = ["max_sim", "mean_sim", "sim_gap", "above_tau", "hops_norm", "has_entry",
            "queue_occ", "link_quality", "link_delay", "in_handover", "ttl_ratio",
            "visited", "is_core", "is_cloud", "is_local"]
N_FEATURES = len(FEATURES)


def candidates(view, pkt):
    """Neighbours a packet may go to: never straight back to the previous hop."""
    cands = [n for n in view.neighbors if n != pkt.prev_hop]
    if not cands and pkt.prev_hop in view.neighbors:
        cands = [pkt.prev_hop]      # dead end: going back is the only option
    return cands


def neighbor_features(view, pkt, with_local=False):
    """Features of every action. Returns (actions, X with shape (len(actions), N_FEATURES)).

    An action is a neighbour id, or LOCAL ("deliver here") if with_local is
    set and a local agent passes tau.
    """
    cands = candidates(view, pkt)
    max_sim, mean_sim, hops = view.sfib.scores(pkt.intent)
    idx = [view.sfib.index[n] for n in cands]
    mx = max_sim[idx]
    best = mx.max() if len(mx) else -1.0
    local_row = None
    if with_local and best_local_agent(view, pkt) is not None:
        sims = view.local_vecs @ pkt.intent
        local_sim = float(sims.max())
        local_row = [local_sim, float(sims.mean()), local_sim - best, 1.0, 0.0, 1.0,
                     0.0, 1.0, 0.0, 0.0, pkt.ttl / pkt.ttl_max, 0.0, 0.0, 0.0, 1.0]
        best = max(best, local_sim)
    visited = set(pkt.visited)
    rows = [] if local_row is None else [local_row]
    for n, i in zip(cands, idx):
        has = max_sim[i] > -1.0
        rows.append([
            max_sim[i] if has else 0.0,
            mean_sim[i] if has else 0.0,
            (max_sim[i] - best) if has else -1.0,
            1.0 if max_sim[i] >= view.tau else 0.0,
            min(hops[i], view.radius + 1) / (view.radius + 1),
            1.0 if has else 0.0,
            view.queue_occ[n],
            view.link_quality[n],
            min(view.link_delay[n] / 0.02, 1.0),
            view.in_handover[n],
            pkt.ttl / pkt.ttl_max,
            1.0 if n in visited else 0.0,
            1.0 if view.kind[n] == "core" else 0.0,
            1.0 if view.kind[n] == "cloud" else 0.0,
            0.0,
        ])
    actions = ([] if local_row is None else [LOCAL]) + cands
    return actions, np.array(rows, dtype=np.float32).reshape(len(actions), N_FEATURES)
```

```
class Policy:
    """Interface. Every policy keeps the same return so that they can be compared."""
    name = "policy"

    def __init__(self, reward=None):
        self.reward = reward or DEFAULT_REWARD
        self.episode_return = 0.0

    def accept(self, view, pkt):
        agent = accept_rule(view, pkt)
        if agent is not None:
            pkt.pending = (None, None, view.now)
        return agent

    def choose(self, view, pkt):
        raise NotImplementedError

    def observe(self, pkt, now, next_view, outcome):
        """Called by the simulator when the result of the last decision is known.

        outcome is None when the packet reached the next router (next_view is
        that router's NodeView), "deliver" when it reached a correct agent, and
        "drop" for any failure (TTL, queue, link loss, wrong agent, ...).
        """
        if pkt.pending is None:
            return
        X, a, t0 = pkt.pending
        pkt.pending = None
        rw = self.reward
        r = -(rw["hop_cost"] + (now - t0) / rw["delay_scale_s"])
        if outcome is not None:
            r += rw["deliver_bonus"] if outcome == "deliver" else -rw["drop_penalty"]
        self.episode_return += r
        self.learn(X, a, r, pkt, next_view, outcome)

    def learn(self, X, a, r, pkt, next_view, outcome):
        pass


class GreedyPolicy(Policy):
    """Forward to the neighbour whose S-FIB summary is most similar to the intent.

    Unvisited neighbours are preferred (loop avoidance). If no neighbour has
    any S-FIB entry, the packet follows the default route towards the core.
    """
    name = "greedy"

    def _pool(self, view, pkt):
        cands = candidates(view, pkt)
        visited = set(pkt.visited)
        fresh = [n for n in cands if n not in visited]
        return fresh if fresh else cands

    def _default_route(self, view, pool):
        if view.upstream in pool:
            return view.upstream
        cores = [n for n in pool if view.kind[n] != "gnb"]
        return min(cores) if cores else min(pool)

    def choose(self, view, pkt):
        pool = self._pool(view, pkt)
        if not pool:
            return None
        pkt.pending = (None, None, view.now)
        max_sim, _, hops = view.sfib.scores(pkt.intent)
        best, best_key = None, None
        for n in pool:
            i = view.sfib.index[n]
            if max_sim[i] <= -1.0:
                continue
            key = (max_sim[i], -hops[i], -n)
            if best_key is None or key > best_key:
                best, best_key = n, key
        if best is not None:
            return best
```

```
        return self._default_route(view, pool)


class LoadAwareGreedy(GreedyPolicy):
    """Hand-written load balancing: among the neighbours within `band` of the
    best S-FIB score, take the least loaded one. This is the heuristic that a
    learned policy has to beat."""
    name = "greedy_load"

    def __init__(self, band=0.05, reward=None):
        super().__init__(reward)
        self.band = band

    def choose(self, view, pkt):
        pool = self._pool(view, pkt)
        if not pool:
            return None
        pkt.pending = (None, None, view.now)
        max_sim, _, hops = view.sfib.scores(pkt.intent)
        known = [n for n in pool if max_sim[view.sfib.index[n]] > -1.0]
        if not known:
            return self._default_route(view, pool)
        best = max(max_sim[view.sfib.index[n]] for n in known)
        near = [n for n in known if max_sim[view.sfib.index[n]] >= best - self.band]
        return min(near, key=lambda n: (round(view.queue_occ[n], 2), hops[view.sfib.index[n]], n))


class SPFRPolicy(GreedyPolicy):
    """SPFR-style forwarding (Qin et al., 2026), used with an exact S-FIB.

    Every visible executor is a potential source: phi = similarity * exp(-omega * hops),
    gated by tau. The task goes one hop towards the dominant executor and is
    executed locally if that executor is attached here. Executor load and
    price are not modelled in this simulator, so the utility is the similarity.
    If no executor is visible, the packet takes the default route to the core.
    """
    name = "spfr"

    def __init__(self, omega=0.08, reward=None):
        super().__init__(reward)
        self.omega = omega
        self._next = None

    def _dominant(self, view, pkt):
        """(potential, next hop) of the best remote executor, or (-1, None)."""
        mat, owner, hops = view.sfib.entries()
        if len(mat) == 0:
            return -1.0, None
        sims = mat @ pkt.intent
        alive = np.array([view.sfib.neighbors[i] in view.queue_occ for i in owner])
        ok = (sims >= view.tau) & alive
        if not ok.any():
            return -1.0, None
        phi = sims * np.exp(-self.omega * hops)
        phi[~ok] = -1.0
        j = int(np.argmax(phi))
        return float(phi[j]), view.sfib.neighbors[owner[j]]

    def accept(self, view, pkt):
        agent = best_local_agent(view, pkt)
        remote, nxt = self._dominant(view, pkt)
        self._next = nxt
        if agent is None:
            return None
        local = float(view.local_vecs[view.local_agents.index(agent)] @ pkt.intent)
        if local >= remote or pkt.ttl <= 0:
            pkt.pending = (None, None, view.now)
            return agent
        return None

    def choose(self, view, pkt):
        if self._next is not None:
            pkt.pending = (None, None, view.now)
            return self._next
        pool = self._pool(view, pkt)
        if not pool:
            return None
```

```
            pkt.pending = (None, None, view.now)
            return self._default_route(view, pool)
```

agentroute/sfib.py

```
"""Semantic FIB (S-FIB) and bounded-radius capability advertisements.

A normal FIB maps an IP prefix to an outgoing port. An S-FIB maps each
neighbour to a summary of the capabilities that can be reached through it.

v2: this is now a real distance-vector protocol on summaries. Every agent has
its own vector, so a router cannot keep one row per capability. What a router
u tells its neighbour w is at most k representative vectors (each with a hop
count and a weight), computed from u's own agents and from what u heard from
its other neighbours. Only those k vectors cross the link, so the size of a
message does not depend on the number of agents, and information is summarised
again at every hop. The "exact" mode keeps one row per agent, as SPFR does.

Advertisements stop after R hops. After a handover the agent's vector travels
as a withdraw or announce, and a router forwards it only if it moved one of
its representatives by more than a threshold. Small changes wait for the
periodic refresh, which recomputes every summary from the true state.

Responsible: Bekir Emre Sarıpınar (v2 revision written with AI assistance)
"""
from collections import deque

import numpy as np

from agentroute.embeddings import header_bytes, quantize

MSG_OVERHEAD = 40       # bytes of headers in every control message
ROW_META = 8            # hop count and weight next to each vector


def kmeans_summary(vecs, weights, hops, k, iters=10, medoid=False):
    """Weighted spherical k-means. Returns (reps, rep_hops, rep_weights).

    Initialisation is deterministic (heaviest vector first, then farthest
    point), so the same input always gives the same summary. With medoid=True
    each centre is replaced by the closest real input vector.
    """
    n = len(vecs)
    if n <= k:
        return vecs.copy(), hops.copy(), weights.copy()
    centers = [int(np.argmax(weights))]
    best_sim = vecs @ vecs[centers[0]]
    for _ in range(1, k):
        nxt = int(np.argmin(best_sim))
        centers.append(nxt)
        best_sim = np.maximum(best_sim, vecs @ vecs[nxt])
    reps = vecs[centers].copy()
    assign = None
    for _ in range(iters):
        new_assign = np.argmax(vecs @ reps.T, axis=1)
        if assign is not None and (new_assign == assign).all():
            break
        assign = new_assign
        c = np.zeros_like(reps)
        np.add.at(c, assign, vecs * weights[:, None])
        norm = np.linalg.norm(c, axis=1)
        ok = norm > 1e-9
        reps[ok] = c[ok] / norm[ok, None]
    sims = vecs @ reps.T
    assign = np.argmax(sims, axis=1)
    keep = np.unique(assign)
    if medoid:
        reps = np.stack([vecs[np.where(assign == j)[0][np.argmax(sims[assign == j, j])]] for j in
keep])
    else:
        reps = reps[keep]
    rep_hops = np.array([hops[assign == j].min() for j in keep])
    rep_weights = np.array([weights[assign == j].sum() for j in keep], dtype=np.float32)
    return reps.astype(np.float32), rep_hops, rep_weights


class SFIB:
```

```
    """S-FIB of a single router: per neighbour index, (reps, hops, weights)."""

    def __init__(self, node, neighbors, dim, radius, k, exact_hops=0):
        self.node = node
        self.neighbors = list(neighbors)
        self.index = {n: i for i, n in enumerate(self.neighbors)}
        self.dim = dim
        self.radius = radius
        self.k = k
        self.exact_hops = exact_hops
        self.rows = [self._empty() for _ in self.neighbors]
        self.exact = [{} for _ in self.neighbors]   # agent -> (vector, hops), exact mode only
        self._matrix = None

    def _empty(self):
        return (np.zeros((0, self.dim), np.float32), np.zeros(0, int), np.zeros(0, np.float32))

    def clear(self):
        self.rows = [self._empty() for _ in self.neighbors]
        self.exact = [{} for _ in self.neighbors]
        self._matrix = None

    def set_row(self, i, reps, hops, weights):
        self.rows[i] = (reps, hops, weights)
        self._matrix = None

    def set_from_exact(self, i):
        """Rebuild the row of neighbour i from its per-agent entries (exact mode)."""
        e = self.exact[i]
        if not e:
            self.rows[i] = self._empty()
        else:
            self.rows[i] = (np.stack([v for v, _ in e.values()]),
                            np.array([h for _, h in e.values()]),
                            np.ones(len(e), np.float32))
        self._matrix = None

    def n_vectors(self, neighbor):
        return len(self.rows[self.index[neighbor]][0])

    def apply_delta(self, i, vec, delta, hop, threshold):
        """One agent arrives (+1) or leaves (-1) behind neighbour i.

        Returns True if a representative moved by more than `threshold` (and
        changes it). False means the change is too small to be worth a message.
        """
        reps, hops, weights = self.rows[i]
        if len(reps) == 0:
            if delta < 0:
                return False
            self.set_row(i, vec[None, :].copy(), np.array([hop]), np.ones(1, np.float32))
            return True
        sims = reps @ vec
        j = int(np.argmax(sims))
        if delta > 0 and hop <= self.exact_hops:      # per-agent row of a nearby agent
            self.set_row(i, np.vstack([reps, vec]), np.append(hops, hop), np.append(weights, 1.0))
            return True
        if delta > 0 and len(reps) < self.k and sims[j] < 0.95:
            self.set_row(i, np.vstack([reps, vec]), np.append(hops, hop), np.append(weights, 1.0))
            return True
        new_weight = weights[j] + delta
        if new_weight <= 0:
            keep = np.arange(len(reps)) != j
            self.set_row(i, reps[keep], hops[keep], weights[keep])
            return True
        c = reps[j] * weights[j] + delta * vec
        norm = np.linalg.norm(c)
        if norm < 1e-6:
            return False
        c = c / norm
        if 1.0 - float(c @ reps[j]) <= threshold:
            return False
        reps = reps.copy()
        hops = hops.copy()
        weights = weights.copy()
        reps[j] = c
        weights[j] = new_weight
```

```
        if delta > 0:
            hops[j] = min(hops[j], hop)
        self.set_row(i, reps, hops, weights)
        return True

    def entries(self):
        """All rows of the table: (vectors, neighbour index of each, hops of each)."""
        if self._matrix is None:
            rows = [r for r in self.rows if len(r[0])]
            owner = [np.full(len(r[0]), i) for i, r in enumerate(self.rows) if len(r[0])]
            if rows:
                self._matrix = (np.vstack([r[0] for r in rows]), np.concatenate(owner),
                                np.concatenate([r[1] for r in rows]))
            else:
                self._matrix = (np.zeros((0, self.dim), np.float32), np.zeros(0, int), np.zeros(0,
int))
        return self._matrix

    def scores(self, intent):
        """Per neighbour index: (max cos, mean cos, hops of the best representative).

        Neighbours without any entry get max = mean = -1 and hops = radius + 1.
        """
        mat, owner, hops = self.entries()
        n = len(self.neighbors)
        max_sim = np.full(n, -1.0)
        mean_sim = np.full(n, -1.0)
        best_hops = np.full(n, self.radius + 1)
        if len(mat) == 0:
            return max_sim, mean_sim, best_hops
        sims = mat @ intent
        for i in np.unique(owner):
            sel = owner == i
            s = sims[sel]
            j = int(np.argmax(s))
            max_sim[i] = s[j]
            mean_sim[i] = s.mean()
            best_hops[i] = hops[sel][j]
        return max_sim, mean_sim, best_hops


class Advertiser:
    """Builds and maintains the S-FIBs of all routers and counts the control traffic."""

    def __init__(self, graph, dim, radius, summary="kmeans", k=8, change_threshold=0.01,
                 encoding="int8", exact_hops=1):
        self.graph = graph
        self.dim = dim
        self.radius = radius
        self.summary = summary
        self.k = 1 if summary == "mean" else k
        self.change_threshold = change_threshold
        self.encoding = encoding
        self.exact_hops = exact_hops    # agents this close keep their own row (two-tier table)
        self.alive = set(graph.nodes)
        self.tables = {u: SFIB(u, sorted(graph.neighbors(u)), dim, radius, self.k, exact_hops)
                       for u in graph.nodes}
        self.local = {u: {} for u in graph.nodes}       # router -> {agent id: vector}
        self.row_bytes = header_bytes(encoding, dim) + ROW_META
        self._dag_cache = {}

    def delta_bytes(self):
        """Size of one withdraw or announce message."""
        return MSG_OVERHEAD + self.row_bytes

    # ---- full rebuild ------------------------------------------------
    def _encode(self, reps):
        if self.encoding == "float32":
            return reps
        return np.stack([quantize(r, self.encoding) for r in reps]).astype(np.float32)

    def _advert(self, u, w):
        """The summary that router u sends to neighbour w, or None if it knows nothing."""
        vecs, hops, weights = [], [], []
        keep = None                     # rows that are sent as they are
        if self.local[u]:
            v = np.stack(list(self.local[u].values()))
```

```
            if self.exact_hops >= 1:
                keep = v
            else:
                vecs.append(v)
                hops.append(np.ones(len(v), int))
                weights.append(np.ones(len(v), np.float32))
        t = self.tables[u]
        for i, n in enumerate(t.neighbors):
            if n == w or n not in self.alive:
                continue
            reps, h, wt = t.rows[i]
            near = h + 1 <= self.radius
            if near.any():
                vecs.append(reps[near])
                hops.append(h[near] + 1)
                weights.append(wt[near])
        rows = []
        if keep is not None:
            rows.append((self._encode(keep), np.ones(len(keep), int), np.ones(len(keep), np.float32)))
        if vecs:
            reps, h, wt = kmeans_summary(np.vstack(vecs), np.concatenate(weights),
np.concatenate(hops),
                                         self.k, medoid=self.summary == "medoid")
            rows.append((self._encode(reps), h, wt))
        if not rows:
            return None
        return (np.vstack([r[0] for r in rows]), np.concatenate([r[1] for r in rows]),
                np.concatenate([r[2] for r in rows]))

    def _build_summaries(self):
        for t in self.tables.values():
            t.clear()
        for _ in range(self.radius):
            adverts = []
            for u in self.alive:
                for w in self.tables[u].neighbors:
                    if w in self.alive:
                        adv = self._advert(u, w)
                        if adv is not None:
                            adverts.append((u, w, adv))
            for u, w, (reps, h, wt) in adverts:
                t = self.tables[w]
                t.set_row(t.index[u], reps, h, wt)

    def dag(self, v):
        """All (d, u, n): router u is d hops from v and learns v's agents via neighbour n."""
        if v in self._dag_cache:
            return self._dag_cache[v]
        dist = {v: 0}
        queue = deque([v])
        while queue:
            x = queue.popleft()
            if dist[x] == self.radius:
                continue
            for y in self.graph.neighbors(x):
                if y in self.alive and y not in dist:
                    dist[y] = dist[x] + 1
                    queue.append(y)
        out = []
        for u, d in dist.items():
            if d == 0:
                continue
            for n in self.graph.neighbors(u):
                if n in dist and dist[n] == d - 1:
                    out.append((d, u, n))
        out.sort()
        self._dag_cache[v] = out
        return out

    def _build_exact(self):
        for t in self.tables.values():
            t.clear()
        for v, agents in self.local.items():
            if v not in self.alive or not agents:
                continue
            for d, u, n in self.dag(v):
                t = self.tables[u]
```

```
                i = t.index[n]
                for a, vec in agents.items():
                    t.exact[i][a] = (vec, d)
        for t in self.tables.values():
            for i in range(len(t.neighbors)):
                t.set_from_exact(i)

    def build(self):
        """Recompute every table from the true state. Returns (messages, bytes) of one full round."""
        if self.summary == "exact":
            self._build_exact()
        else:
            self._build_summaries()
        return self.round_cost()

    def round_cost(self):
        """One advertisement per (neighbour -> router) pair that has entries."""
        msgs, n_bytes = 0, 0
        for u in self.alive:
            t = self.tables[u]
            for i, n in enumerate(t.neighbors):
                if n in self.alive and len(t.rows[i][0]):
                    msgs += 1
                    n_bytes += MSG_OVERHEAD + len(t.rows[i][0]) * self.row_bytes
        return msgs, n_bytes

    # ---- incremental updates (handover) ----------------------------------
    def spread(self, v, frontier, seen, agent, vec, delta, d):
        """Move a withdraw/announce one hop further.

        Returns (changed links as (sender, receiver), next frontier).
        """
        changed = []
        if self.summary == "exact":
            for dist, u, n in self.dag(v):
                if dist != d:
                    continue
                t = self.tables[u]
                i = t.index[n]
                if delta > 0:
                    t.exact[i][agent] = (vec, d)
                else:
                    t.exact[i].pop(agent, None)
                t.set_from_exact(i)
                changed.append((n, u))
            return changed, frontier
        next_frontier = []
        for x in frontier:
            for w in self.graph.neighbors(x):
                if w in seen or w not in self.alive:
                    continue
                t = self.tables[w]
                if t.apply_delta(t.index[x], vec, delta, d, self.change_threshold):
                    changed.append((x, w))
                    seen.add(w)
                    next_frontier.append(w)
        return changed, next_frontier

    # ---- failures --------------------------------------------------------
    def fail(self, nodes):
        """Remove failed routers and rebuild. Returns (messages, bytes) of the changed adverts."""
        before = self._snapshot()
        self.alive -= set(nodes)
        for v in nodes:
            self.local[v] = {}
        self._dag_cache = {}
        self.build()
        after = self._snapshot()
        msgs, n_bytes = 0, 0
        for key, value in after.items():
            if before.get(key) != value[0]:
                msgs += 1
                n_bytes += MSG_OVERHEAD + value[1] * self.row_bytes
        return msgs, n_bytes

    def _snapshot(self):
        snap = {}
```

```
        for u in self.alive:
            t = self.tables[u]
            for i, n in enumerate(t.neighbors):
                reps = t.rows[i][0]
                if n in self.alive and len(reps):
                    snap[(n, u)] = (reps.tobytes(), len(reps))
        return {k: v for k, v in snap.items()}
```

agentroute/sim/events.py

```
"""JSONL event log and topology snapshot.

The simulator writes every event as one JSON line; the visualisations only
read these files. Example lines:
    {"t": 12.3, "type": "hop", "pkt": 7, "from": 3, "to": 5, "method": "agentroute"}
    {"t": 14.1, "type": "deliver", "pkt": 7, "agent": 42, "hops": 4, "method": "agentroute"}
    {"t": 15.0, "type": "drop", "pkt": 9, "at": 11, "reason": "ttl", "method": "greedy"}
    {"t": 20.0, "type": "handover", "agent": 42, "from_cell": 2, "to_cell": 3}
    {"t": 0.5, "type": "adv", "from": 3, "to": 5, "n_vectors": 4}

Responsible: Ömer Faruk Gedikli
"""
import json
from pathlib import Path


class EventLog:
    def __init__(self, log_dir=None):
        self.dir = Path(log_dir) if log_dir else None
        self.enabled = self.dir is not None
        self.lines = []

    def log(self, **event):
        if not self.enabled:
            return
        if "t" in event:
            event["t"] = round(float(event["t"]), 5)
        self.lines.append(json.dumps(event, separators=(",", ":")))

    def write_topology(self, snapshot):
        if not self.enabled:
            return
        self.dir.mkdir(parents=True, exist_ok=True)
        with open(self.dir / "topology.json", "w") as f:
            json.dump(snapshot, f)

    def close(self, summary=None):
        if not self.enabled:
            return
        self.dir.mkdir(parents=True, exist_ok=True)
        with open(self.dir / "events.jsonl", "w") as f:
            f.write("\n".join(self.lines))
            f.write("\n")
        if summary is not None:
            with open(self.dir / "summary.json", "w") as f:
                json.dump(summary, f, indent=2)
```

agentroute/sim/mobility.py

```python
"""Random-waypoint mobility and handover decisions.

Random waypoint: a node picks a random destination in the area, moves there in
a straight line at its own speed, pauses, and repeats (Camp et al., 2002).

Handover: a UE is attached to the nearest gNB, but it only switches when the
new gNB is clearly closer (hysteresis), which avoids ping-pong at cell edges.

Responsible: Ömer Faruk Gedikli
"""
import numpy as np


class RandomWaypoint:
    def __init__(self, n, bounds, speed, pause, rng):
        xmin, ymin, xmax, ymax = bounds
        self.low = np.array([xmin, ymin])
        self.high = np.array([xmax, ymax])
        self.rng = rng
        self.pos = rng.uniform(self.low, self.high, size=(n, 2))
        self.dest = rng.uniform(self.low, self.high, size=(n, 2))
        self.speed = speed * rng.uniform(0.5, 1.5, size=n)
        self.pause = pause
        self.pause_left = np.zeros(n)

    def step(self, dt):
        moving = self.pause_left <= 0
        self.pause_left[~moving] -= dt
        delta = self.dest - self.pos
        dist = np.linalg.norm(delta, axis=1)
        travel = self.speed * dt
        arrive = moving & (dist <= travel)
        go = moving & ~arrive
        self.pos[go] += delta[go] / dist[go, None] * travel[go, None]
        self.pos[arrive] = self.dest[arrive]
        n_arrive = int(arrive.sum())
        if n_arrive:
            self.pause_left[arrive] = self.pause
            self.dest[arrive] = self.rng.uniform(self.low, self.high, size=(n_arrive, 2))


def nearest_cell(points, cell_xy, alive=None):
    d = np.linalg.norm(points[:, None, :] - cell_xy[None, :, :], axis=2)
    if alive is not None:
        d[:, ~alive] = np.inf
    return np.argmin(d, axis=1)


def handover_targets(points, current, cell_xy, alive, hysteresis):
    """Return the cell each UE should be attached to after this step."""
    d = np.linalg.norm(points[:, None, :] - cell_xy[None, :, :], axis=2)
    d[:, ~alive] = np.inf
    best = np.argmin(d, axis=1)
    rows = np.arange(len(points))
    d_best = d[rows, best]
    d_cur = d[rows, current]
    switch = (best != current) & (d_best < (1.0 - hysteresis) * d_cur)
    return np.where(switch, best, current)
```

agentroute/sim/simulator.py

```
"""SimPy discrete-event simulator for AgentRoute and the baselines.

What is modelled
  - routers: one SimPy server each (service time + queue limit) plus bursty
    background traffic (M/M/1 waiting time; some routers are "hotspots"),
  - links: propagation delay and loss probability,
  - agents: fixed edge agents, cloud agents and random-waypoint mobile agents,
  - handovers with a short interruption, node failures with detection delay,
  - ten discovery methods: AgentRoute (DQN and GNN), greedy and load-aware
    greedy forwarding, SPFR-style forwarding, TTL-limited flooding, a central
    and a replicated registry, Chord and a semantic DHT.

Every method sees the same topology, mobility and query sequence for a given
seed, because each kind of randomness has its own random stream.

Responsible: Ömer Faruk Gedikli (v2 revision written with AI assistance)
"""
import json
import math
from collections import Counter

import networkx as nx
import numpy as np
import simpy

from agentroute.config import RESULTS, ROOT
from agentroute.data.capabilities import class_names
from agentroute.embeddings import agent_vectors, header_bytes, load_dataset_vectors, quantize
from agentroute.forwarding import GreedyPolicy, LoadAwareGreedy, NodeView, Packet, SPFRPolicy,
best_local_agent
from agentroute.sfib import MSG_OVERHEAD, Advertiser
from agentroute.sim.events import EventLog
from agentroute.sim.mobility import RandomWaypoint, handover_targets, nearest_cell
from agentroute.sim.topology import area_bounds, build_topology

METHODS = ["agentroute", "agentroute_gnn", "greedy", "greedy_load", "spfr",
           "flooding", "registry", "registry3", "chord", "sdht"]
SEMANTIC = ("agentroute", "agentroute_gnn", "greedy", "greedy_load", "spfr")
ADV_HOP_DELAY = 0.005   # an advertisement needs about 5 ms per hop
SMALL_MSG = 64          # bytes of a heartbeat, location update or answer

_DATA = None


def dataset():
    global _DATA
    if _DATA is None:
        _DATA = load_dataset_vectors()
    return _DATA


def load_tau(cfg):
    tau = cfg["forwarding"]["tau"]
    if tau is None:
        with open(RESULTS / "tau.json") as f:
            tau = json.load(f)["tau"]
    return float(tau)


def make_policy(method, cfg):
    if method == "greedy":
        return GreedyPolicy()
    if method == "greedy_load":
        return LoadAwareGreedy(cfg["forwarding"]["load_band"])
    if method == "spfr":
        return SPFRPolicy(cfg["forwarding"]["omega"])
    from agentroute.dqn.policy import DQNPolicy
    kind = "gnn" if method == "agentroute_gnn" else "mlp"
    key = "gnn_model_path" if kind == "gnn" else "model_path"
    return DQNPolicy.load(ROOT / cfg["dqn"][key], kind=kind)
```

```
class Router:
    def __init__(self, env, kind):
        self.kind = kind
        self.res = simpy.Resource(env, capacity=1)
        self.alive = True
        self.hotspot = False
        self.busy = False
        self.last_handover = -1e9
        self.occ = 0.0


class Simulator:
    def __init__(self, cfg, method, seed=None, policy=None, intents="test", log_dir=None):
        assert method in METHODS, method
        self.cfg = cfg
        self.method = method
        self.seed = cfg["seed"] if seed is None else seed
        self.semantic = method in SEMANTIC
        self.env = simpy.Environment()
        # separate random streams -> identical scenario for every method
        self.rng_topo = np.random.default_rng([self.seed, 1])
        self.rng_mob = np.random.default_rng([self.seed, 2])
        self.rng_traffic = np.random.default_rng([self.seed, 3])
        self.rng_net = np.random.default_rng([self.seed, 4])
        self.rng_fail = np.random.default_rng([self.seed, 5])

        sim, rt = cfg["sim"], cfg["routers"]
        self.duration = sim["duration_s"]
        self.warmup = sim["warmup_s"]
        self.load = sim["load"]
        self.tau = load_tau(cfg)
        self.accept_mode = cfg["forwarding"]["accept"]
        self.omega = cfg["forwarding"]["omega"]
        self.ttl = cfg["forwarding"]["ttl"]
        self.header = cfg["forwarding"]["header"]
        self.radius = cfg["sfib"]["radius"]
        self.service = rt["service_ms"] / 1000.0
        self.queue_limit = rt["queue_limit"]
        self.bg_service = rt["bg_service_ms"] / 1000.0
        self.bg_limit = rt["bg_queue_limit"]
        self.ho_window = cfg["sfib"]["handover_window_s"]

        data = dataset()
        self.card_vecs = data["desc_vecs"]
        self.card_class = data["desc_class"]
        self.class_vecs = data["class_vecs"]
        self.dim = self.card_vecs.shape[1]
        self.card_text = data["desc_text"]
        split = "test" if intents == "test" else "train"
        self.intent_vecs = data[f"{split}_vecs"]
        self.intent_class = data[f"{split}_class"]
        self.intent_text = data[f"{split}_text"]

        self.G = build_topology(cfg, self.rng_topo)
        self.kind = {u: self.G.nodes[u]["kind"] for u in self.G.nodes}
        self.gnbs = [u for u in self.G.nodes if self.kind[u] == "gnb"]
        self.cores = [u for u in self.G.nodes if self.kind[u] == "core"]
        self.cloud = next(u for u in self.G.nodes if self.kind[u] == "cloud")
        self.cell_xy = np.array([[self.G.nodes[u]["x"], self.G.nodes[u]["y"]] for u in self.gnbs])
        self.routers = {u: Router(self.env, self.kind[u]) for u in self.G.nodes}
        n_hot = int(round(rt["hotspot_fraction"] * (len(self.gnbs) + len(self.cores))))
        self.hotspots = sorted(int(u) for u in self.rng_topo.choice(self.gnbs + self.cores, n_hot,
replace=False))
        for u in self.hotspots:
            self.routers[u].hotspot = True
        self.known_dead = set()
        self.live = self.G.copy()
        self._paths = {}
        self._dist = {}
        self._compute_upstream()

        self.log = EventLog(log_dir)
        self.results = []
        self.ctrl_msgs = 0
        self.ctrl_bytes = 0
        self.query_msgs = 0
        self.query_bytes = 0
```

```
        self.query_size = header_bytes(self.header, self.dim) + MSG_OVERHEAD
        self.update_msgs = 0        # S-FIB messages triggered by handovers
        self.handovers = 0
        self._init_agents()

        self.policy = None
        self.adv = None
        if self.semantic:
            self.policy = policy if policy is not None else make_policy(method, cfg)
            sf = cfg["sfib"]
            summary = "exact" if method == "spfr" else sf["summary"]
            if method == "spfr":
                self.radius = cfg["baselines"]["spfr_radius"]
            self.adv = Advertiser(self.G, self.dim, self.radius, summary, sf["k"],
                                  sf["change_threshold"], sf["encoding"], sf["exact_hops"])
            for a in range(self.n_agents):
                if self.attached[a] >= 0:
                    self.adv.local[self.attached[a]][a] = self.agent_vec[a]
            self.adv.build()
        elif method == "flooding":
            from agentroute.baselines.flooding import Flooding
            self.flood = Flooding(self, cfg["baselines"]["flood_ttl"], cfg["baselines"]
["flood_window_s"])
        elif method in ("registry", "registry3"):
            from agentroute.baselines.registry import Registry
            replicas = 1 if method == "registry" else cfg["baselines"]["registry3_replicas"]
            self.registry = Registry(self, cfg["baselines"], replicas)
        elif method == "chord":
            from agentroute.baselines.chord import Chord
            self.chord = Chord(self, cfg["baselines"]["chord_bits"])
        elif method == "sdht":
            from agentroute.baselines.chord import SemanticDHT
            self.chord = SemanticDHT(self, cfg["baselines"])
        self.log.write_topology(self.snapshot())

    # ------------------------------------------------------------------
    # agents
    def _init_agents(self):
        ag = self.cfg["agents"]
        n = ag["n_agents"]
        n_cloud = int(round(n * ag["cloud_fraction"]))
        n_mobile = int(round(n * ag["mobile_fraction"]))
        n_edge = n - n_cloud - n_mobile
        self.n_agents = n
        self.agent_kind = np.array(["edge"] * n_edge + ["mobile"] * n_mobile + ["cloud"] * n_cloud)
        # capability supply is long-tailed: a few classes are offered by many
        # agents, most by few (Zipf over a random ranking of the classes)
        n_cls = len(class_names())
        rank = self.rng_topo.permutation(n_cls)
        weights = 1.0 / (np.arange(1, n_cls + 1) ** ag["class_zipf"])
        cls = rank[self.rng_topo.choice(n_cls, size=n, p=weights / weights.sum())]
        var = self.rng_topo.integers(2, size=n)
        self.agent_card = 2 * cls + var
        assert (self.card_class[self.agent_card] == cls).all()
        self.agent_class = cls
        self.agent_vec = agent_vectors(self.class_vecs, cls, self.rng_topo, ag["card_alpha"])
        bounds = area_bounds(self.G)
        self.agent_xy = np.zeros((n, 2))
        self.agent_xy[:n_edge] = self.rng_topo.uniform(bounds[:2], bounds[2:], size=(n_edge, 2))
        self.agent_xy[n_edge + n_mobile:] = [self.G.nodes[self.cloud]["x"], self.G.nodes[self.cloud]
["y"]]
        self.mobile_idx = np.arange(n_edge, n_edge + n_mobile)
        self.rwp = RandomWaypoint(n_mobile, bounds, ag["speed_mps"], ag["pause_s"], self.rng_mob)
        self.agent_xy[self.mobile_idx] = self.rwp.pos
        self.attached = np.full(n, -1)
        self.down = np.zeros(n, dtype=bool)
        self.ho_pending = np.zeros(n, dtype=bool)
        self.at_router = {u: [] for u in self.G.nodes}
        cells = nearest_cell(self.agent_xy[:n_edge + n_mobile], self.cell_xy)
        for a in range(n_edge + n_mobile):
            self._attach(a, self.gnbs[cells[a]])
        for a in range(n_edge + n_mobile, n):
            self._attach(a, self.cloud)
        self.requesters = np.arange(n_edge + n_mobile)  # cloud agents only serve

    def _attach(self, a, u):
        self.attached[a] = u
```

```
        self.at_router[u].append(int(a))

    def _detach(self, a):
        u = self.attached[a]
        if u >= 0:
            self.at_router[u].remove(int(a))
        self.attached[a] = -1
        return u

    # ------------------------------------------------------------------
    # routers and links
    def rho(self, r):
        base = self.cfg["routers"]["rho_busy"] if r.busy else self.cfg["routers"]["rho_base"]
        return min(0.97, base * self.load)

    def transit(self, u):
        """Queue and process a packet at router u. Returns a drop reason or None."""
        r = self.routers[u]
        if not r.alive:
            return "node_down"
        if len(r.res.queue) >= self.queue_limit:
            return "queue"
        with r.res.request() as req:
            yield req
            yield self.env.timeout(self.service)
        # waiting behind background traffic: number in system of an M/M/1 queue
        rho = self.rho(r)
        n_bg = int(math.log(1.0 - self.rng_net.random()) / math.log(rho))
        if n_bg >= self.bg_limit:
            return "queue"
        if n_bg > 0:
            yield self.env.timeout(self.rng_net.gamma(n_bg, self.bg_service))
        return None

    def link(self, u, v):
        e = self.G.edges[u, v]
        yield self.env.timeout(e["delay"])
        if self.rng_net.random() < e["loss"]:
            return "link"
        if not self.routers[v].alive:
            return "node_down"
        return None

    def radio(self):
        topo = self.cfg["topology"]
        yield self.env.timeout(topo["delay_ms"]["radio"] / 1000.0)
        if self.rng_net.random() < topo["loss"]["radio"]:
            return "link"
        return None

    def count_query(self, msgs=1, size=None):
        self.query_msgs += msgs
        self.query_bytes += msgs * (self.query_size if size is None else size)

    def count_ctrl(self, msgs, n_bytes):
        self.ctrl_msgs += msgs
        self.ctrl_bytes += n_bytes

    def traverse(self, pkt, path, log_hops=True, size=None):
        """Send a packet along a router path (used by the baselines).

        Returns (drop reason or None, router where it stopped)."""
        if not path:
            return "no_route", None     # failures partitioned the network
        for u, v in zip(path, path[1:]):
            reason = yield from self.transit(u)
            if reason:
                return reason, u
            self.count_query(1, size)
            pkt.hops += 1
            if log_hops:
                self.log.log(t=self.env.now, type="hop", pkt=pkt.pid, method=self.method,
                             **{"from": int(u), "to": int(v)})
            reason = yield from self.link(u, v)
            if reason:
                return reason, v
        reason = yield from self.transit(path[-1])
```

```
        return reason, path[-1]

    def hop_distance(self, a):
        """Hop count from router a to every router of the live graph (cached)."""
        if a not in self._dist:
            self._dist[a] = nx.single_source_shortest_path_length(self.live, a)
        return self._dist[a]

    def path(self, a, b):
        if a not in self._paths:
            self._paths[a] = nx.single_source_dijkstra_path(self.live, a, weight="delay")
        return self._paths[a].get(b)

    def _compute_upstream(self):
        self.upstream = {}
        for u in self.G.nodes:
            ups = [n for n in self.G.adj[u] if self.kind[n] != "gnb" and n not in self.known_dead]
            if self.kind[u] == "gnb" and ups:
                self.upstream[u] = min(ups, key=lambda n: (self.G.edges[u, n]["delay"], n))
            else:
                self.upstream[u] = None

    def view(self, u, pkt):
        now = self.env.now
        nbrs = [n for n in self.G.adj[u] if n not in self.known_dead]
        ids = [a for a in self.at_router[u] if a != pkt.src_agent]
        vecs = self.agent_vec[ids] if ids else np.zeros((0, self.card_vecs.shape[1]), np.float32)
        return NodeView(
            node=u, neighbors=nbrs, sfib=self.adv.tables[u] if self.adv else None,
            local_agents=ids, local_vecs=vecs,
            queue_occ={n: self.routers[n].occ for n in nbrs},
            link_quality={n: 1.0 - self.G.edges[u, n]["loss"] for n in nbrs},
            link_delay={n: self.G.edges[u, n]["delay"] for n in nbrs},
            in_handover={n: 1.0 if now - self.routers[n].last_handover < self.ho_window else 0.0
                         for n in nbrs},
            kind=self.kind, upstream=self.upstream.get(u), tau=self.tau, radius=self.radius,
            now=now, accept_mode=self.accept_mode, omega=self.omega)

    # ------------------------------------------------------------------
    # results
    def finish(self, pkt, success, reason, at=None, agent=None, time=None):
        now = self.env.now
        elapsed = now - pkt.created if time is None else time
        self.results.append({"pid": pkt.pid, "success": success, "reason": reason,
                             "time": elapsed, "hops": pkt.hops, "cls": pkt.intent_class,
                             "oracle_best": pkt.oracle_best, "oracle_any": pkt.oracle_any})
        if success:
            self.log.log(t=now, type="deliver", pkt=pkt.pid, agent=int(agent), hops=pkt.hops,
                         method=self.method)
        else:
            self.log.log(t=now, type="drop", pkt=pkt.pid, at=None if at is None else int(at),
                         reason=reason, method=self.method)

    def deliver_to(self, pkt, u, agent):
        """Last radio hop from router u to the chosen agent. Returns a drop reason or None."""
        reason = yield from self.radio()
        if reason:
            return reason
        if self.attached[agent] != u:
            return "handover"
        if self.agent_class[agent] != pkt.intent_class:
            return "wrong_agent"
        return None

    # ------------------------------------------------------------------
    # semantic forwarding (AgentRoute and greedy)
    def semantic_query(self, pkt, start):
        reason = yield from self.radio()
        if reason:
            return self.finish(pkt, False, reason, at=start)
        u = start
        while True:
            reason = yield from self.transit(u)
            if reason:
                self.policy.observe(pkt, self.env.now, None, "drop")
                return self.finish(pkt, False, reason, at=u)
            view = self.view(u, pkt)
```

```
            if pkt.ttl <= 0 and best_local_agent(view, pkt) is None:
                self.policy.observe(pkt, self.env.now, None, "drop")
                return self.finish(pkt, False, "ttl", at=u)
            if pkt.pending is not None:
                self.policy.observe(pkt, self.env.now, view, None)
            agent = self.policy.accept(view, pkt)
            if agent is not None:
                reason = yield from self.deliver_to(pkt, u, agent)
                self.policy.observe(pkt, self.env.now, None, "drop" if reason else "deliver")
                if reason:
                    return self.finish(pkt, False, reason, at=u)
                return self.finish(pkt, True, None, agent=agent)
            nxt = self.policy.choose(view, pkt)
            if nxt is None:
                self.policy.observe(pkt, self.env.now, None, "drop")
                return self.finish(pkt, False, "no_route", at=u)
            pkt.ttl -= 1
            pkt.prev_hop = u
            pkt.visited.append(u)
            pkt.hops += 1
            self.count_query()
            self.log.log(t=self.env.now, type="hop", pkt=pkt.pid, method=self.method,
                         **{"from": int(u), "to": int(nxt)})
            reason = yield from self.link(u, nxt)
            if reason:
                self.policy.observe(pkt, self.env.now, None, "drop")
                return self.finish(pkt, False, reason, at=nxt)
            u = nxt

    def propagate(self, v, agent, delta):
        """Withdraw (delta=-1) or announce (+1) one agent at router v, hop by hop."""
        if v not in self.adv.alive:
            return
        vec = self.agent_vec[agent]
        if delta > 0:
            self.adv.local[v][agent] = vec
        else:
            self.adv.local[v].pop(agent, None)
        kind = "adv" if delta > 0 else "withdraw"
        frontier, seen = [v], {v}
        for d in range(1, self.radius + 1):
            yield self.env.timeout(ADV_HOP_DELAY)
            changed, frontier = self.adv.spread(v, frontier, seen, agent, vec, delta, d)
            self.count_ctrl(len(changed), len(changed) * self.adv.delta_bytes())
            self.update_msgs += len(changed)
            if self.log.enabled:
                for n, u in changed:
                    nv = self.adv.tables[u].n_vectors(n) if kind == "adv" else 0
                    self.log.log(t=self.env.now, type=kind, agent=int(agent), n_vectors=nv,
                                 **{"from": int(n), "to": int(u)})
            if not changed and self.adv.summary != "exact":
                break

    # ------------------------------------------------------------------
    # processes
    def query_source(self):
        rate = self.cfg["sim"]["query_rate"] * self.load
        yield self.env.timeout(self.warmup)
        pid = 0
        while True:
            yield self.env.timeout(self.rng_traffic.exponential(1.0 / rate))
            if self.env.now >= self.duration:
                return
            a = int(self.rng_traffic.choice(self.requesters))
            k = int(self.rng_traffic.integers(len(self.intent_class)))
            if self.attached[a] < 0:
                continue        # requester is in the middle of a handover
            start = int(self.attached[a])
            pkt = Packet(pid=pid, intent=quantize(self.intent_vecs[k], self.header),
                         intent_class=int(self.intent_class[k]), ttl=self.ttl, ttl_max=self.ttl,
                         src_agent=a, created=self.env.now, method=self.method,
                         text=self.intent_text[k])
            self._set_oracle(pkt, a)
            self.log.log(t=self.env.now, type="query", pkt=pid, agent=a, router=start,
                         cls=pkt.intent_class, text=pkt.text, method=self.method)
            pid += 1
            if self.semantic:
```

```
                self.env.process(self.semantic_query(pkt, start))
            elif self.method == "flooding":
                self.env.process(self.flood.query(pkt, start))
            elif self.method in ("registry", "registry3"):
                self.env.process(self.registry.query(pkt, start))
            else:
                self.env.process(self.chord.query(pkt, start))

    def _set_oracle(self, pkt, src):
        """What a method with perfect knowledge of all attached agents could achieve."""
        on = self.attached >= 0
        on[src] = False
        if not on.any():
            return
        sims = self.agent_vec @ pkt.intent
        sims[~on] = -2.0
        best = int(np.argmax(sims))
        right = self.agent_class == pkt.intent_class
        pkt.oracle_best = bool(sims[best] >= self.tau and right[best])
        pkt.oracle_any = bool(((sims >= self.tau) & right).any())

    def mobility(self):
        ag = self.cfg["agents"]
        dt = ag["mobility_dt_s"]
        mob = self.mobile_idx
        if len(mob) == 0:
            return
        while True:
            yield self.env.timeout(dt)
            self.rwp.step(dt)
            self.agent_xy[mob] = self.rwp.pos
            if self.log.enabled:
                for a in mob:
                    self.log.log(t=self.env.now, type="move", agent=int(a),
                                 x=round(float(self.agent_xy[a, 0]), 1),
                                 y=round(float(self.agent_xy[a, 1]), 1))
            alive = np.array([self.routers[u].alive for u in self.gnbs])
            cur = self.attached[mob]
            cur_cell = np.where(cur >= 0, cur, 0)
            target = handover_targets(self.rwp.pos, cur_cell, self.cell_xy, alive, ag["hysteresis"])
            nearest = nearest_cell(self.rwp.pos, self.cell_xy, alive)
            for k in np.nonzero((target != cur) | (cur < 0))[0]:
                a = mob[k]
                if self.ho_pending[a]:
                    continue
                self.ho_pending[a] = True
                if cur[k] < 0:
                    self.env.process(self.handover(a, None, self.gnbs[nearest[k]]))
                else:
                    self.env.process(self.handover(a, int(cur[k]), self.gnbs[target[k]]))

    def handover(self, a, old, new):
        now = self.env.now
        if old is not None:
            self._detach(a)
            self.routers[old].last_handover = now
            self.handovers += 1
            self.log.log(t=now, type="handover", agent=int(a), from_cell=int(old), to_cell=int(new))
            if self.semantic:
                self.env.process(self.propagate(old, a, -1))
            elif self.method in ("chord", "sdht"):
                self.chord.on_move(a)
        self.routers[new].last_handover = now
        yield self.env.timeout(self.cfg["agents"]["handover_interrupt_s"])
        self.ho_pending[a] = False
        if not self.routers[new].alive:
            return              # the target died meanwhile; mobility will try again
        self._attach(a, new)
        if self.semantic:
            self.env.process(self.propagate(new, a, +1))
        elif self.method in ("registry", "registry3"):
            self.env.process(self.registry.update(a))
        elif self.method in ("chord", "sdht"):
            self.chord.on_move(a)

    def hello(self):
        """Neighbours learn each other's queue occupancy every hello period."""
```

```
        hello_s = self.cfg["routers"]["hello_s"]
        while True:
            for r in self.routers.values():
                rho = self.rho(r)
                bg = min(1.0, rho / (1.0 - rho) / self.bg_limit)
                r.occ = max(len(r.res.queue) / self.queue_limit, bg)
            yield self.env.timeout(hello_s)

    def hotspot(self, u):
        r = self.routers[u]
        r.busy = bool(self.rng_topo.random() < 0.5)
        mean = self.cfg["routers"]["busy_mean_s"]
        while True:
            self.log.log(t=self.env.now, type="load", node=int(u), busy=r.busy)
            yield self.env.timeout(self.rng_topo.exponential(mean))
            r.busy = not r.busy

    def refresh(self):
        """Periodic soft-state refresh of the control plane."""
        period = self.cfg["sfib"]["refresh_s"]
        first = True
        while True:
            if self.semantic:
                msgs, n_bytes = self.adv.build()    # recompute every summary (soft state)
                n_reg = int((self.attached >= 0).sum())
                self.count_ctrl(msgs + n_reg, n_bytes + n_reg * SMALL_MSG)
                if first and self.log.enabled:
                    for u in self.adv.alive:
                        t = self.adv.tables[u]
                        for n in t.neighbors:
                            nv = t.n_vectors(n)
                            if nv:
                                self.log.log(t=0.0, type="adv", n_vectors=nv,
                                             **{"from": int(n), "to": int(u)})
            elif self.method in ("registry", "registry3"):
                self.env.process(self.registry.refresh(period, initial=first))
            elif self.method in ("chord", "sdht"):
                self.chord.refresh()
            first = False
            yield self.env.timeout(period)

    def failures(self):
        frac = self.cfg["sim"]["failure_fraction"]
        if frac <= 0:
            return
        yield self.env.timeout(self.cfg["sim"]["failure_time_frac"] * self.duration)
        pool = self.gnbs + self.cores
        k = int(round(frac * len(pool)))
        failed = [int(u) for u in self.rng_fail.choice(pool, size=k, replace=False)]
        for u in failed:
            self.routers[u].alive = False
            self.log.log(t=self.env.now, type="fail", node=u)
            for a in list(self.at_router[u]):
                if self.agent_kind[a] == "edge":
                    self._detach(a)
                    self.down[a] = True
        yield self.env.timeout(self.cfg["sfib"]["failure_detect_s"])
        self.known_dead |= set(failed)
        self.live = self.G.subgraph([u for u in self.G.nodes if u not in self.known_dead]).copy()
        self._paths = {}
        self._compute_upstream()
        self._dist = {}
        if self.semantic:
            self.count_ctrl(*self.adv.fail(failed))
        elif self.method in ("chord", "sdht"):
            self.chord.rebuild()

    # ------------------------------------------------------------------
    def run(self):
        self.env.process(self.query_source())
        self.env.process(self.mobility())
        self.env.process(self.hello())
        self.env.process(self.refresh())
        self.env.process(self.failures())
        for u in self.hotspots:
            self.env.process(self.hotspot(u))
        self.env.run(until=self.duration + 3.0)
```

```
        summary = self.summary()
        self.log.close(summary)
        return summary

    def summary(self):
        res = self.results
        n = len(res)
        ok = [r for r in res if r["success"]]
        reasons = Counter(r["reason"] for r in res if not r["success"])
        times = np.array([r["time"] for r in ok]) * 1000.0
        out = {
            "method": self.method,
            "seed": self.seed,
            "n_agents": self.n_agents,
            "n_routers": self.G.number_of_nodes(),
            "n_queries": n,
            "success_rate": len(ok) / n if n else 0.0,
            "time_ms": float(times.mean()) if len(ok) else float("nan"),
            "time_ms_p95": float(np.percentile(times, 95)) if len(ok) else float("nan"),
            "hops": float(np.mean([r["hops"] for r in ok])) if ok else float("nan"),
            "ctrl_msgs": self.ctrl_msgs,
            "query_msgs": self.query_msgs,
            "msgs_per_query": (self.ctrl_msgs + self.query_msgs) / max(n, 1),
            "ctrl_msgs_per_s": self.ctrl_msgs / self.duration,
            "ctrl_kB_per_s": self.ctrl_bytes / self.duration / 1000.0,
            "query_msgs_per_query": self.query_msgs / max(n, 1),
            "query_kB_per_query": self.query_bytes / max(n, 1) / 1000.0,
            "oracle_best": float(np.mean([r["oracle_best"] for r in res])) if n else 0.0,
            "oracle_any": float(np.mean([r["oracle_any"] for r in res])) if n else 0.0,
            "success_solvable": float(np.mean([r["success"] for r in res if r["oracle_best"]]))
            if any(r["oracle_best"] for r in res) else float("nan"),
            "precision": len(ok) / max(len(ok) + reasons.get("wrong_agent", 0), 1),
            "handovers": self.handovers,
            "update_msgs": self.update_msgs,
        }
        if self.method in ("registry", "registry3"):
            out["servers_alive"] = sum(self.routers[u].alive for u in self.registry.nodes)
        for reason in ["ttl", "queue", "link", "node_down", "no_route", "wrong_agent", "handover",
                       "not_found", "stale_location", "registry_queue"]:
            out[f"drop_{reason}"] = reasons.get(reason, 0) / n if n else 0.0
        return out

    def snapshot(self):
        names = class_names()
        return {
            "meta": {"method": self.method, "seed": self.seed, "n_agents": self.n_agents,
                     "radius": self.radius, "tau": self.tau, "duration": self.duration,
                     "classes": names},
            "nodes": [{"id": int(u), "kind": self.kind[u], "x": self.G.nodes[u]["x"],
                       "y": self.G.nodes[u]["y"], "hotspot": u in self.hotspots}
                      for u in self.G.nodes],
            "links": [{"u": int(u), "v": int(v), "kind": d["kind"],
                       "delay_ms": d["delay"] * 1000.0, "loss": d["loss"]}
                      for u, v, d in self.G.edges(data=True)],
            "agents": [{"id": a, "kind": str(self.agent_kind[a]), "cls": int(self.agent_class[a]),
                        "class_name": names[self.agent_class[a]],
                        "card": self.card_text[self.agent_card[a]],
                        "x": round(float(self.agent_xy[a, 0]), 1),
                        "y": round(float(self.agent_xy[a, 1]), 1),
                        "router": int(self.attached[a])}
                       for a in range(self.n_agents)],
        }
```

agentroute/sim/topology.py

```
"""Mobile cellular / edge topology.

Layout
  - gNB + edge routers on a square grid of cells, neighbouring gNBs joined by
    Xn links,
  - core routers on a coarser grid, joined to each other in a grid mesh,
  - every gNB has a backhaul link to its nearest core router; half of them
    also to the second nearest (redundancy for the failure experiments),
  - one cloud node attached to two core routers.

The number of cells grows with the number of agents (about 40 agents per
cell), so the 10,000-agent scenario has 256 cells and 36 core routers.

Responsible: Ömer Faruk Gedikli
"""
import math

import networkx as nx
import numpy as np


def grid_side(n_agents, topo):
    cells = math.ceil(n_agents / topo["agents_per_cell"])
    cells = min(max(cells, topo["min_cells"]), topo["max_cells"])
    return math.ceil(math.sqrt(cells))


def _add_link(g, u, v, kind, topo):
    loss = topo["loss"]["radio"] if kind == "radio" else topo["loss"]["wired"]
    g.add_edge(u, v, kind=kind, delay=topo["delay_ms"][kind] / 1000.0, loss=loss)


def build_topology(cfg, rng):
    topo = cfg["topology"]
    side = grid_side(cfg["agents"]["n_agents"], topo)
    spacing = topo["cell_spacing_m"]
    g = nx.Graph()
    g.graph["side"] = side
    g.graph["spacing"] = spacing

    # gNB + edge routers
    for i in range(side):
        for j in range(side):
            g.add_node(i * side + j, kind="gnb", x=float(j * spacing), y=float(i * spacing))
    n_cells = side * side
    for i in range(side):
        for j in range(side):
            u = i * side + j
            if j + 1 < side:
                _add_link(g, u, u + 1, "xn", topo)
            if i + 1 < side:
                _add_link(g, u, u + side, "xn", topo)

    # core routers on a coarse grid over the same area
    core_side = max(2, round(math.sqrt(n_cells / topo["cells_per_core"])))
    extent = (side - 1) * spacing
    cores = []
    for i in range(core_side):
        for j in range(core_side):
            cid = n_cells + len(cores)
            x = (j + 0.5) * extent / core_side if side > 1 else 0.0
            y = (i + 0.5) * extent / core_side if side > 1 else 0.0
            g.add_node(cid, kind="core", x=float(x), y=float(y))
            cores.append(cid)
    for i in range(core_side):
        for j in range(core_side):
            c = cores[i * core_side + j]
            if j + 1 < core_side:
                _add_link(g, c, cores[i * core_side + j + 1], "core", topo)
            if i + 1 < core_side:
                _add_link(g, c, cores[(i + 1) * core_side + j], "core", topo)
```

```
    # backhaul: nearest core, plus the second nearest for a checkerboard half of the cells
    core_xy = np.array([[g.nodes[c]["x"], g.nodes[c]["y"]] for c in cores])
    for u in range(n_cells):
        xy = np.array([g.nodes[u]["x"], g.nodes[u]["y"]])
        order = np.argsort(np.linalg.norm(core_xy - xy, axis=1), kind="stable")
        _add_link(g, u, cores[order[0]], "backhaul", topo)
        i, j = divmod(u, side)
        if (i + j) % 2 == 0:
            _add_link(g, u, cores[order[1]], "backhaul", topo)

    # cloud data centre, attached to two opposite core routers
    cloud = n_cells + len(cores)
    g.add_node(cloud, kind="cloud", x=extent / 2, y=-1.2 * spacing)
    _add_link(g, cloud, cores[0], "cloud", topo)
    _add_link(g, cloud, cores[-1], "cloud", topo)

    # a few bad wired links with higher loss
    wired = [(u, v) for u, v, d in g.edges(data=True) if d["kind"] in ("xn", "backhaul", "core")]
    n_bad = int(round(topo["bad_link_fraction"] * len(wired)))
    for idx in rng.choice(len(wired), size=n_bad, replace=False):
        u, v = wired[idx]
        g.edges[u, v]["loss"] = topo["loss"]["bad"]
    return g


def area_bounds(g):
    """Rectangle covered by the cells: (xmin, ymin, xmax, ymax)."""
    side, spacing = g.graph["side"], g.graph["spacing"]
    half = spacing / 2
    return -half, -half, (side - 1) * spacing + half, (side - 1) * spacing + half
```

experiments/common.py

```python
"""Shared helpers for the experiment scripts.

A sweep runs every (parameter value, method, seed) combination in parallel
worker processes and writes one CSV row per run to results/raw/<name>.csv.

Written with AI assistance (experiment tooling).
"""
import argparse
import copy
import sys
import time
from multiprocessing import Pool
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

import pandas as pd  # noqa: E402

from agentroute.config import RESULTS, deep_update, load_config, set_by_path  # noqa: E402

ALL_METHODS = ["agentroute", "agentroute_gnn", "greedy", "greedy_load", "spfr",
               "flooding", "registry", "registry3", "chord", "sdht"]
SEMANTIC_METHODS = ["agentroute", "agentroute_gnn", "greedy", "greedy_load", "spfr"]
SEEDS = list(range(1, 11))


def parse_args(description):
    p = argparse.ArgumentParser(description=description)
    p.add_argument("--quick", action="store_true", help="few values, one seed, short runs")
    p.add_argument("--workers", type=int, default=6)
    p.add_argument("--config", default=None)
    return p.parse_args()


def _run_one(job):
    import torch
    torch.set_num_threads(1)
    from agentroute.sim.simulator import Simulator
    cfg, method, seed, extra = job
    t0 = time.time()
    row = Simulator(cfg, method, seed=seed).run()
    row.update(extra)
    row["wall_s"] = round(time.time() - t0, 2)
    return row


def sweep(name, values, apply, methods, seeds, quick=False, workers=6, config=None,
          base=None, label="value"):
    """Run a parameter sweep.

    values  : list of parameter values
    apply   : dotted config key ("sfib.radius") or a function value -> overrides dict
    """
    jobs = []
    for v in values:
        for m in methods:
            for seed in seeds:
                cfg = load_config(config, base)
                if quick:
                    cfg["sim"]["duration_s"] = min(cfg["sim"]["duration_s"], 15.0)
                if callable(apply):
                    deep_update(cfg, apply(v))
                else:
                    set_by_path(cfg, apply, v)
                jobs.append((cfg, m, seed, {"experiment": name, label: v}))
    # heaviest runs first so that the workers stay busy until the end
    jobs.sort(key=lambda j: -j[0]["agents"]["n_agents"])
    t0 = time.time()
    rows = []
    with Pool(workers) as pool:
        for i, row in enumerate(pool.imap_unordered(_run_one, jobs), 1):
```

```
            rows.append(row)
            if i % 10 == 0 or i == len(jobs):
                print(f"[{name}] {i}/{len(jobs)} runs done ({time.time() - t0:.0f}s)", flush=True)
    df = pd.DataFrame(rows).sort_values([label, "method", "seed"])
    # quick runs never overwrite the full results that the report uses
    out = RESULTS / "raw" / (f"{name}_quick.csv" if quick else f"{name}.csv")
    out.parent.mkdir(parents=True, exist_ok=True)
    df.to_csv(out, index=False)
    print(f"[{name}] wrote {out}")
    return df


def seeds_for(quick):
    return [1] if quick else SEEDS


def describe(df, label="value", cols=("success_rate", "time_ms", "hops", "msgs_per_query")):
    """Print a compact mean table (used at the end of each script)."""
    table = df.groupby([label, "method"])[list(cols)].mean().round(3)
    print(table.to_string())
    return table


__all__ = ["ALL_METHODS", "SEMANTIC_METHODS", "parse_args", "sweep", "seeds_for", "describe", "copy"]
```

experiments/compare_encoders.py

```
"""Compare pre-trained sentence encoders on the capability dataset.

The encoder sets the ceiling of every semantic method, so it is chosen first.
For each candidate we build the capability cards, then measure on the *train*
paraphrases (the choice) and report the *test* paraphrases next to them:

    top1     : the most similar card is of the right class
    f1_tau   : acceptance threshold that maximises F1 of "accept the best card"
    ceiling  : best card is of the right class and passes tau

Writes results/encoders.json.

Written with AI assistance (experiment tooling).
"""
import json
import sys
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from agentroute.config import RESULTS  # noqa: E402
from agentroute.data.capabilities import capability_cards, split_intents  # noqa: E402
from agentroute.embeddings import embed_texts  # noqa: E402

# (model name, prefix of a query, prefix of a capability sentence)
CANDIDATES = [
    ("sentence-transformers/all-MiniLM-L6-v2", "", ""),
    ("sentence-transformers/multi-qa-MiniLM-L6-cos-v1", "", ""),
    ("sentence-transformers/all-mpnet-base-v2", "", ""),
    ("BAAI/bge-small-en-v1.5", "Represent this sentence for searching relevant passages: ", ""),
    ("BAAI/bge-base-en-v1.5", "Represent this sentence for searching relevant passages: ", ""),
    ("intfloat/e5-small-v2", "query: ", "passage: "),
]


def score(intent_vecs, intent_class, card_vecs, card_class, tau):
    sims = intent_vecs @ card_vecs.T
    best = sims.argmax(axis=1)
    right = card_class[best] == intent_class
    passes = sims.max(axis=1) >= tau
    return {"top1": float(right.mean()),
            "ceiling": float((right & passes).mean()),
            "wrong_accept": float((~right & passes).mean())}


def best_tau(intent_vecs, intent_class, card_vecs, card_class):
    """Tau that maximises F1 of the decision "accept the most similar card"."""
    sims = intent_vecs @ card_vecs.T
    best = sims.argmax(axis=1)
    right = card_class[best] == intent_class
    top = sims.max(axis=1)
    out = (0.0, 0.0)
    for tau in np.arange(0.05, 0.95, 0.005):
        tp = (right & (top >= tau)).sum()
        fp = (~right & (top >= tau)).sum()
        fn = (right & (top < tau)).sum()
        f1 = 2 * tp / max(2 * tp + fp + fn, 1)
        if f1 > out[0]:
            out = (float(f1), float(tau))
    return out[1]


def main():
    cards = capability_cards()
    train, test = split_intents()
    card_class = np.array([c[0] for c in cards])
    out = {}
    for name, q_prefix, doc_prefix in CANDIDATES:
        flat = [doc_prefix + s for c in cards for s in c[2]]
        v = embed_texts(flat, name)
        card_vecs = np.stack([v[i:i + 3].mean(axis=0) for i in range(0, len(v), 3)])
```

```
        card_vecs = card_vecs / np.linalg.norm(card_vecs, axis=1, keepdims=True)
        tr_vecs = embed_texts([q_prefix + t for _, t in train], name)
        te_vecs = embed_texts([q_prefix + t for _, t in test], name)
        tr_y = np.array([c for c, _ in train])
        te_y = np.array([c for c, _ in test])
        tau = best_tau(tr_vecs, tr_y, card_vecs, card_class)
        out[name] = {"dim": int(card_vecs.shape[1]), "tau": round(tau, 3),
                     "train": score(tr_vecs, tr_y, card_vecs, card_class, tau),
                     "test": score(te_vecs, te_y, card_vecs, card_class, tau)}
        print(name, json.dumps(out[name]), flush=True)
    with open(RESULTS / "encoders.json", "w") as f:
        json.dump(out, f, indent=2)


if __name__ == "__main__":
    main()
```

experiments/compare_projection.py

```
"""Would a small trained layer on top of the encoder help?

A linear map W, initialised to the identity, is trained on the train
paraphrases with a contrastive loss and a penalty that keeps it close to the
identity. It is tested twice on the held-out paraphrases: for classes it has
seen, and (5 folds of 10 classes) for classes left out of training, which is
the case that matters if new capabilities appear. Writes results/projection.json.

Written with AI assistance (experiment tooling).
"""
import json
import sys
from pathlib import Path

import numpy as np
import torch
import torch.nn.functional as F

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from agentroute.config import RESULTS  # noqa: E402
from agentroute.data.capabilities import capability_cards, split_intents  # noqa: E402
from agentroute.embeddings import MODEL_NAME, embed_texts  # noqa: E402

ENCODERS = [(MODEL_NAME, ""),
            ("BAAI/bge-small-en-v1.5", "Represent this sentence for searching relevant passages: ")]
PENALTIES = [0.3, 0.1, 0.03]


def load(name, prefix):
    cards = capability_cards()
    train, test = split_intents()
    card_class = np.array([c[0] for c in cards])
    v = embed_texts([s for c in cards for s in c[2]], name)
    C = np.stack([v[i:i + 3].mean(axis=0) for i in range(0, len(v), 3)])
    C = C / np.linalg.norm(C, axis=1, keepdims=True)
    Xtr = embed_texts([prefix + t for _, t in train], name)
    Xte = embed_texts([prefix + t for _, t in test], name)
    return C, card_class, Xtr, np.array([c for c, _ in train]), Xte, np.array([c for c, _ in test])


def fit(C, card_class, X, y, classes, penalty, steps=400, temperature=0.05):
    torch.manual_seed(0)
    d = C.shape[1]
    W = torch.nn.Parameter(torch.eye(d))
    opt = torch.optim.Adam([W], lr=1e-3)
    keep = np.isin(card_class, classes)
    cards = torch.tensor(C[keep])
    index = {c: i for i, c in enumerate(sorted(set(card_class[keep].tolist())))}
    labels = torch.tensor([index[c] for c in card_class[keep].tolist()])
    use = np.isin(y, classes)
    Xt = torch.tensor(X[use])
    yt = torch.tensor([index[c] for c in y[use].tolist()])
    for _ in range(steps):
        zc = F.normalize(cards @ W.T, dim=1)
        zx = F.normalize(Xt @ W.T, dim=1)
        s = zx @ zc.T / temperature
        per_class = torch.stack([s[:, labels == k].max(dim=1).values for k in range(len(index))],
dim=1)
        loss = F.cross_entropy(per_class, yt) + penalty * ((W - torch.eye(d)) ** 2).sum()
        opt.zero_grad()
        loss.backward()
        opt.step()
    return W.detach().numpy()


def top1(C, card_class, X, y, W=None):
    if W is not None:
        C = C @ W.T
        X = X @ W.T
        C = C / np.linalg.norm(C, axis=1, keepdims=True)
        X = X / np.linalg.norm(X, axis=1, keepdims=True)
    return float((card_class[(X @ C.T).argmax(axis=1)] == y).mean())
```

```
def main():
    torch.set_num_threads(2)
    out = {}
    for name, prefix in ENCODERS:
        C, cc, Xtr, ytr, Xte, yte = load(name, prefix)
        row = {"base_test_top1": round(top1(C, cc, Xte, yte), 4), "penalty": {}}
        perm = np.random.default_rng(0).permutation(50)
        for penalty in PENALTIES:
            seen = top1(C, cc, Xte, yte, fit(C, cc, Xtr, ytr, np.arange(50), penalty))
            unseen = []
            for f in range(5):
                out_cls = perm[f * 10:(f + 1) * 10]
                W = fit(C, cc, Xtr, ytr, np.setdiff1d(np.arange(50), out_cls), penalty, steps=300)
                mask = np.isin(yte, out_cls)
                unseen.append(top1(C, cc, Xte[mask], yte[mask], W))
            row["penalty"][str(penalty)] = {"seen_classes": round(seen, 4), "unseen_classes":
round(float(np.mean(unseen)), 4)}
        out[name] = row
        print(name, json.dumps(row), flush=True)
    with open(RESULTS / "projection.json", "w") as f:
        json.dump(out, f, indent=2)


if __name__ == "__main__":
    main()
```

experiments/run_ablation.py

```
"""Ablations (1,000 agents):
  1. S-FIB content: mean / k = 2..32 / two-tier off / exact table
  2. intent header compression: float32 / int8 / binary
  3. acceptance rule: first agent above tau (v1) / best known candidate
  4. handover suppression threshold
  5. flooding collection window

Written with AI assistance (experiment tooling).
"""
from common import describe, parse_args, seeds_for, sweep

SUMMARIES = {
    "mean": {"sfib": {"summary": "mean", "exact_hops": 0}},
    "k=8, no tier": {"sfib": {"k": 8, "exact_hops": 0}},
    "k=2": {"sfib": {"k": 2}},
    "k=4": {"sfib": {"k": 4}},
    "k=8": {"sfib": {"k": 8}},
    "k=16": {"sfib": {"k": 16}},
    "k=32": {"sfib": {"k": 32}},
    "exact": {"sfib": {"summary": "exact"}},
}
METHODS = ["agentroute", "greedy", "greedy_load"]

if __name__ == "__main__":
    args = parse_args(__doc__)
    base = {"agents": {"n_agents": 1000}}
    seeds = seeds_for(args.quick)
    common = dict(quick=args.quick, workers=args.workers, config=args.config, base=base)

    names = ["mean", "k=8"] if args.quick else list(SUMMARIES)
    df = sweep("ablation_summary", names, lambda v: SUMMARIES[v], METHODS, seeds, label="summary",
**common)
    describe(df, label="summary", cols=("success_rate", "time_ms", "hops", "ctrl_kB_per_s"))

    headers = ["float32", "binary"] if args.quick else ["float32", "int8", "binary"]
    df = sweep("ablation_header", headers, "forwarding.header", METHODS, seeds, label="header",
**common)
    describe(df, label="header", cols=("success_rate", "time_ms", "hops", "query_kB_per_query"))

    df = sweep("ablation_accept", ["first", "best_known"], "forwarding.accept", ["greedy",
"greedy_load"],
               seeds, label="accept", **common)
    describe(df, label="accept", cols=("success_rate", "precision", "time_ms", "hops"))

    thresholds = [0.0, 0.01] if args.quick else [0.0, 0.001, 0.01, 0.05, 0.2]
    df = sweep("ablation_threshold", thresholds, "sfib.change_threshold", ["greedy"], seeds,
               label="threshold", **common)
    describe(df, label="threshold", cols=("success_rate", "update_msgs", "ctrl_kB_per_s"))

    windows = [0.05, 0.2] if args.quick else [0.02, 0.05, 0.1, 0.2, 0.5, 1.0]
    df = sweep("ablation_window", windows, "baselines.flood_window_s", ["flooding"], seeds,
               label="window", **common)
    describe(df, label="window", cols=("success_rate", "time_ms"))
```

experiments/run_all.py

```
"""Run the whole pipeline: tau -> training -> experiments -> figures -> demo -> animations.

    python experiments/run_all.py            # full run (about 1 hour on a MacBook Air)
    python experiments/run_all.py --quick    # smoke test (a few minutes)
    python experiments/run_all.py --skip-train

Written with AI assistance (experiment tooling).
"""
import argparse
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
PY = sys.executable


def step(name, args):
    print(f"\n=== {name} ===", flush=True)
    t0 = time.time()
    subprocess.run([PY] + args, cwd=ROOT, check=True)
    print(f"=== {name} done in {time.time() - t0:.0f}s ===", flush=True)


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--quick", action="store_true")
    p.add_argument("--skip-train", action="store_true")
    p.add_argument("--workers", default="6")
    args = p.parse_args()
    quick = ["--quick"] if args.quick else []
    workers = ["--workers", args.workers]

    step("compare encoders", ["experiments/compare_encoders.py"])
    step("tune tau", ["experiments/tune_tau.py"])
    models = [ROOT / "results" / "models" / "dqn.pt", ROOT / "results" / "models" / "gnn.pt"]
    if not args.skip_train and not all(m.exists() for m in models):
        step("train DQN + GNN", ["experiments/run_train.py"] + (["--episodes", "20"] if args.quick else
[]))
    for name in ["scale", "mobility", "failure", "radius", "load", "rate", "tau", "ablation",
                 "matched", "handover_cost"]:
        step(name, [f"experiments/run_{name}.py"] + quick + (workers if name != "handover_cost" else
[]))
    step("figures", ["viz/plots.py"])
    step("demo runs", ["experiments/run_demo.py"])
    step("animations", ["viz/animate.py"])
    step("export to web player", ["viz/export_web_data.py"])


if __name__ == "__main__":
    main()
```

experiments/run_demo.py

```python
"""Run the small demo scenario once per method with full event logging.

Writes results/logs/demo/<method>/{topology.json, events.jsonl, summary.json},
which the animation, the Streamlit app and the React player read.

Written with AI assistance (experiment tooling).
"""
import argparse
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from agentroute.config import load_config  # noqa: E402
from agentroute.sim.simulator import METHODS, Simulator  # noqa: E402


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--config", default=str(ROOT / "configs" / "demo.yaml"))
    p.add_argument("--methods", nargs="*", default=METHODS)
    p.add_argument("--out", default=str(ROOT / "results" / "logs" / "demo"))
    args = p.parse_args()
    for method in args.methods:
        cfg = load_config(args.config)
        s = Simulator(cfg, method, log_dir=Path(args.out) / method).run()
        print(f"{method:15s} queries={s['n_queries']:4d} success={s['success_rate']:.3f} "
              f"time={s['time_ms']:.1f} ms hops={s['hops']:.2f}")


if __name__ == "__main__":
    main()
```

experiments/run_failure.py

```
"""Robustness to node failures: fraction of failed routers vs success (1,000 agents).

Routers fail at one third of the run; neighbours notice after the detection delay.

Written with AI assistance (experiment tooling).
"""
from common import ALL_METHODS, describe, parse_args, seeds_for, sweep

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [0.0, 0.2] if args.quick else [0.0, 0.05, 0.1, 0.2, 0.3]
    df = sweep("failure", values, "sim.failure_fraction", ALL_METHODS, seeds_for(args.quick),
               quick=args.quick, workers=args.workers, config=args.config,
               base={"agents": {"n_agents": 1000}}, label="failure")
    describe(df, label="failure")
```

experiments/run_handover_cost.py

```python
"""Cost of mobility in the control plane.

For each network size we measure, per second and per handover, what the S-FIB
updates cost and put that next to two reference points: the periodic refresh
of the same network and the update traffic of a replicated registry.

Written with AI assistance (experiment tooling).
"""
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from agentroute.config import RESULTS, load_config  # noqa: E402
from agentroute.sim.simulator import Simulator  # noqa: E402


def run(method, n, seed):
    cfg = load_config(overrides={"agents": {"n_agents": n, "speed_mps": 5.0}})
    sim = Simulator(cfg, method, seed=seed)
    s = sim.run()
    return sim, s, cfg["sim"]["duration_s"]


def main():
    quick = "--quick" in sys.argv
    sizes = [1000] if quick else [1000, 3000, 10000]
    seeds = [1] if quick else [1, 2, 3]
    out = {}
    for n in sizes:
        rows = []
        for seed in seeds:
            sim, s, dur = run("greedy", n, seed)
            full_msgs, full_bytes = sim.adv.round_cost()
            reg_sim, reg, _ = run("registry3", n, seed)
            rows.append({"ho": s["handovers"], "upd": s["update_msgs"], "routers": s["n_routers"],
                         "full_msgs": full_msgs, "full_kB": full_bytes / 1000.0, "dur": dur,
                         "delta_B": sim.adv.delta_bytes()})
        ho = sum(r["ho"] for r in rows)
        dur, period = rows[0]["dur"], 10.0
        upd_per_s = sum(r["upd"] for r in rows) / len(rows) / dur
        full_per_s = sum(r["full_msgs"] for r in rows) / len(rows) / period
        out[str(n)] = {
            "routers": rows[0]["routers"],
            "handovers_per_s": round(ho / len(rows) / dur, 1),
            "msgs_per_handover": round(sum(r["upd"] for r in rows) / max(ho, 1), 2),
            "handover_msgs_per_s": round(upd_per_s, 1),
            "handover_kB_per_s": round(upd_per_s * rows[0]["delta_B"] / 1000.0, 1),
            "refresh_msgs_per_s": round(full_per_s, 1),
            "refresh_kB_per_s": round(sum(r["full_kB"] for r in rows) / len(rows) / period, 1),
        }
        print(n, out[str(n)], flush=True)
    path = RESULTS / "raw" / ("handover_cost_quick.json" if quick else "handover_cost.json")
    path.write_text(json.dumps(out, indent=2))
    print("wrote", path)


if __name__ == "__main__":
    main()
```

experiments/run_load.py

```python
"""Network load vs success and completion time (1,000 agents).

The load factor scales both the task request rate and the background
utilisation of every router.

Written with AI assistance (experiment tooling).
"""
from common import ALL_METHODS, describe, parse_args, seeds_for, sweep

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [1.0, 4.0] if args.quick else [0.5, 1.0, 2.0, 3.0, 4.0]
    df = sweep("load", values, "sim.load", ALL_METHODS, seeds_for(args.quick),
               quick=args.quick, workers=args.workers, config=args.config,
               base={"agents": {"n_agents": 1000}}, label="load")
    describe(df, label="load")
```

experiments/run_matched.py

```python
"""Paired comparisons on the same requests.

Mean completion time over successful requests is biased: a method that gives up
on slow requests looks fast. Here two methods are compared only on the requests
that both delivered. Differences in success are computed per seed and given
with a bootstrap 95% interval. Writes results/raw/matched.json.

Written with AI assistance (experiment tooling).
"""
import json
import sys
from multiprocessing import Pool
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

import numpy as np  # noqa: E402

from agentroute.config import RESULTS, load_config  # noqa: E402
from common import SEEDS, parse_args  # noqa: E402

SCENARIOS = {
    "1000 agents, load 1": {"agents": {"n_agents": 1000}},
    "1000 agents, load 4": {"agents": {"n_agents": 1000}, "sim": {"load": 4.0}},
    "10000 agents, load 1": {"agents": {"n_agents": 10000}},
}
METHODS = ["agentroute", "agentroute_gnn", "greedy", "greedy_load", "spfr"]
PAIRS = [("agentroute", "greedy"), ("agentroute", "greedy_load"), ("agentroute", "spfr"),
         ("agentroute_gnn", "agentroute"), ("greedy_load", "greedy")]


def run(job):
    import torch
    torch.set_num_threads(1)
    from agentroute.sim.simulator import Simulator
    name, over, method, seed = job
    sim = Simulator(load_config(overrides=over), method, seed=seed)
    sim.run()
    return name, method, seed, {r["pid"]: (r["success"], r["time"] * 1000.0, r["hops"]) for r in
sim.results}


def bootstrap_ci(x, rng, n=2000):
    means = [np.mean(rng.choice(x, size=len(x))) for _ in range(n)]
    return float(np.percentile(means, 2.5)), float(np.percentile(means, 97.5))


def main():
    args = parse_args(__doc__)
    seeds = SEEDS[:2] if args.quick else SEEDS
    scenarios = dict(list(SCENARIOS.items())[:1]) if args.quick else SCENARIOS
    jobs = [(name, over, m, s) for name, over in scenarios.items() for m in METHODS for s in seeds]
    with Pool(args.workers) as pool:
        rows = pool.map(run, jobs)
    res = {(n, m, s): r for n, m, s, r in rows}
    rng = np.random.default_rng(0)
    out = {}
    for name in scenarios:
        out[name] = {}
        for a, b in PAIRS:
            ta, tb, ha, hb, diff, only_a, only_b = [], [], [], [], [], 0, 0
            for s in seeds:
                ra, rb = res[(name, a, s)], res[(name, b, s)]
                common = sorted(set(ra) & set(rb))
                diff.append(np.mean([ra[p][0] for p in common]) - np.mean([rb[p][0] for p in common]))
                for p in common:
                    if ra[p][0] and rb[p][0]:
                        ta.append(ra[p][1])
                        tb.append(rb[p][1])
                        ha.append(ra[p][2])
                        hb.append(rb[p][2])
```

```
                    elif ra[p][0]:
                        only_a += 1
                    elif rb[p][0]:
                        only_b += 1
            low, high = bootstrap_ci(np.array(diff) * 100, rng)
            out[name][f"{a} vs {b}"] = {
                "both_delivered": len(ta),
                "time_ms": [round(float(np.mean(ta)), 2), round(float(np.mean(tb)), 2)],
                "hops": [round(float(np.mean(ha)), 2), round(float(np.mean(hb)), 2)],
                "median_time_diff_ms": round(float(np.median(np.array(ta) - np.array(tb))), 2),
                "only_first_delivered": only_a, "only_second_delivered": only_b,
                "success_diff_points": round(float(np.mean(diff)) * 100, 2),
                "success_diff_ci95": [round(low, 2), round(high, 2)],
            }
            print(name, a, "vs", b, out[name][f"{a} vs {b}"], flush=True)
    path = RESULTS / "raw" / ("matched_quick.json" if args.quick else "matched.json")
    path.write_text(json.dumps(out, indent=2))
    print("wrote", path)


if __name__ == "__main__":
    main()
```

experiments/run_mobility.py

```
"""Robustness to mobility: random-waypoint speed vs success (1,000 agents).

Two cell sizes: the default 500 m macro cells and 150 m small cells, where the
handover rate is more than three times higher.

Written with AI assistance (experiment tooling).
"""
from common import ALL_METHODS, describe, parse_args, seeds_for, sweep

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [0.0, 10.0] if args.quick else [0.0, 2.0, 5.0, 10.0, 20.0, 30.0]
    for name, spacing in (("mobility", 500), ("mobility_small", 150)):
        df = sweep(name, values, "agents.speed_mps", ALL_METHODS, seeds_for(args.quick),
                   quick=args.quick, workers=args.workers, config=args.config,
                   base={"agents": {"n_agents": 1000}, "topology": {"cell_spacing_m": spacing}},
                   label="speed")
        describe(df, label="speed", cols=("success_rate", "time_ms", "handovers", "ctrl_msgs_per_s",
"ctrl_kB_per_s"))
```

experiments/run_radius.py

```python
"""Advertisement radius R vs success and control overhead (semantic methods, 1,000 agents).

Written with AI assistance (experiment tooling).
"""
from common import describe, parse_args, seeds_for, sweep

METHODS = ["agentroute", "agentroute_gnn", "greedy", "greedy_load"]     # SPFR has its own horizon

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [1, 3] if args.quick else [1, 2, 3, 4, 5, 6]
    df = sweep("radius", values, "sfib.radius", METHODS, seeds_for(args.quick),
               quick=args.quick, workers=args.workers, config=args.config,
               base={"agents": {"n_agents": 1000}}, label="radius")
    describe(df, label="radius", cols=("success_rate", "time_ms", "ctrl_msgs_per_s", "ctrl_kB_per_s"))
```

experiments/run_rate.py

```python
"""Request rate vs traffic (1,000 agents).

Control traffic is paid per second, query traffic per request, so "messages
per request" depends on the request rate that was assumed. This sweep shows
the total traffic of every method at each rate and where the curves cross.

Written with AI assistance (experiment tooling).
"""
from common import describe, parse_args, seeds_for, sweep

METHODS = ["agentroute", "greedy", "greedy_load", "spfr", "flooding", "registry", "registry3", "sdht"]

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [1.0, 30.0] if args.quick else [1.0, 3.0, 10.0, 30.0, 100.0, 300.0]
    df = sweep("rate", values, "sim.query_rate", METHODS, seeds_for(args.quick),
               quick=args.quick, workers=args.workers, config=args.config,
               base={"agents": {"n_agents": 1000}}, label="rate")
    describe(df, label="rate", cols=("success_rate", "time_ms", "ctrl_kB_per_s", "query_kB_per_query"))
```

experiments/run_scale.py

```python
"""Scalability: number of agents (100 -> 10,000) vs success, overhead, hops, time.

Written with AI assistance (experiment tooling).
"""
from common import ALL_METHODS, describe, parse_args, seeds_for, sweep

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [100, 300] if args.quick else [100, 300, 1000, 3000, 10000]
    df = sweep("scale", values, "agents.n_agents", ALL_METHODS, seeds_for(args.quick),
               quick=args.quick, workers=args.workers, config=args.config, label="n_agents_param")
    describe(df, label="n_agents_param")
```

experiments/run_tau.py

```
"""Acceptance threshold tau vs success and wrong deliveries (1,000 agents).

A low tau finds more agents but hands more tasks to the wrong one. Every point
of this sweep is an operating point; tau = results/tau.json is the one we use.

Written with AI assistance (experiment tooling).
"""
from common import describe, parse_args, seeds_for, sweep

METHODS = ["agentroute", "greedy", "greedy_load", "spfr", "flooding", "registry"]

if __name__ == "__main__":
    args = parse_args(__doc__)
    values = [0.3, 0.45] if args.quick else [0.25, 0.30, 0.355, 0.40, 0.45, 0.50]
    df = sweep("tau", values, "forwarding.tau", METHODS, seeds_for(args.quick),
               quick=args.quick, workers=args.workers, config=args.config,
               base={"agents": {"n_agents": 1000}}, label="tau")
    describe(df, label="tau", cols=("success_rate", "precision", "oracle_best"))
```

experiments/run_train.py

```python
"""Train the DQN (and the GNN variant) and save the weights and learning curves.

    python experiments/run_train.py              # both models
    python experiments/run_train.py --kind mlp --episodes 30

Responsible: Bekir Göktepe
"""
import argparse
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from agentroute.dqn.train import train  # noqa: E402


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--kind", choices=["mlp", "gnn", "both"], default="both")
    p.add_argument("--episodes", type=int, default=None)
    p.add_argument("--config", default=None)
    args = p.parse_args()
    kinds = ["mlp", "gnn"] if args.kind == "both" else [args.kind]
    for kind in kinds:
        best, greedy = train(kind, episodes=args.episodes, config_path=args.config)
        print(f"{kind}: best validation return {best:.3f} (greedy success {greedy[0]:.3f})")


if __name__ == "__main__":
    main()
```

experiments/tune_tau.py

```python
"""Tune the acceptance threshold tau on the training paraphrases.

An agent accepts a task when cos(intent, capability) >= tau. v2 chooses tau
for the question the network really faces: "is the best agent I can see good
enough?". For every training intent we take sampled agents of all classes and
score each outcome the way the DQN reward does:

    +1  the best agent has the right class and passes tau
    -1  the best agent has the wrong class and passes tau
     0  nothing passes tau (the requester knows that nothing was found)

A capable agent exists with probability P_EXISTS; in the other case the
intent's class is removed and any accepted agent counts as wrong. tau maximises
the expected score. Results are written to results/tau.json and read by the
simulator.

Responsible: Bekir Emre Sarıpınar (v2 revision written with AI assistance)
"""
import json
import sys
from pathlib import Path

import numpy as np

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from agentroute.config import RESULTS, load_config  # noqa: E402
from agentroute.embeddings import agent_vectors, load_dataset_vectors  # noqa: E402

AGENTS_PER_CLASS = 20
P_EXISTS = 0.9


def open_set_scores(intent_vecs, intent_class, agent_vecs, agent_class, tau):
    """Returns (success if capable agents exist, rejection rate if none exists, wrong accepts)."""
    sims = intent_vecs @ agent_vecs.T
    best = sims.argmax(axis=1)
    right = agent_class[best] == intent_class
    passes = sims.max(axis=1) >= tau
    other = np.where(agent_class[None, :] == intent_class[:, None], -2.0, sims)
    reject = other.max(axis=1) < tau
    return float((right & passes).mean()), float(reject.mean()), float((~right & passes).mean())


def main():
    cfg = load_config()
    data = load_dataset_vectors()
    n_cls = data["class_vecs"].shape[0]
    rng = np.random.default_rng(0)
    cls = np.repeat(np.arange(n_cls), AGENTS_PER_CLASS)
    vecs = agent_vectors(data["class_vecs"], cls, rng, cfg["agents"]["card_alpha"])
    best = (-1.0, 0.0)
    curve = []
    for tau in np.arange(0.10, 0.80, 0.005):
        acc, rej, wrong = open_set_scores(data["train_vecs"], data["train_class"], vecs, cls, tau)
        score = P_EXISTS * (acc - wrong) - (1 - P_EXISTS) * (1 - rej)
        curve.append({"tau": round(float(tau), 3), "accept_right": round(acc, 4),
                      "reject_when_absent": round(rej, 4), "wrong_accept": round(wrong, 4)})
        if score > best[0]:
            best = (score, float(tau))
    tau = round(best[1], 3)
    acc, rej, wrong = open_set_scores(data["test_vecs"], data["test_class"], vecs, cls, tau)
    tr_acc, tr_rej, tr_wrong = open_set_scores(data["train_vecs"], data["train_class"], vecs, cls, tau)
    top1 = float((cls[(data["test_vecs"] @ vecs.T).argmax(axis=1)] == data["test_class"]).mean())
    result = {"tau": tau, "train_score": round(best[0], 4),
              "train_accept_right": round(tr_acc, 4), "train_reject_when_absent": round(tr_rej, 4),
              "train_wrong_accept": round(tr_wrong, 4),
              "test_top1": round(top1, 4), "test_accept_right": round(acc, 4),
              "test_reject_when_absent": round(rej, 4), "test_wrong_accept": round(wrong, 4),
              "curve": curve}
    RESULTS.mkdir(exist_ok=True)
    with open(RESULTS / "tau.json", "w") as f:
        json.dump(result, f, indent=2)
```

```
    print(json.dumps({k: v for k, v in result.items() if k != "curve"}, indent=2))


if __name__ == "__main__":
    main()
```

viz/animate.py

```
"""Presentation animations (GIF + MP4) from the demo event logs.

    python viz/animate.py              # both videos
    python viz/animate.py --only demo

1. agentroute_demo: advertisements fill the S-FIBs, a request travels hop by
   hop to a capable agent, a phone changes cell (withdraw / announce).
2. greedy_vs_agentroute: the same request in the same scenario, routed by
   greedy forwarding and by the DQN side by side.

Every scene is built from real events of the log; nothing is staged. If the
log has no suitable event for a scene, the scene is skipped.

Written with AI assistance (visualisation).
"""
import argparse
import json
from collections import defaultdict
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
from matplotlib import animation  # noqa: E402

ROOT = Path(__file__).resolve().parent.parent
LOGS = ROOT / "results" / "logs" / "demo"
OUT = ROOT / "results" / "figures"

FPS = 20
SURFACE, INK, INK2, MUTED = "#fcfcfb", "#0b0b0b", "#52514e", "#898781"
LINK, GRID = "#c3c2b7", "#e1e0d9"
BLUE, AQUA = "#2a78d6", "#1baf7a"
GOOD, CRITICAL, SERIOUS = "#0ca30c", "#d03b3b", "#ec835a"
METHOD_COLOR = {"agentroute": BLUE, "agentroute_gnn": "#eb6834", "greedy": AQUA}
METHOD_LABEL = {"agentroute": "AgentRoute (DQN)", "agentroute_gnn": "AgentRoute (GNN)",
                "greedy": "Greedy semantic"}


# ---------------------------------------------------------------------------
class Run:
    """Parsed log of one demo run."""

    def __init__(self, method, log_dir=None):
        d = Path(log_dir) if log_dir else LOGS / method
        self.topo = json.loads((d / "topology.json").read_text())
        self.method = self.topo["meta"]["method"]
        self.events = [json.loads(line) for line in (d / "events.jsonl").read_text().splitlines() if
line]
        self.pos = {n["id"]: (n["x"], n["y"]) for n in self.topo["nodes"]}
        self.kind = {n["id"]: n["kind"] for n in self.topo["nodes"]}
        self.agents = {a["id"]: a for a in self.topo["agents"]}
        self.queries, self.hops, self.outcome = {}, defaultdict(list), {}
        self.handovers, self.adverts = [], []
        self.moves = defaultdict(list)
        self.load = defaultdict(list)
        for e in self.events:
            t = e["type"]
            if t == "query":
                self.queries[e["pkt"]] = e
            elif t == "hop":
                self.hops[e["pkt"]].append(e)
            elif t in ("deliver", "drop"):
                self.outcome[e["pkt"]] = e
            elif t == "handover":
                self.handovers.append(e)
            elif t in ("adv", "withdraw"):
                self.adverts.append(e)
            elif t == "move":
                self.moves[e["agent"]].append((e["t"], e["x"], e["y"]))
```

```
            elif t == "load":
                self.load[e["node"]].append((e["t"], e["busy"]))
        self.classes = self.topo["meta"]["classes"]

    def agent_xy(self, a, t):
        tr = self.moves.get(a)
        if not tr:
            return self.agents[a]["x"], self.agents[a]["y"]
        ts = [p[0] for p in tr]
        if t <= ts[0]:
            return tr[0][1], tr[0][2]
        i = int(np.searchsorted(ts, t))
        if i >= len(tr):
            return tr[-1][1], tr[-1][2]
        (t0, x0, y0), (t1, x1, y1) = tr[i - 1], tr[i]
        f = (t - t0) / max(t1 - t0, 1e-9)
        return x0 + f * (x1 - x0), y0 + f * (y1 - y0)

    def busy(self, node, t):
        state = False
        for ts, b in self.load.get(node, []):
            if ts <= t:
                state = b
        return state

    def path(self, pid):
        hops = self.hops.get(pid, [])
        return [h["from"] for h in hops] + ([hops[-1]["to"]] if hops else [])


# ---------------------------------------------------------------------------
def draw_static(ax, run, title=None):
    ax.set_facecolor(SURFACE)
    xs = [p[0] for p in run.pos.values()]
    ys = [p[1] for p in run.pos.values()]
    pad = 320
    ax.set_xlim(min(xs) - pad, max(xs) + pad)
    ax.set_ylim(min(ys) - pad, max(ys) + pad)
    ax.set_aspect("equal")
    ax.axis("off")
    spacing = 500
    for n, (x, y) in run.pos.items():
        if run.kind[n] == "gnb":
            ax.add_patch(plt.Circle((x, y), spacing * 0.56, color="#eef3fb", zorder=0, lw=0))
    style = {"xn": (LINK, 1.0, "-"), "backhaul": (LINK, 0.8, "-"), "core": ("#898781", 1.6, "-"),
             "cloud": ("#898781", 1.6, (0, (4, 3)))}
    for l in run.topo["links"]:
        c, w, ls = style[l["kind"]]
        (x1, y1), (x2, y2) = run.pos[l["u"]], run.pos[l["v"]]
        ax.plot([x1, x2], [y1, y2], color=c, lw=w, ls=ls, zorder=1)
    for n, (x, y) in run.pos.items():
        k = run.kind[n]
        if k == "gnb":
            ax.scatter(x, y, s=150, marker="s", color="#ffffff", edgecolor=INK2, lw=1.4, zorder=4)
        elif k == "core":
            ax.scatter(x, y, s=230, marker="o", color="#52514e", edgecolor=SURFACE, lw=1.5, zorder=4)
        else:
            ax.scatter(x, y, s=420, marker="h", color="#52514e", edgecolor=SURFACE, lw=1.5, zorder=4)
            ax.text(x, y - 150, "Cloud", ha="center", va="top", color=INK2, fontsize=9)
    if title:
        ax.set_title(title, color=INK, fontsize=12, fontweight="bold", loc="left")


def draw_legend(ax):
    ax.axis("off")
    items = [
        ("s", "#ffffff", INK2, 120, "gNB + edge router (AgentRoute node)"),
        ("o", "#52514e", SURFACE, 160, "Core router (AgentRoute node)"),
        ("h", "#52514e", SURFACE, 220, "Cloud data centre"),
        ("o", MUTED, MUTED, 30, "Agent (fixed or mobile)"),
        ("o", BLUE, SURFACE, 90, "Task packet / tracked phone"),
    ]
    for i, (marker, fc, ec, size, label) in enumerate(items):
        y = 0.97 - i * 0.065
        ax.scatter([0.03], [y], marker=marker, s=size, color=fc, edgecolor=ec, lw=1.4,
                   transform=ax.transAxes, clip_on=False)
        ax.text(0.09, y, label, fontsize=11, color=INK2, va="center", transform=ax.transAxes)
```

```
    rings = [(GOOD, "Serving agent / announce"), (CRITICAL, "Withdraw (old cell)")]
    for i, (color, label) in enumerate(rings):
        y = 0.97 - (len(items) + i) * 0.065
        ax.scatter([0.03], [y], s=110, facecolor="none", edgecolor=color, lw=2.2,
                   transform=ax.transAxes, clip_on=False)
        ax.text(0.09, y, label, fontsize=11, color=INK2, va="center", transform=ax.transAxes)
    ax.plot([0.0, 0.06], [0.97 - 7 * 0.065] * 2, color=BLUE, lw=2, ls=(0, (3, 2)),
            transform=ax.transAxes, clip_on=False)
    ax.text(0.09, 0.97 - 7 * 0.065, "Capability advertisement", fontsize=11, color=INK2,
            va="center", transform=ax.transAxes)


def lerp(a, b, f):
    return a[0] + (b[0] - a[0]) * f, a[1] + (b[1] - a[1]) * f


class Animator:
    """Turns a list of frame specs into matplotlib artists."""

    def __init__(self, fig, axes, runs, info_ax=None, steps=None):
        self.fig, self.axes, self.runs = fig, axes, runs
        self.info = info_ax
        self.dynamic = []
        self.agent_sc = []
        for ax, run in zip(axes, runs):
            ids = sorted(run.agents)
            xy = np.array([run.agent_xy(a, 0) for a in ids])
            self.agent_sc.append((ids, ax.scatter(xy[:, 0], xy[:, 1], s=12, color=MUTED,
                                                  alpha=0.65, lw=0, zorder=3)))
        self.header = fig.text(0.03, 0.955, "", fontsize=17, fontweight="bold", color=INK, va="top")
        self.sub = fig.text(0.03, 0.905, "", fontsize=12, color=INK2, va="top")
        self.footer = fig.text(0.03, 0.03, "", fontsize=11, color=INK2, va="bottom")
        self.step_texts = []
        if info_ax is not None:
            draw_legend(info_ax)
            for i, step in enumerate(steps or []):
                self.step_texts.append(info_ax.text(0.0, 0.40 - i * 0.075, step, fontsize=12,
                                                    transform=info_ax.transAxes, color=MUTED))
            self.caption = info_ax.text(0.0, 0.10, "", fontsize=12, color=INK, wrap=True,
                                        transform=info_ax.transAxes, va="top")

    def clear(self):
        for a in self.dynamic:
            a.remove()
        self.dynamic = []

    def update(self, spec):
        self.clear()
        self.header.set_text(spec.get("title", ""))
        self.sub.set_text(spec.get("subtitle", ""))
        self.footer.set_text(spec.get("footer", ""))
        for i, t in enumerate(self.step_texts):
            current = spec.get("scene") == i + 1
            t.set_color(INK if current else MUTED)
            t.set_fontweight("bold" if current else "normal")
        if self.info is not None:
            self.caption.set_text(spec.get("caption", ""))
        for i, (ax, run) in enumerate(zip(self.axes, self.runs)):
            p = spec["panels"][i] if "panels" in spec else spec
            ids, sc = self.agent_sc[i]
            t = p.get("t", 0.0)
            sc.set_offsets(np.array([run.agent_xy(a, t) for a in ids]))
            for node in p.get("busy", []):
                x, y = run.pos[node]
                self.dynamic.append(ax.scatter(x, y, s=600, color=SERIOUS, alpha=0.35, lw=0, zorder=2))
                self.dynamic.append(ax.text(x + 60, y + 60, "busy", color=INK2, fontsize=8, zorder=6))
            for (a, b, alpha, color) in p.get("lines", []):
                self.dynamic += ax.plot([a[0], b[0]], [a[1], b[1]], color=color, lw=2.2, alpha=alpha,
                                        ls=(0, (3, 2)), zorder=5)
            for seg, color in p.get("trail", []):
                self.dynamic += ax.plot([seg[0][0], seg[1][0]], [seg[0][1], seg[1][1]], color=color,
                                        lw=3.0, zorder=6, solid_capstyle="round")
            for (x, y, color, size) in p.get("rings", []):
                self.dynamic.append(ax.scatter(x, y, s=size, facecolor="none", edgecolor=color,
                                               lw=2.5, zorder=7))
            for (x, y, color) in p.get("dots", []):
                self.dynamic.append(ax.scatter(x, y, s=110, color=color, edgecolor=SURFACE, lw=2,
```

```
                                            zorder=8))
            for (x, y, text) in p.get("labels", []):
                self.dynamic.append(ax.text(x, y, text, fontsize=9, color=INK, zorder=9,
                                            bbox=dict(boxstyle="round,pad=0.3", fc="#ffffff",
                                                      ec=GRID, lw=0.8)))
            if p.get("caption") and self.info is None:
                self.dynamic.append(ax.text(0.02, 0.02, p["caption"], transform=ax.transAxes,
                                            fontsize=10.5, color=INK, va="bottom",
                                            bbox=dict(boxstyle="round,pad=0.4", fc="#ffffff",
                                                      ec=GRID, lw=0.8)))
        return []


def packet_frames(run, pid, n_per_hop, color, t_agents, title, subtitle, hold=20):
    """Frames of one packet moving along its real path."""
    path = run.path(pid)
    out = run.outcome.get(pid, {})
    q = run.queries[pid]
    frames = []
    src = run.agent_xy(q["agent"], q["t"])
    start = run.pos[q["router"]]
    legs = [(src, start)] + [(run.pos[a], run.pos[b]) for a, b in zip(path, path[1:])]
    final = None
    if out.get("type") == "deliver":
        final = run.agent_xy(out["agent"], out["t"])
        legs.append((run.pos[path[-1]] if path else start, final))
    trail = []
    for a, b in legs:
        for k in range(n_per_hop):
            f = (k + 1) / n_per_hop
            frames.append({"t": t_agents,
                           "trail": [(s, color) for s in trail] + [((a, lerp(a, b, f)), color)],
                           "dots": [(*lerp(a, b, f), color)],
                           "rings": [(*src, color, 160)], "title": title, "subtitle": subtitle,
                           "caption": "Each router scores its neighbours\nwith its S-FIB and forwards
the\npacket to the best one"})
        trail.append((a, b))
    end = {"t": t_agents, "trail": [(s, color) for s in trail], "rings": [(*src, color, 160)],
           "title": title, "subtitle": subtitle}
    if out.get("type") == "deliver":
        ag = run.agents[out["agent"]]
        end["rings"].append((*final, GOOD, 260))
        end["labels"] = [(final[0] + 70, final[1] + 70, f"agent {ag['id']}: {ag['class_name']}")]
        end["caption"] = (f"Delivered to a capable agent\nin {out['hops']} hops, "
                          f"{(out['t'] - q['t']) * 1000:.0f} ms")
    else:
        where = out.get("at")
        if where is not None:
            end["rings"].append((*run.pos[where], CRITICAL, 300))
        end["caption"] = f"Dropped ({out.get('reason', 'unknown')})"
    frames += [dict(end) for _ in range(hold)]
    return frames


def pick_packet(run, after=3.0, min_hops=2, max_hops=5):
    best = None
    for pid, q in run.queries.items():
        out = run.outcome.get(pid)
        if q["t"] < after or not out or out["type"] != "deliver":
            continue
        n = len(run.hops.get(pid, []))
        if min_hops <= n <= max_hops and any(run.kind[h["to"]] == "core" for h in run.hops[pid]):
            if best is None or n > len(run.hops[best]):
                best = pid
    return best


def demo_frames(run):
    color = METHOD_COLOR.get(run.method, BLUE)
    frames = []
    # scene 1: advertisements at t = 0
    initial = [e for e in run.adverts if e["t"] == 0.0 and e["type"] == "adv"]
    n = 50
    for k in range(n):
        f = (k + 1) / n
        lines = [(run.pos[e["from"]], lerp(run.pos[e["from"]], run.pos[e["to"]], f), 0.55, BLUE)
                 for e in initial]
```

```
        frames.append({"t": 0.0, "lines": lines, "scene": 1,
                       "title": "1. Agents advertise their capabilities",
                       "subtitle": f"Each router learns, per neighbour, which capabilities lie behind
it "
                                   f"(radius R = {run.topo['meta']['radius']}, k-means summary)"})
    frames += [dict(frames[-1]) for _ in range(10)]
    # scene 2: one request, hop by hop
    pid = pick_packet(run)
    if pid is not None:
        q = run.queries[pid]
        scene = packet_frames(run, pid, 12, color, q["t"],
                              "2. A request is routed by meaning, hop by hop",
                              f"Intent: \"{q['text']}\"  (class: {run.classes[q['cls']]})")
        for f in scene:
            f["scene"] = 2
        frames += scene
    # scene 3: a phone changes cell
    ho = next((h for h in run.handovers if 6.0 < h["t"] < run.topo["meta"]["duration"] - 3
               and h["agent"] in run.moves), None)
    if ho is not None:
        a = ho["agent"]
        t0, t1 = ho["t"] - 3.0, ho["t"] + 1.0
        after = [e for e in run.adverts if ho["t"] <= e["t"] <= ho["t"] + 0.1]
        n = 80
        for k in range(n):
            t = t0 + (t1 - t0) * k / (n - 1)
            xy = run.agent_xy(a, t)
            spec = {"t": t, "dots": [(*xy, BLUE)], "scene": 3,
                    "title": "3. A phone changes cell",
                    "subtitle": f"Agent {a} ({run.agents[a]['class_name']}) moves; only nearby "
                                f"S-FIB entries are updated, nothing is re-flooded"}
            if t >= ho["t"]:
                spec["rings"] = [(*run.pos[ho["from_cell"]], CRITICAL, 900),
                                 (*run.pos[ho["to_cell"]], GOOD, 900)]
                spec["lines"] = [(run.pos[e["from"]], run.pos[e["to"]], 0.8,
                                  CRITICAL if e["type"] == "withdraw" else GOOD) for e in after]
                spec["caption"] = (f"Handover: cell {ho['from_cell']} -> {ho['to_cell']}\n"
                                   f"{len(after)} S-FIB update messages\n(withdraw + announce)")
            frames.append(spec)
        frames += [dict(frames[-1]) for _ in range(20)]
    return frames


def pick_comparison(a_run, b_run):
    """A request that both runs saw, where the paths differ and B (DQN) does better."""
    best, best_key = None, None
    for pid, qa in a_run.queries.items():
        if pid not in b_run.queries or qa["t"] < 3.0:
            continue
        oa, ob = a_run.outcome.get(pid), b_run.outcome.get(pid)
        if not oa or not ob or a_run.path(pid) == b_run.path(pid):
            continue
        if len(a_run.hops.get(pid, [])) < 1 or len(b_run.hops.get(pid, [])) < 1:
            continue
        a_ok, b_ok = oa["type"] == "deliver", ob["type"] == "deliver"
        if b_ok and not a_ok:
            key = (2, -len(b_run.hops[pid]))
        elif a_ok and b_ok and ob["t"] < oa["t"]:
            key = (1, (oa["t"] - qa["t"]) - (ob["t"] - qa["t"]))
        else:
            continue
        if best_key is None or key > best_key:
            best, best_key = pid, key
    return best


def comparison_frames(runs, pid):
    per = []
    for run in runs:
        q = run.queries[pid]
        color = METHOD_COLOR.get(run.method, BLUE)
        fr = packet_frames(run, pid, 24, color, q["t"], "", "", hold=1)
        busy_nodes = [n for n in run.pos if run.busy(n, q["t"])]
        for f in fr:
            f["busy"] = busy_nodes
        per.append(fr)
    n = max(len(f) for f in per)
```

```
    frames = []
    q = runs[0].queries[pid]
    for k in range(n + 60):
        panels = [fr[min(k, len(fr) - 1)] for fr in per]
        frames.append({"panels": panels,
                       "title": "Selected example, chosen because the paths differ: greedy vs learned",
                       "subtitle": f"Intent: \"{q['text']}\"   (orange = busy router at that moment)"})
    return frames


def render(fig, animator, frames, name, gif_dpi=60):
    anim = animation.FuncAnimation(fig, lambda i: animator.update(frames[i]), frames=len(frames),
                                   interval=1000 / FPS, blit=False)
    OUT.mkdir(parents=True, exist_ok=True)
    try:
        import imageio_ffmpeg
        plt.rcParams["animation.ffmpeg_path"] = imageio_ffmpeg.get_ffmpeg_exe()
        anim.save(OUT / f"{name}.mp4", writer=animation.FFMpegWriter(fps=FPS, bitrate=2400), dpi=100)
        print("wrote", OUT / f"{name}.mp4")
    except Exception as exc:  # ffmpeg missing: GIF only
        print("mp4 skipped:", exc)
    anim.save(OUT / f"{name}.gif", writer=animation.PillowWriter(fps=FPS), dpi=gif_dpi)
    print("wrote", OUT / f"{name}.gif", f"({len(frames) / FPS:.1f} s)")
    # poster frame for the slides: the most informative frame (end of the request scene)
    if "panels" in frames[0]:
        poster = len(frames) - 1
    else:
        delivered = [i for i, f in enumerate(frames) if f.get("labels")]
        poster = delivered[0] if delivered else len(frames) // 2
    animator.update(frames[poster])
    fig.savefig(OUT / f"{name}_poster.png", dpi=100)


def make_demo(method):
    run = Run(method)
    fig = plt.figure(figsize=(12.8, 7.2))
    fig.patch.set_facecolor(SURFACE)
    ax = fig.add_axes([0.0, 0.03, 0.60, 0.82])
    info = fig.add_axes([0.62, 0.06, 0.36, 0.78])
    draw_static(ax, run)
    steps = ["1  Advertise capabilities", "2  Route the request by meaning", "3  Handover: local
update"]
    animator = Animator(fig, [ax], [run], info_ax=info, steps=steps)
    render(fig, animator, demo_frames(run), "agentroute_demo")
    plt.close(fig)


def make_comparison(baseline, learned):
    a, b = Run(baseline), Run(learned)
    pid = pick_comparison(a, b)
    if pid is None:
        print("no request where the learned policy beats greedy in the demo log; comparison skipped")
        return None
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 7.2))
    fig.patch.set_facecolor(SURFACE)
    fig.subplots_adjust(left=0.01, right=0.99, top=0.84, bottom=0.08, wspace=0.03)
    for ax, run in zip(axes, (a, b)):
        draw_static(ax, run, METHOD_LABEL.get(run.method, run.method))
    animator = Animator(fig, list(axes), [a, b])
    render(fig, animator, comparison_frames([a, b], pid), "greedy_vs_agentroute")
    plt.close(fig)
    # what the video shows, for the slide notes
    info = {"pid": pid, "text": a.queries[pid]["text"]}
    for run in (a, b):
        out, q = run.outcome[pid], run.queries[pid]
        info[run.method] = {"type": out["type"], "reason": out.get("reason"),
                            "hops": len(run.hops.get(pid, [])),
                            "time_ms": round((out["t"] - q["t"]) * 1000, 1),
                            "agent_class": run.agents[out["agent"]]["class_name"] if out["type"] ==
"deliver" else None}
    (OUT / "greedy_vs_agentroute.json").write_text(json.dumps(info, indent=2))
    return pid


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--only", choices=["demo", "compare"], default=None)
```

```
    p.add_argument("--method", default="agentroute")
    args = p.parse_args()
    if args.only in (None, "demo"):
        make_demo(args.method)
    if args.only in (None, "compare"):
        pid = make_comparison("greedy", args.method)
        if pid is not None:
            print("comparison uses request", pid)


if __name__ == "__main__":
    main()
```

viz/app_streamlit.py

```python
"""Interactive AgentRoute simulator (Streamlit).

    streamlit run viz/app_streamlit.py

Pick the scenario in the sidebar, run the simulation for several methods,
then compare the results, replay single requests on the network, browse the
experiment figures, or export the run to the React log player.

Written with AI assistance (visualisation).
"""
import json
import sys
from pathlib import Path

import altair as alt
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import pandas as pd  # noqa: E402
import streamlit as st  # noqa: E402

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "viz"))

from agentroute.config import load_config  # noqa: E402
from agentroute.sim.simulator import Simulator  # noqa: E402
from animate import BLUE, CRITICAL, GOOD, SERIOUS, Run, draw_static  # noqa: E402
from export_web_data import export_run, write_index  # noqa: E402

LOG_ROOT = ROOT / "results" / "logs" / "streamlit"
FIGURES = ROOT / "results" / "figures"
LABELS = {"agentroute": "AgentRoute (DQN)", "agentroute_gnn": "AgentRoute (GNN)",
          "greedy": "Greedy semantic", "greedy_load": "Load-aware greedy", "spfr": "SPFR-style",
          "flooding": "Flooding", "registry": "Central registry", "registry3": "Registry (3 replicas)",
          "chord": "Chord (exact)", "sdht": "Semantic DHT"}
COLORS = {"agentroute": "#2a78d6", "agentroute_gnn": "#eb6834", "greedy": "#1baf7a",
          "greedy_load": "#0f8a8a", "spfr": "#6250d6", "flooding": "#eda100", "registry": "#e87ba4",
          "registry3": "#a8437a", "chord": "#008300", "sdht": "#7a6a5a"}
REASONS = ["wrong_agent", "not_found", "ttl", "queue", "link", "node_down", "handover",
           "stale_location", "registry_queue", "no_route"]

st.set_page_config(page_title="AgentRoute simulator", layout="wide")


def models_available():
    cfg = load_config()
    return {"agentroute": (ROOT / cfg["dqn"]["model_path"]).exists(),
            "agentroute_gnn": (ROOT / cfg["dqn"]["gnn_model_path"]).exists()}


# ---------------------------------------------------------------- sidebar
st.sidebar.header("Scenario")
n_agents = st.sidebar.slider("Number of agents", 100, 3000, 300, step=100)
avail = models_available()
options = [m for m in LABELS if avail.get(m, True)]
methods = st.sidebar.multiselect("Methods", options, default=[m for m in
                                 ["agentroute", "greedy", "flooding", "registry"] if m in options],
                                 format_func=LABELS.get)
if not all(avail.values()):
    st.sidebar.caption("Train the DQN first (experiments/run_train.py) to enable AgentRoute.")
radius = st.sidebar.slider("Advertisement radius R (hops)", 1, 6, 3)
summary = st.sidebar.selectbox("S-FIB summary", ["kmeans", "mean", "exact"])
speed = st.sidebar.slider("Mobile agent speed (m/s)", 0.0, 30.0, 5.0, step=1.0)
load = st.sidebar.slider("Network load factor", 0.5, 4.0, 1.0, step=0.5)
failure = st.sidebar.slider("Failed routers (fraction)", 0.0, 0.3, 0.0, step=0.05)
duration = st.sidebar.slider("Simulated time (s)", 10, 60, 20, step=5)
seed = st.sidebar.number_input("Seed", 1, 9999, 1)
log_events = st.sidebar.checkbox("Log every event (needed for the replay)", value=n_agents <= 1000)
run_clicked = st.sidebar.button("Run simulation", type="primary", use_container_width=True)
```

```
st.title("AgentRoute: routing packets to capabilities")
st.caption("Packets are addressed to what an agent can do; routers learn the next hop. "
           "Each run uses the same topology, mobility and requests for every method.")

if run_clicked and methods:
    overrides = {"agents": {"n_agents": n_agents, "speed_mps": speed},
                 "sfib": {"radius": radius, "summary": summary},
                 "sim": {"duration_s": float(duration), "load": load, "failure_fraction": failure}}
    rows = []
    progress = st.progress(0.0, text="Simulating ...")
    for i, m in enumerate(methods):
        cfg = load_config(overrides=overrides)
        log_dir = LOG_ROOT / m if log_events else None
        rows.append(Simulator(cfg, m, seed=int(seed), log_dir=log_dir).run())
        progress.progress((i + 1) / len(methods), text=f"{LABELS[m]} done")
    progress.empty()
    st.session_state["results"] = pd.DataFrame(rows)
    st.session_state["logged"] = methods if log_events else []

res = st.session_state.get("results")
tab_res, tab_net, tab_fig, tab_web = st.tabs(["Results", "Network replay", "Experiment figures",
                                              "Web player"])

# ---------------------------------------------------------------- results
with tab_res:
    if res is None:
        st.info("Choose a scenario in the sidebar and press **Run simulation**.")
    else:
        df = res.copy()
        df["Method"] = df["method"].map(LABELS)
        cols = st.columns(len(df))
        for col, (_, r) in zip(cols, df.iterrows()):
            col.metric(r["Method"], f"{r['success_rate'] * 100:.1f}%",
                       help="Discovery success rate (request delivered to a capable agent)")
        table = df[["Method", "n_queries", "success_rate", "time_ms", "hops", "msgs_per_query",
                    "handovers"]].rename(columns={
                        "n_queries": "Requests", "success_rate": "Success", "time_ms": "Time (ms)",
                        "hops": "Hops", "msgs_per_query": "Messages / request",
                        "handovers": "Handovers"})
        st.dataframe(table.style.format({"Success": "{:.1%}", "Time (ms)": "{:.1f}", "Hops": "{:.2f}",
                                         "Messages / request": "{:.0f}"}),
                     hide_index=True, use_container_width=True)
        scale = alt.Scale(domain=[LABELS[m] for m in COLORS], range=list(COLORS.values()))
        c1, c2 = st.columns(2)
        base = alt.Chart(df).encode(
            y=alt.Y("Method:N", sort=list(df["Method"]), title=None),
            color=alt.Color("Method:N", scale=scale, legend=None))
        c1.altair_chart(base.mark_bar(cornerRadiusEnd=4, size=18).encode(
            x=alt.X("success_rate:Q", title="Success rate", axis=alt.Axis(format="%")),
            tooltip=["Method", alt.Tooltip("success_rate:Q", format=".1%")]).properties(
            title="Discovery success"), use_container_width=True)
        c2.altair_chart(base.mark_bar(cornerRadiusEnd=4, size=18).encode(
            x=alt.X("time_ms:Q", title="Completion time (ms)"),
            tooltip=["Method", alt.Tooltip("time_ms:Q", format=".1f")]).properties(
            title="Task completion time"), use_container_width=True)
        long = df.melt(id_vars=["Method"], value_vars=[f"drop_{r}" for r in REASONS if f"drop_{r}" in
df],
                       var_name="reason", value_name="share")
        long = long[long["share"] > 0]
        long["reason"] = long["reason"].str.replace("drop_", "").str.replace("_", " ")
        st.altair_chart(alt.Chart(long).mark_bar(size=18).encode(
            y=alt.Y("Method:N", title=None), x=alt.X("share:Q", title="Share of requests",
                                                     axis=alt.Axis(format="%")),
            color=alt.Color("reason:N", title="Failure reason"),
            tooltip=["Method", "reason", alt.Tooltip("share:Q", format=".1%")]).properties(
            title="Why requests fail"), use_container_width=True)

# ---------------------------------------------------------------- replay
with tab_net:
    logged = st.session_state.get("logged", [])
    if not logged:
        st.info("Run a simulation with event logging to replay single requests here.")
    else:
        m = st.selectbox("Method", logged, format_func=LABELS.get, key="replay_method")
        run = Run(m, LOG_ROOT / m)
        pids = [p for p in run.queries if p in run.outcome]
        only_ok = st.checkbox("Only delivered requests", value=True)
```

```
        if only_ok:
            pids = [p for p in pids if run.outcome[p]["type"] == "deliver"]
        if pids:
            pid = st.select_slider("Request id", options=pids, value=pids[min(10, len(pids) - 1)])
            q, out = run.queries[pid], run.outcome[pid]
            st.write(f"**Intent:** \"{q['text']}\" (class `{run.classes[q['cls']]}`) - "
                     f"**{out['type']}**" + (f" to agent {out['agent']} "
                                             f"({run.agents[out['agent']]['class_name']})"
                                             if out["type"] == "deliver" else f" ({out['reason']})") +
                     f", {(out['t'] - q['t']) * 1000:.1f} ms, {len(run.hops.get(pid, []))} hops")
            fig, ax = plt.subplots(figsize=(8, 6))
            draw_static(ax, run)
            for a in run.agents:
                x, y = run.agent_xy(a, q["t"])
                ax.scatter(x, y, s=8, color="#898781", alpha=0.6, lw=0, zorder=3)
            for node in run.pos:
                if run.busy(node, q["t"]):
                    ax.scatter(*run.pos[node], s=600, color=SERIOUS, alpha=0.35, lw=0, zorder=2)
            path = run.path(pid)
            for a, b in zip(path, path[1:]):
                (x1, y1), (x2, y2) = run.pos[a], run.pos[b]
                ax.annotate("", xy=(x2, y2), xytext=(x1, y1), zorder=6,
                            arrowprops=dict(arrowstyle="-|>", color=COLORS.get(m, BLUE), lw=2.5))
            ax.scatter(*run.agent_xy(q["agent"], q["t"]), s=160, facecolor="none",
                       edgecolor=COLORS.get(m, BLUE), lw=2.5, zorder=7)
            if out["type"] == "deliver":
                ax.scatter(*run.agent_xy(out["agent"], out["t"]), s=260, facecolor="none",
                           edgecolor=GOOD, lw=2.5, zorder=7)
            elif out.get("at") is not None:
                ax.scatter(*run.pos[out["at"]], s=300, facecolor="none", edgecolor=CRITICAL, lw=2.5,
                           zorder=7)
            st.pyplot(fig, use_container_width=True)
            st.caption("Ring = requester; green ring = serving agent; red ring = where it was dropped;
"
                       "orange halo = router busy with background traffic at that moment.")
        else:
            st.warning("No matching requests in this run.")

# ---------------------------------------------------------------- figures
with tab_fig:
    figs = sorted(FIGURES.glob("*.png"))
    if not figs:
        st.info("Run the experiments and viz/plots.py to create the figures.")
    for i in range(0, len(figs), 2):
        cols = st.columns(2)
        for col, f in zip(cols, figs[i:i + 2]):
            col.image(str(f), caption=f.stem.replace("_", " "), use_container_width=True)

# ---------------------------------------------------------------- web player
with tab_web:
    st.write("The React + D3 player (viz/web) replays any logged run in the browser, "
             "with timeline controls and per-request delay waterfalls.")
    logged = st.session_state.get("logged", [])
    if logged and st.button("Export this run to the web player"):
        for m in logged:
            export_run(LOG_ROOT / m, f"streamlit_{m}")
        runs = write_index()
        st.success(f"Exported. The player now lists {len(runs)} runs.")
    st.code("cd viz/web\nnpm install\nnpm run dev", language="bash")
    index = ROOT / "viz" / "web" / "public" / "data" / "index.json"
    if index.exists():
        st.json(json.loads(index.read_text()), expanded=False)
```

viz/export_web_data.py

```
"""Copy simulation logs into the React player's public/data folder.

    python viz/export_web_data.py                # every run under results/logs/demo
    python viz/export_web_data.py --src results/logs/streamlit --name my_run

Each run becomes public/data/<name>/{topology.json, events.jsonl, summary.json}
and is listed in public/data/index.json.

Written with AI assistance (visualisation tooling).
"""
import argparse
import json
import shutil
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
WEB_DATA = ROOT / "viz" / "web" / "public" / "data"


def export_run(src, name):
    dst = WEB_DATA / name
    dst.mkdir(parents=True, exist_ok=True)
    for f in ("topology.json", "events.jsonl", "summary.json"):
        if (src / f).exists():
            shutil.copy(src / f, dst / f)
    summary = json.loads((src / "summary.json").read_text()) if (src / "summary.json").exists() else {}
    return {"name": name, "method": summary.get("method", name),
            "n_agents": summary.get("n_agents"), "success_rate": summary.get("success_rate")}


def write_index():
    runs = []
    for d in sorted(WEB_DATA.iterdir()) if WEB_DATA.exists() else []:
        if d.is_dir() and (d / "events.jsonl").exists():
            s = json.loads((d / "summary.json").read_text()) if (d / "summary.json").exists() else {}
            runs.append({"name": d.name, "method": s.get("method", d.name),
                         "n_agents": s.get("n_agents"), "success_rate": s.get("success_rate")})
    (WEB_DATA / "index.json").write_text(json.dumps({"runs": runs}, indent=2))
    return runs


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--src", default=str(ROOT / "results" / "logs" / "demo"))
    p.add_argument("--name", default=None, help="export a single run directory under this name")
    args = p.parse_args()
    src = Path(args.src)
    WEB_DATA.mkdir(parents=True, exist_ok=True)
    if args.name:
        export_run(src, args.name)
    else:
        for d in sorted(src.iterdir()):
            if d.is_dir() and (d / "events.jsonl").exists():
                export_run(d, f"demo_{d.name}")
    runs = write_index()
    print(f"{len(runs)} runs in {WEB_DATA / 'index.json'}")


if __name__ == "__main__":
    main()
```

viz/plots.py

```
"""All figures of the report and the slides, from results/raw/*.csv.

    python viz/plots.py

Style: one fixed colour per method (never re-ordered between charts), marker
shapes as a second identity channel, hairline grid, mean with a 95% CI band.

Written with AI assistance (plotting).
"""
import json
import sys
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
RAW = ROOT / "results" / "raw"
FIG = ROOT / "results" / "figures"

# reference categorical palette, slots 1-6 in fixed order
METHODS = {
    "agentroute":     ("AgentRoute (DQN)",      "#2a78d6", "o"),
    "agentroute_gnn": ("AgentRoute (GNN)",      "#eb6834", "D"),
    "greedy":         ("Greedy semantic",       "#1baf7a", "s"),
    "greedy_load":    ("Load-aware greedy",     "#0f8a8a", "P"),
    "spfr":           ("SPFR-style",            "#6250d6", "*"),
    "flooding":       ("Flooding",              "#eda100", "^"),
    "registry":       ("Central registry",      "#e87ba4", "v"),
    "registry3":      ("Registry (3 replicas)", "#a8437a", "<"),
    "sdht":           ("Semantic DHT",          "#7a6a5a", "h"),
    "chord":          ("Chord (exact)",         "#008300", "X"),
}
# the GNN is close to the DQN and would only clutter the main charts
MAIN = [m for m in METHODS if m != "agentroute_gnn"]
ROUTED = [m for m in MAIN if m != "chord"]      # Chord never succeeds, no hop/time data
INK, INK2, MUTED = "#0b0b0b", "#52514e", "#898781"
GRID, AXIS, SURFACE = "#e1e0d9", "#c3c2b7", "#fcfcfb"

plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": ["Helvetica Neue", "Helvetica", "Arial", "DejaVu Sans"],
    "font.size": 10,
    "axes.edgecolor": AXIS,
    "axes.labelcolor": INK2,
    "axes.titlecolor": INK,
    "axes.titlesize": 11,
    "axes.titleweight": "bold",
    "axes.titlelocation": "left",
    "axes.facecolor": SURFACE,
    "figure.facecolor": SURFACE,
    "xtick.color": MUTED,
    "ytick.color": MUTED,
    "xtick.labelcolor": INK2,
    "ytick.labelcolor": INK2,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "axes.grid.axis": "y",
    "grid.color": GRID,
    "grid.linewidth": 0.8,
    "legend.frameon": False,
    "legend.fontsize": 9,
    "lines.solid_capstyle": "round",
    "lines.solid_joinstyle": "round",
})
```

```
def save(fig, name):
    FIG.mkdir(parents=True, exist_ok=True)
    fig.savefig(FIG / f"{name}.png", dpi=200, bbox_inches="tight")
    fig.savefig(FIG / f"{name}.pdf", bbox_inches="tight")
    plt.close(fig)
    print("wrote", FIG / f"{name}.png")


def ci95(x):
    x = np.asarray(x, dtype=float)
    x = x[~np.isnan(x)]
    if len(x) < 2:
        return 0.0
    return 1.96 * x.std(ddof=1) / np.sqrt(len(x))


def agg(df, by, col):
    g = df.groupby([by, "method"])[col]
    out = g.mean().to_frame("mean")
    out["ci"] = g.apply(ci95)
    return out.reset_index()


def line_panel(ax, df, by, col, methods, ylabel, logx=False, logy=False, pct=False):
    data = agg(df, by, col)
    for m in methods:
        if m not in data["method"].values:
            continue
        label, color, marker = METHODS[m]
        d = data[data["method"] == m].sort_values(by)
        d = d[~d["mean"].isna()]
        if d.empty:
            continue
        y = d["mean"] * (100 if pct else 1)
        ci = d["ci"] * (100 if pct else 1)
        ax.fill_between(d[by], y - ci, y + ci, color=color, alpha=0.10, linewidth=0)
        ax.plot(d[by], y, color=color, linewidth=1.8, marker=marker, markersize=6,
                markeredgecolor=SURFACE, markeredgewidth=1.2, label=label)
    if logx:
        ax.set_xscale("log")
    if logy:
        ax.set_yscale("log")
    ax.set_ylabel(ylabel)
    if pct:
        ax.set_ylim(bottom=0)


def legend_above(fig, axes, n_cols=3):
    handles, labels = [], []
    for ax in np.atleast_1d(axes):
        for h, l in zip(*ax.get_legend_handles_labels()):
            if l not in labels:
                handles.append(h)
                labels.append(l)
    fig.legend(handles, labels, loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=n_cols,
               handlelength=2.2, columnspacing=1.6)


def agent_ticks(ax, values):
    ax.set_xticks(values)
    ax.set_xticklabels([f"{v // 1000}K" if v >= 1000 else str(v) for v in values])
    ax.minorticks_off()


def load_csv(name):
    path = RAW / f"{name}.csv"
    if not path.exists():
        print("skip (missing)", path)
        return None
    return pd.read_csv(path)


# ---------------------------------------------------------------------------
def oracle_line(ax, df, by, factor=1.0):
    """What a method with perfect knowledge of all attached agents could achieve."""
    g = df.groupby(by)["oracle_best"].mean() * 100
```

```
    ax.plot(np.asarray(g.index, dtype=float) * factor, g.values, color=INK, linewidth=1.1,
            linestyle=(0, (2, 2)), label="Matching ceiling (oracle)")


def fig_scale():
    df = load_csv("scale")
    if df is None:
        return
    by = "n_agents_param"
    values = sorted(df[by].unique())

    fig, ax = plt.subplots(figsize=(7.2, 4.0))
    line_panel(ax, df, by, "success_rate", MAIN, "Discovery success (%)", logx=True, pct=True)
    oracle_line(ax, df, by)
    agent_ticks(ax, values)
    ax.set_xlabel("Number of agents")
    ax.set_title("Success rate vs number of agents")
    legend_above(fig, ax, n_cols=4)
    save(fig, "scale_success")

    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.8))
    line_panel(axes[0], df, by, "ctrl_kB_per_s", [m for m in ROUTED if m != "flooding"],
               "Control traffic, whole network (kB/s, log)", logx=True, logy=True)
    line_panel(axes[1], df, by, "query_kB_per_query", ROUTED, "Query traffic per request (kB, log)",
               logx=True, logy=True)
    for ax, title in zip(axes, ["Paid per second (advertisements, registrations)",
                                "Paid per request (lookups, floods, forwarding)"]):
        agent_ticks(ax, values)
        ax.set_xlabel("Number of agents")
        ax.set_title(title)
    legend_above(fig, axes, n_cols=4)
    fig.tight_layout()
    save(fig, "scale_overhead")

    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))
    line_panel(axes[0], df, by, "hops", ROUTED, "Hops per delivered request", logx=True)
    line_panel(axes[1], df, by, "time_ms", [m for m in ROUTED if m != "flooding"],
               "Completion time (ms)", logx=True)
    for ax, title in zip(axes, ["Hop count", "Task completion time (flooding: 230+ ms, not shown)"]):
        agent_ticks(ax, values)
        ax.set_xlabel("Number of agents")
        ax.set_title(title)
        ax.set_ylim(bottom=0)
    legend_above(fig, axes, n_cols=4)
    fig.tight_layout()
    save(fig, "scale_hops_time")


def fig_simple(name, by, xlabel, title, fname, pct_x=False):
    df = load_csv(name)
    if df is None:
        return
    fig, ax = plt.subplots(figsize=(7.2, 4.0))
    x = df.copy()
    if pct_x:
        x[by] = x[by] * 100
    line_panel(ax, x, by, "success_rate", MAIN, "Discovery success (%)", pct=True)
    oracle_line(ax, x, by)
    ax.set_xlabel(xlabel)
    ax.set_title(title)
    legend_above(fig, ax, n_cols=4)
    save(fig, fname)


def fig_mobility_cost():
    a, b = load_csv("mobility"), load_csv("mobility_small")
    if a is None or b is None:
        return
    methods = ["agentroute", "greedy", "spfr", "registry3", "sdht"]
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))
    for ax, df, title in zip(axes, (a, b), ("500 m cells", "150 m cells")):
        line_panel(ax, df, "speed", "ctrl_kB_per_s", methods, "Control traffic (kB/s, log)", logy=True)
        ax.set_xlabel("Mobile agent speed (m/s)")
        ax.set_title(f"Cost of mobility, {title}")
    legend_above(fig, axes, n_cols=5)
    fig.tight_layout()
    save(fig, "mobility_cost")
```

```
def fig_radius():
    df = load_csv("radius")
    if df is None:
        return
    methods = ["agentroute", "greedy", "greedy_load"]
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))
    line_panel(axes[0], df, "radius", "success_rate", methods, "Discovery success (%)", pct=True)
    oracle_line(axes[0], df, "radius")
    line_panel(axes[1], df, "radius", "ctrl_kB_per_s", methods, "Control traffic (kB/s, log)",
logy=True)
    for ax, title in zip(axes, ["Success vs advertisement radius R", "Control overhead vs R"]):
        ax.set_xlabel("Advertisement radius R (hops)")
        ax.set_title(title)
        ax.set_xticks(sorted(df["radius"].unique()))
    legend_above(fig, axes)
    fig.tight_layout()
    save(fig, "radius")


def fig_load():
    df = load_csv("load")
    if df is None:
        return
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))
    line_panel(axes[0], df, "load", "success_rate", MAIN, "Discovery success (%)", pct=True)
    oracle_line(axes[0], df, "load")
    line_panel(axes[1], df, "load", "time_ms", [m for m in ROUTED if m != "flooding"],
               "Completion time (ms)")
    axes[1].set_ylim(bottom=0)
    for ax, title in zip(axes, ["Success vs network load", "Completion time vs network load"]):
        ax.set_xlabel("Load factor (requests and background traffic)")
        ax.set_title(title)
    legend_above(fig, axes, n_cols=4)
    fig.tight_layout()
    save(fig, "load")


def fig_rate():
    df = load_csv("rate")
    if df is None:
        return
    df = df.copy()
    df["total_kB_per_s"] = df["ctrl_kB_per_s"] + df["query_kB_per_query"] * df["rate"]
    methods = [m for m in MAIN if m in df["method"].unique()]
    fig, ax = plt.subplots(figsize=(7.2, 4.0))
    line_panel(ax, df, "rate", "total_kB_per_s", methods, "Total traffic, whole network (kB/s, log)",
               logx=True, logy=True)
    ax.set_xlabel("Requests per second in the whole network (log)")
    ax.set_title("Total traffic vs request rate (1,000 agents)")
    ax.set_xticks(sorted(df["rate"].unique()))
    ax.set_xticklabels([f"{v:g}" for v in sorted(df["rate"].unique())])
    ax.minorticks_off()
    legend_above(fig, ax, n_cols=4)
    save(fig, "rate_crossover")


def fig_tau():
    df = load_csv("tau")
    if df is None:
        return
    fig, ax = plt.subplots(figsize=(7.2, 4.0))
    for m in [m for m in MAIN if m in df["method"].unique()]:
        label, color, marker = METHODS[m]
        d = df[df["method"] == m].groupby("tau")[["success_rate", "precision"]].mean().sort_index()
        ax.plot((1 - d["precision"]) * 100, d["success_rate"] * 100, color=color, linewidth=1.6,
                marker=marker, markersize=6, markeredgecolor=SURFACE, markeredgewidth=1.0, label=label)
    tau = json.loads((ROOT / "results" / "tau.json").read_text())["tau"]
    ax.set_xlabel("Wrong deliveries among all deliveries (%)")
    ax.set_ylabel("Discovery success (%)")
    ax.set_title(f"Threshold tau: more success costs more wrong deliveries (we use tau = {tau})")
    ax.grid(axis="x")
    legend_above(fig, ax, n_cols=3)
    save(fig, "tau_tradeoff")
```

```python
def fig_matched():
    path = RAW / "matched.json"
    if not path.exists():
        return
    data = json.loads(path.read_text())
    pairs = [("agentroute vs greedy", "DQN vs greedy"), ("agentroute vs greedy_load", "DQN vs load-
aware"),
             ("agentroute vs spfr", "DQN vs SPFR-style")]
    names = list(data)
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))
    width = 0.8 / len(pairs)
    colors = ["#1baf7a", "#0f8a8a", "#6250d6"]
    for i, ((key, label), color) in enumerate(zip(pairs, colors)):
        x = np.arange(len(names)) + (i - (len(pairs) - 1) / 2) * width
        diff = [data[n][key]["success_diff_points"] for n in names]
        lo = [data[n][key]["success_diff_ci95"][0] for n in names]
        hi = [data[n][key]["success_diff_ci95"][1] for n in names]
        err = [np.array(diff) - np.array(lo), np.array(hi) - np.array(diff)]
        axes[0].bar(x, diff, width=width * 0.92, color=color, label=label, yerr=err,
                    error_kw={"ecolor": INK2, "elinewidth": 0.8, "capsize": 2})
        t = [data[n][key]["time_ms"][0] - data[n][key]["time_ms"][1] for n in names]
        axes[1].bar(x, t, width=width * 0.92, color=color)
    for ax in axes:
        ax.axhline(0, color=AXIS, linewidth=1.0)
        ax.set_xticks(np.arange(len(names)))
        ax.set_xticklabels([n.replace(", ", "\n") for n in names])
    axes[0].set_ylabel("Success difference (points)")
    axes[0].set_title("Success: DQN minus the other method (95% CI)")
    axes[1].set_ylabel("Time difference (ms)")
    axes[1].set_title("Time on requests both delivered (negative = DQN faster)")
    legend_above(fig, axes, n_cols=3)
    fig.tight_layout()
    save(fig, "matched")


def smooth(y, w=15):
    y = np.asarray(y, dtype=float)
    if len(y) < w:
        return y
    k = np.ones(w) / w
    pad = np.concatenate([np.full(w // 2, y[0]), y, np.full(w - 1 - w // 2, y[-1])])
    return np.convolve(pad, k, mode="valid")


def fig_learning():
    """Validation only (no exploration). The training return would rise anyway as epsilon falls."""
    curves = {k: load_csv(f"train_{k}") for k in ("mlp", "gnn")}
    curves = {k: v for k, v in curves.items() if v is not None}
    if not curves:
        return
    fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.6))
    keys = {"mlp": "agentroute", "gnn": "agentroute_gnn"}
    for kind, df in curves.items():
        label, color, marker = METHODS[keys[kind]]
        val = df.dropna(subset=["val_success"])
        for ax, col, factor in zip(axes, ("val_return", "val_success"), (1, 100)):
            ax.plot(val["episode"], val[col] * factor, color=color, linewidth=1.8, marker=marker,
                    markersize=5, markeredgecolor=SURFACE, markeredgewidth=1.0, label=label)
    g = next(iter(curves.values()))
    for method, ret, succ in (("greedy", "greedy_val_return", "greedy_val_success"),
                              ("greedy_load", "load_val_return", "load_val_success")):
        label, color, _ = METHODS[method]
        axes[0].axhline(g[ret].iloc[0], color=color, linewidth=1.8, linestyle=(0, (4, 3)), label=f"
{label} (reference)")
        axes[1].axhline(g[succ].iloc[0] * 100, color=color, linewidth=1.8, linestyle=(0, (4, 3)))
    axes[0].set_title("Validation return per request")
    axes[0].set_ylabel("Return per request")
    axes[1].set_title("Validation success")
    axes[1].set_ylabel("Discovery success (%)")
    for ax in axes:
        ax.set_xlabel("Episode")
    legend_above(fig, axes, n_cols=4)
    fig.tight_layout()
    save(fig, "learning_curve")


def bar_groups(ax, df, by, order, methods, col="success_rate", pct=True):
```

```
    data = agg(df, by, col)
    width = min(0.8 / len(methods), 0.3)
    for i, m in enumerate(methods):
        label, color, _ = METHODS[m]
        d = data[data["method"] == m].set_index(by).reindex(order)
        x = np.arange(len(order)) + (i - (len(methods) - 1) / 2) * (width + 0.02)
        factor = 100 if pct else 1
        ax.bar(x, d["mean"] * factor, width=width, color=color, label=label, edgecolor=SURFACE,
               linewidth=1.0, yerr=d["ci"] * factor, error_kw={"ecolor": INK2, "elinewidth": 0.8,
                                                               "capsize": 2})
    ax.set_xticks(np.arange(len(order)))
    ax.set_xticklabels(order)
    if pct:
        ax.set_ylabel("Discovery success (%)")


def fig_ablation():
    df = load_csv("ablation_summary")
    if df is not None:
        order = [x for x in ["mean", "k=8, no tier", "k=2", "k=4", "k=8", "k=16", "k=32", "exact"]
                 if x in df["summary"].unique()]
        methods = ["agentroute", "greedy", "greedy_load"]
        fig, axes = plt.subplots(1, 2, figsize=(10.4, 3.8))
        bar_groups(axes[0], df, "summary", order, methods)
        axes[0].set_title("S-FIB content vs success")
        axes[0].tick_params(axis="x", labelrotation=30)
        bar_groups(axes[1], df, "summary", order, methods, col="ctrl_kB_per_s", pct=False)
        axes[1].set_ylabel("Control traffic (kB/s, log)")
        axes[1].set_yscale("log")
        axes[1].set_title("S-FIB content vs control traffic")
        axes[1].tick_params(axis="x", labelrotation=30)
        legend_above(fig, axes, n_cols=3)
        fig.tight_layout()
        save(fig, "ablation_summary")
    df = load_csv("ablation_header")
    if df is not None:
        order = [h for h in ["float32", "int8", "binary"] if h in df["header"].unique()]
        fig, ax = plt.subplots(figsize=(5.6, 3.6))
        bar_groups(ax, df, "header", order, ["agentroute", "greedy", "greedy_load"])
        sizes = {"float32": "1536 B", "int8": "388 B", "binary": "48 B"}
        ax.set_xticklabels([f"{h}\n{sizes[h]}" for h in order])
        ax.set_xlabel("Intent header encoding")
        ax.set_title("Effect of compressing the intent header")
        legend_above(fig, ax, n_cols=3)
        save(fig, "ablation_header")


def fig_drop_reasons():
    df = load_csv("scale")
    if df is None:
        return
    d = df[df["n_agents_param"] == 1000]
    if d.empty:
        return
    reasons = {"wrong_agent": "Wrong agent accepted", "not_found": "No match found",
               "ttl": "TTL expired", "queue": "Queue overflow", "link": "Link loss",
               "stale_location": "Stale location", "handover": "Agent in handover",
               "node_down": "Node down", "registry_queue": "Registry overloaded",
               "no_route": "No route"}
    present = [r for r in reasons if f"drop_{r}" in d and d[f"drop_{r}"].mean() > 0.002]
    palette = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#008300", "#6250d6", "#e34948"]
    methods = MAIN
    means = d.groupby("method")[[f"drop_{r}" for r in present] +
["success_rate"]].mean().reindex(methods)
    fig, ax = plt.subplots(figsize=(7.2, 3.8))
    y = np.arange(len(methods))
    left = np.zeros(len(methods))
    for i, r in enumerate(present[:8]):
        vals = means[f"drop_{r}"].values * 100
        ax.barh(y, vals, left=left, height=0.55, color=palette[i], label=reasons[r],
                edgecolor=SURFACE, linewidth=1.0)
        left += vals
    ax.set_yticks(y)
    ax.set_yticklabels([f"{METHODS[m][0]}  ({s * 100:.0f}% success)"
                        for m, s in zip(methods, means["success_rate"].values)])
    ax.invert_yaxis()
    ax.grid(axis="x")
```

```
    ax.grid(axis="y", visible=False)
    ax.set_xlabel("Share of requests (%)")
    ax.set_title("Why requests fail (1,000 agents)")
    ax.set_xlim(0, 100)
    legend_above(fig, ax, n_cols=3)
    save(fig, "drop_reasons")


def fig_similarity():
    tau_path = ROOT / "results" / "tau.json"
    if not tau_path.exists():
        return
    from agentroute.embeddings import load_dataset_vectors
    data = load_dataset_vectors()
    sims = data["test_vecs"] @ data["desc_vecs"].T
    same = data["test_class"][:, None] == data["desc_class"][None, :]
    tau = json.loads(tau_path.read_text())["tau"]
    fig, ax = plt.subplots(figsize=(6.4, 3.4))
    bins = np.linspace(-0.2, 0.9, 45)
    ax.hist(sims[~same], bins=bins, density=True, color="#898781", alpha=0.45,
            label="Request vs other-class capability", edgecolor=SURFACE, linewidth=0.6)
    ax.hist(sims[same], bins=bins, density=True, color="#2a78d6", alpha=0.75,
            label="Request vs same-class capability", edgecolor=SURFACE, linewidth=0.6)
    ax.axvline(tau, color=INK, linewidth=1.2)
    ax.text(tau + 0.01, ax.get_ylim()[1] * 0.92, f"tau = {tau:.3f}", color=INK, fontsize=9)
    ax.set_xlabel("Cosine similarity (MiniLM, test paraphrases)")
    ax.set_ylabel("Density")
    ax.set_title("Paraphrased requests vs capability cards")
    legend_above(fig, ax, n_cols=2)
    save(fig, "similarity")


EXPERIMENTS = [("scale", "n_agents_param"), ("mobility", "speed"), ("mobility_small", "speed"),
               ("failure", "failure"), ("radius", "radius"), ("load", "load"), ("rate", "rate"),
               ("tau", "tau"), ("ablation_summary", "summary"), ("ablation_header", "header"),
               ("ablation_accept", "accept"), ("ablation_threshold", "threshold"), ("ablation_window",
"window")]
METRICS = ["success_rate", "oracle_best", "success_solvable", "precision", "time_ms", "hops",
           "msgs_per_query", "ctrl_msgs_per_s", "ctrl_kB_per_s", "query_msgs_per_query",
           "query_kB_per_query", "handovers", "update_msgs"]


def export_tables():
    """Mean of every metric per (value, method): results/summary.json + results/tables.md.

    The report and the slides take their numbers from these files.
    """
    out, md = {}, []
    for name, by in EXPERIMENTS:
        df = load_csv(name)
        if df is None:
            continue
        values = list(dict.fromkeys(df.sort_values(by)[by].tolist()))
        methods = [m for m in METHODS if m in df["method"].unique()]
        g = df.groupby([by, "method"])[METRICS].mean()
        entry = {"param": by, "values": values, "methods": {}}
        for m in methods:
            entry["methods"][m] = {k: [None if pd.isna(g.loc[(v, m), k]) else round(float(g.loc[(v, m),
k]), 4)
                                       for v in values] for k in METRICS}
        drop_cols = [c for c in df.columns if c.startswith("drop_")]
        if name == "scale":
            ref = 1000 if 1000 in values else values[len(values) // 2]
            d = df[df[by] == ref].groupby("method")[drop_cols].mean()
            entry["drops_at"] = ref
            entry["drops"] = {m: {c[5:]: round(float(d.loc[m, c]), 4) for c in drop_cols}
                              for m in methods if m in d.index}
        if name == "failure":
            # the registry's mean hides whether its server happened to be among the failed routers
            entry["registry_by_server"] = {}
            for v in values:
                r = df[(df[by] == v) & (df["method"] == "registry")]
                up, down = r[r["servers_alive"] > 0], r[r["servers_alive"] == 0]
                entry["registry_by_server"][str(v)] = {
                    "up": None if up.empty else round(float(up["success_rate"].mean()), 4), "n_up":
len(up),
                    "down": None if down.empty else round(float(down["success_rate"].mean()), 4),
```

```
"n_down": len(down)}
        out[name] = entry
        md.append(f"\n### {name} (success %, mean of {df['seed'].nunique()} seeds)\n")
        md.append("| method | " + " | ".join(str(v) for v in values) + " |")
        md.append("|---" * (len(values) + 1) + "|")
        for m in methods:
            row = [f"{x * 100:.1f}" for x in entry["methods"][m]["success_rate"]]
            md.append(f"| {METHODS[m][0]} | " + " | ".join(row) + " |")
        for metric, label, fmt in (("oracle_best", "matching ceiling (fraction)", "{:.3f}"),
                                   ("precision", "precision (right / delivered)", "{:.3f}"),
                                   ("time_ms", "time ms", "{:.1f}"), ("hops", "hops", "{:.2f}"),
                                   ("ctrl_kB_per_s", "control kB/s", "{:.0f}"),
                                   ("query_kB_per_query", "query kB / request", "{:.1f}")):
            md.append(f"\n{name}: {label}\n")
            md.append("| method | " + " | ".join(str(v) for v in values) + " |")
            md.append("|---" * (len(values) + 1) + "|")
            for m in methods:
                row = ["-" if x is None else fmt.format(x) for x in entry["methods"][m][metric]]
                md.append(f"| {METHODS[m][0]} | " + " | ".join(row) + " |")
    for kind in ("mlp", "gnn"):
        df = load_csv(f"train_{kind}")
        if df is not None:
            val = df.dropna(subset=["val_success"])
            out[f"train_{kind}"] = {
                "episode": val["episode"].tolist(),
                "val_success": val["val_success"].round(4).tolist(),
                "val_time_ms": val["val_time_ms"].round(2).tolist(),
                "val_return": val["val_return"].round(4).tolist(),
                "greedy_val_success": float(df["greedy_val_success"].iloc[0]),
                "greedy_val_time_ms": float(df["greedy_val_time_ms"].iloc[0]),
                "greedy_val_return": float(df["greedy_val_return"].iloc[0]),
                "load_val_success": float(df["load_val_success"].iloc[0]),
                "load_val_time_ms": float(df["load_val_time_ms"].iloc[0]),
                "load_val_return": float(df["load_val_return"].iloc[0]),
            }
    tau_path = ROOT / "results" / "tau.json"
    if tau_path.exists():
        from agentroute.embeddings import embed_texts, load_dataset_vectors
        data = load_dataset_vectors()
        sims = data["test_vecs"] @ data["desc_vecs"].T
        same = data["test_class"][:, None] == data["desc_class"][None, :]
        edges = np.round(np.arange(-0.2, 0.901, 0.05), 3)
        h_same, _ = np.histogram(sims[same], bins=edges)
        h_other, _ = np.histogram(sims[~same], bins=edges)
        example = embed_texts(["find the cars in this photo"])[0]
        card_sims = data["desc_vecs"] @ example
        order = np.argsort(-card_sims)[:3]
        out["similarity"] = {
            "tau": json.loads(tau_path.read_text()),
            "bins": [f"{0.0 if abs(a) < 1e-9 else a:.2f}" for a in edges[:-1]],
            "same_pct": (100 * h_same / h_same.sum()).round(2).tolist(),
            "other_pct": (100 * h_other / h_other.sum()).round(2).tolist(),
            "example": [{"card": data["desc_text"][i], "cos": round(float(card_sims[i]), 3)} for i in
order],
        }
    for name in ("handover_cost", "matched"):
        path = RAW / f"{name}.json"
        if path.exists():
            out[name] = json.loads(path.read_text())
    (ROOT / "results" / "summary.json").write_text(json.dumps(out, indent=1))
    (ROOT / "results" / "tables.md").write_text("\n".join(md) + "\n")
    print("wrote results/summary.json and results/tables.md")


def main():
    export_tables()
    fig_similarity()
    fig_scale()
    fig_simple("mobility", "speed", "Mobile agent speed (m/s)", "Success rate vs mobility (500 m
cells)",
               "mobility_success")
    fig_simple("mobility_small", "speed", "Mobile agent speed (m/s)", "Success rate vs mobility (150 m
cells)",
               "mobility_small_success")
    fig_mobility_cost()
    fig_simple("failure", "failure", "Failed routers (%)", "Success rate vs node failures",
               "failure_success", pct_x=True)
```

```
    fig_radius()
    fig_load()
    fig_rate()
    fig_tau()
    fig_matched()
    fig_learning()
    fig_ablation()
    fig_drop_reasons()


if __name__ == "__main__":
    main()
```

tests/test_core.py

```
"""Tests for the dataset, the S-FIB and greedy forwarding."""
import networkx as nx
import numpy as np

from agentroute.data.capabilities import CLASSES, EXAMPLES, capability_cards, split_intents
from agentroute.embeddings import agent_vectors, quantize
from agentroute.forwarding import (FEATURES, LOCAL, N_FEATURES, GreedyPolicy, LoadAwareGreedy,
NodeView, Packet,
                                   SPFRPolicy, accept_rule, candidates, neighbor_features)
from agentroute.sfib import SFIB, Advertiser, kmeans_summary


def _norm(x):
    return (x / np.linalg.norm(x, axis=-1, keepdims=True)).astype(np.float32)


def test_dataset_shape():
    assert len(CLASSES) == 50
    for name, descs, intents in CLASSES:
        assert len(descs) == 2 and len(intents) == 10 and len(EXAMPLES[name]) == 3
    train, test = split_intents()
    assert len(train) == 350 and len(test) == 150
    assert not set(train) & set(test)


def test_intents_never_repeat_capability_text():
    card_sentences = [s.lower() for _, _, card in capability_cards() for s in card]
    for _, _, intents in CLASSES:
        for intent in intents:
            low = intent.lower()
            for s in card_sentences:
                assert low != s
                assert s not in low and low not in s


def test_quantize_keeps_unit_norm():
    v = _norm(np.random.default_rng(0).normal(size=384))
    for header in ("float32", "int8", "binary"):
        q = quantize(v, header)
        assert abs(np.linalg.norm(q) - 1) < 1e-4
        assert q @ v > 0.7


def test_kmeans_summary_small_input_is_exact():
    vecs = _norm(np.eye(3, 384))
    reps, hops, weights = kmeans_summary(vecs, np.ones(3), np.array([1, 2, 3]), k=4)
    assert np.allclose(reps, vecs) and list(hops) == [1, 2, 3] and list(weights) == [1, 1, 1]


def test_kmeans_summary_compresses_to_k():
    vecs = _norm(np.random.default_rng(0).normal(size=(50, 384)))
    reps, hops, weights = kmeans_summary(vecs, np.ones(50), np.ones(50, int), k=4)
    assert len(reps) <= 4 and weights.sum() == 50


def test_every_agent_has_its_own_vector():
    class_vecs = _norm(np.random.default_rng(1).normal(size=(3, 5, 384)))
    v = agent_vectors(class_vecs, np.array([0, 0, 1, 2]), np.random.default_rng(2))
    assert abs(np.linalg.norm(v, axis=1) - 1).max() < 1e-5
    assert v[0] @ v[1] < 0.9999 and v[0] @ v[1] > v[0] @ v[2]


def _vecs(n):
    return _norm(np.eye(n, 384))


def _adv(g, radius, summary="exact", k=8, exact_hops=0):
    return Advertiser(g, 384, radius, summary=summary, k=k, change_threshold=0.01,
                      encoding="float32", exact_hops=exact_hops)


def test_exact_table_respects_radius():
```

```
    # line 0-1-2-3-4, one agent at router 4, radius 2
    adv = _adv(nx.path_graph(5), radius=2)
    adv.local[4][7] = _vecs(1)[0]
    adv.build()
    assert list(adv.tables[3].rows[adv.tables[3].index[4]][1]) == [1]
    assert list(adv.tables[2].rows[adv.tables[2].index[3]][1]) == [2]
    assert len(adv.tables[1].entries()[0]) == 0         # 3 hops away: outside R


def test_summary_distance_vector_respects_radius():
    adv = _adv(nx.path_graph(5), radius=2, summary="kmeans", k=2)
    adv.local[4][7] = _vecs(1)[0]
    adv.build()
    assert list(adv.tables[3].rows[adv.tables[3].index[4]][1]) == [1]
    assert list(adv.tables[2].rows[adv.tables[2].index[3]][1]) == [2]
    assert len(adv.tables[1].entries()[0]) == 0


def test_summary_size_does_not_depend_on_agent_count():
    g = nx.path_graph(3)
    vecs = _norm(np.random.default_rng(0).normal(size=(200, 384)))
    sizes = []
    for n in (20, 200):
        adv = _adv(g, radius=2, summary="kmeans", k=4)
        for a in range(n):
            adv.local[0][a] = vecs[a]
        adv.build()
        sizes.append(adv.tables[1].n_vectors(0))
    assert sizes == [4, 4]


def test_two_tier_table_keeps_nearby_agents_exact():
    g = nx.path_graph(3)
    vecs = _norm(np.random.default_rng(0).normal(size=(10, 384)))
    adv = _adv(g, radius=2, summary="kmeans", k=2, exact_hops=1)
    for a in range(10):
        adv.local[0][a] = vecs[a]
    adv.build()
    assert adv.tables[1].n_vectors(0) == 10         # 1 hop: one row per agent
    assert adv.tables[2].n_vectors(1) == 2          # 2 hops: summarised


def test_handover_withdraw_and_announce():
    v = _vecs(2)
    adv = _adv(nx.path_graph(5), radius=4)
    adv.local[0][5] = v[1]
    adv.build()
    # the agent moves from router 0 to router 4
    adv.local[0].pop(5)
    changed_w = []
    for d in range(1, 5):
        changed, _ = adv.spread(0, [0], {0}, 5, v[1], -1, d)
        changed_w += changed
    adv.local[4][5] = v[1]
    changed_a = []
    for d in range(1, 5):
        changed, _ = adv.spread(4, [4], {4}, 5, v[1], +1, d)
        changed_a += changed
    assert changed_w and changed_a
    assert len(adv.tables[2].rows[adv.tables[2].index[1]][0]) == 0
    assert len(adv.tables[2].rows[adv.tables[2].index[3]][0]) == 1


def test_small_changes_are_suppressed():
    vecs = _norm(np.random.default_rng(0).normal(size=(2, 384)))
    t = SFIB(0, [1], 384, 2, k=1)
    t.set_row(0, vecs[:1].copy(), np.array([1]), np.array([1000.0], np.float32))
    before = t.rows[0][0].copy()
    assert t.apply_delta(0, vecs[1], +1, 1, threshold=0.01) is False    # heavy summary hardly moves
    assert np.allclose(t.rows[0][0], before)
    t.set_row(0, vecs[:1].copy(), np.array([1]), np.array([1.0], np.float32))
    assert t.apply_delta(0, vecs[1], +1, 1, threshold=0.01) is True     # a rare capability does


def _view(adv, node, tau=0.5, accept_mode="best_known", local=None):
    nbrs = sorted(adv.graph.neighbors(node))
    ids = list(local) if local else []
```

```
    vecs = np.stack([local[a] for a in ids]) if local else np.zeros((0, 384), np.float32)
    return NodeView(node=node, neighbors=nbrs, sfib=adv.tables[node], local_agents=ids,
                    local_vecs=vecs,
                    queue_occ={n: 0.0 for n in nbrs}, link_quality={n: 1.0 for n in nbrs},
                    link_delay={n: 0.002 for n in nbrs}, in_handover={n: 0 for n in nbrs},
                    kind={n: "gnb" for n in nbrs}, upstream=None, tau=tau, radius=adv.radius,
                    accept_mode=accept_mode, omega=0.0)


def _pkt(intent, ttl=5):
    return Packet(pid=0, intent=intent, intent_class=1, ttl=ttl, ttl_max=5, src_agent=0,
                  created=0.0, method="greedy")


def _star():
    # router 0 with neighbours 1 and 2; agent 10 behind 1, agent 11 behind 2
    v = _vecs(2)
    adv = _adv(nx.Graph([(0, 1), (0, 2)]), radius=2)
    adv.local[1][10] = v[0]
    adv.local[2][11] = v[1]
    adv.build()
    return adv, v


def test_greedy_follows_similarity_and_never_goes_back():
    adv, v = _star()
    pkt = _pkt(v[1])
    view = _view(adv, 0)
    assert GreedyPolicy().choose(view, pkt) == 2
    pkt.prev_hop = 2
    assert 2 not in candidates(view, pkt)
    cands, X = neighbor_features(view, pkt)
    assert X.shape == (len(cands), N_FEATURES)


def test_best_known_keeps_going_past_a_weak_local_match():
    adv, v = _star()
    intent = _norm(0.9 * v[1] + 0.3 * v[0])
    weak = {5: _norm(0.3 * v[1] + 0.9 * _vecs(3)[2])}
    assert accept_rule(_view(adv, 0, tau=0.1, accept_mode="first", local=weak), _pkt(intent)) == 5
    assert accept_rule(_view(adv, 0, tau=0.1, accept_mode="best_known", local=weak), _pkt(intent)) is
None


def test_load_aware_greedy_avoids_the_busy_neighbour():
    v = _vecs(1)
    adv = _adv(nx.Graph([(0, 1), (0, 2)]), radius=2)
    adv.local[1][10] = v[0]
    adv.local[2][11] = v[0]
    adv.build()
    view = _view(adv, 0)
    view.queue_occ[1] = 0.9
    assert LoadAwareGreedy().choose(view, _pkt(v[0])) == 2


def test_spfr_goes_to_the_dominant_executor_and_executes_locally():
    adv, v = _star()
    policy = SPFRPolicy(omega=0.08)
    assert policy.accept(_view(adv, 0, tau=0.1), _pkt(v[1])) is None
    assert policy.choose(_view(adv, 0, tau=0.1), _pkt(v[1])) == 2
    assert policy.accept(_view(adv, 0, tau=0.1, local={5: v[1]}), _pkt(v[1])) == 5


def test_local_action_is_offered_only_above_tau():
    adv, v = _star()
    actions, X = neighbor_features(_view(adv, 0, tau=0.5, local={5: v[1]}), _pkt(v[1]),
with_local=True)
    assert actions[0] == LOCAL and X[0, FEATURES.index("is_local")] == 1.0
    actions, _ = neighbor_features(_view(adv, 0, tau=0.5, local={5: v[0]}), _pkt(v[1]),
with_local=True)
    assert LOCAL not in actions
```

tests/test_sim.py

```
"""Tests for the simulator, the baselines and the DQN pieces."""
import numpy as np
import torch

from agentroute.baselines.chord import chash, normalize
from agentroute.config import load_config
from agentroute.dqn.model import MAX_DEG
from agentroute.dqn.policy import DQNPolicy, build_model
from agentroute.forwarding import N_FEATURES
from agentroute.sim.simulator import Simulator


def small_cfg(**sim):
    cfg = load_config(overrides={"agents": {"n_agents": 100}, "sim": {"duration_s": 8.0, **sim}})
    return cfg


def test_same_seed_same_result():
    a = Simulator(small_cfg(), "greedy", seed=3).run()
    b = Simulator(small_cfg(), "greedy", seed=3).run()
    assert a == b


def test_all_methods_see_the_same_queries():
    counts = set()
    for m in ("greedy", "greedy_load", "spfr", "flooding", "registry", "registry3", "chord", "sdht"):
        counts.add(Simulator(small_cfg(), m, seed=2).run()["n_queries"])
    assert len(counts) == 1


def test_chord_fails_on_paraphrases():
    s = Simulator(small_cfg(), "chord", seed=1).run()
    assert s["success_rate"] == 0.0
    assert s["drop_not_found"] > 0.5
    assert normalize("Object  detection, in images!") == "object detection in images"
    assert chash("a", 32) != chash("b", 32)


def test_flooding_is_bounded_by_dedup():
    sim = Simulator(small_cfg(), "flooding", seed=1)
    s = sim.run()
    assert sim.flood.copies / s["n_queries"] <= 2 * sim.G.number_of_edges()


def test_flooding_time_includes_the_collection_window():
    cfg = small_cfg()
    s = Simulator(cfg, "flooding", seed=1).run()
    assert s["time_ms"] >= 1000.0 * cfg["baselines"]["flood_window_s"]


def test_semantic_dht_finds_paraphrases_that_chord_cannot():
    a = Simulator(small_cfg(), "sdht", seed=1).run()
    b = Simulator(small_cfg(), "chord", seed=1).run()
    assert a["success_rate"] > 0.0 and b["success_rate"] == 0.0


def test_replicated_registry_survives_when_the_central_one_is_lost():
    sim = Simulator(small_cfg(), "registry3", seed=1)
    assert len(sim.registry.nodes) == 3
    sim.known_dead.add(sim.registry.nodes[0])
    server, path = sim.registry.nearest(sim.gnbs[0])
    assert server in sim.registry.nodes[1:] and path


def test_oracle_bounds_are_consistent():
    s = Simulator(small_cfg(), "registry", seed=1).run()
    assert 0.0 <= s["oracle_best"] <= s["oracle_any"] <= 1.0
    assert s["ctrl_kB_per_s"] > 0 and s["query_kB_per_query"] > 0


def test_node_failures_are_applied():
    sim = Simulator(small_cfg(failure_fraction=0.3), "greedy", seed=1)
```

```
    sim.run()
    assert sum(not r.alive for r in sim.routers.values()) > 0
    assert sim.known_dead


def test_baselines_survive_heavy_failures():
    # 30% failed routers can partition the network; lookups must fail cleanly, not crash
    for m in ("chord", "sdht", "registry", "registry3", "flooding"):
        for seed in (1, 2, 3):
            s = Simulator(small_cfg(failure_fraction=0.3), m, seed=seed).run()
            assert s["n_queries"] > 0


def test_models_score_each_neighbour():
    for kind in ("mlp", "gnn"):
        model = build_model(kind, hidden=16)
        x = torch.randn(4, MAX_DEG, N_FEATURES)
        mask = torch.zeros(4, MAX_DEG, dtype=torch.bool)
        mask[:, :3] = True
        q = model(x, mask)
        assert q.shape == (4, MAX_DEG)
        assert (q[:, 3:] < -1e8).all()


def test_dqn_policy_runs_and_saves(tmp_path):
    policy = DQNPolicy(build_model("mlp", hidden=16))
    s = Simulator(small_cfg(), "agentroute", seed=1, policy=policy).run()
    assert s["n_queries"] > 0
    path = tmp_path / "m.pt"
    policy.save(path, hidden=16)
    loaded = DQNPolicy.load(path)
    X = np.random.rand(5, N_FEATURES).astype(np.float32)
    assert np.allclose(loaded.q_values(X), policy.q_values(X), atol=1e-6)


def test_every_policy_gets_a_return():
    for m in ("greedy", "greedy_load", "spfr"):
        sim = Simulator(small_cfg(), m, seed=1)
        sim.run()
        assert sim.policy.episode_return != 0.0
```

docs/build_pdfs.py

```
"""Build the PDF versions of the documents with headless Chrome.

    python docs/build_pdfs.py

Markdown -> HTML (python-markdown + a print stylesheet) -> PDF (Chrome
--print-to-pdf). Also builds a code appendix with every source file of the
project, as promised in the proposal ("all code ... included in the final
report").

Written with AI assistance (document tooling).
"""
import html
import re
import shutil
import subprocess
import sys
from pathlib import Path

import markdown
from pygments import highlight
from pygments.formatters import HtmlFormatter
from pygments.lexers import get_lexer_for_filename

ROOT = Path(__file__).resolve().parent.parent
OUT = ROOT / "docs" / "pdf"
BUILD = OUT / "_build"
CHROME_CANDIDATES = [
    "/Applications/Google Chrome.app/Contents/MacOS/Google Chrome",
    "/Applications/Chromium.app/Contents/MacOS/Chromium",
    shutil.which("google-chrome") or "",
    shutil.which("chromium") or "",
]

DOCS = [
    ("docs/report/FINAL_REPORT.md", "AgentRoute_Final_Report.pdf", "en"),
    ("docs/KULLANIM_KILAVUZU.md", "Kullanim_Kilavuzu.pdf", "tr"),
    ("docs/KIM_NE_YAPTI.md", "Kim_Ne_Yapti.pdf", "tr"),
    ("docs/NASIL_YAPTIK.md", "Nasil_Yaptik.pdf", "tr"),
]

CSS = """
@page { size: A4; margin: 18mm 17mm 18mm 17mm; }
body { font-family: -apple-system, "Helvetica Neue", Arial, sans-serif; font-size: 10.3pt;
       line-height: 1.45; color: #0b0b0b; }
h1 { font-size: 20pt; margin: 0 0 6pt; }
h2 { font-size: 14pt; margin: 18pt 0 6pt; border-bottom: 1px solid #e1e0d9; padding-bottom: 3pt; }
h3 { font-size: 11.5pt; margin: 12pt 0 4pt; }
p, li { orphans: 3; widows: 3; }
table { border-collapse: collapse; width: 100%; margin: 8pt 0; font-size: 9.2pt; page-break-inside:
avoid; }
th, td { border: 1px solid #e1e0d9; padding: 3pt 5pt; text-align: left; vertical-align: top; }
th { background: #f3f2ee; }
code { font-family: Menlo, Consolas, monospace; font-size: 8.8pt; background: #f3f2ee; padding: 0 2pt;
       border-radius: 2pt; }
pre { background: #f6f5f1; padding: 6pt 8pt; border-radius: 4pt; overflow-x: hidden;
      white-space: pre-wrap; font-size: 8.4pt; page-break-inside: avoid; }
pre code { background: none; padding: 0; }
img { max-width: 100%; display: block; margin: 6pt auto; page-break-inside: avoid; }
figure { margin: 8pt 0; page-break-inside: avoid; }
blockquote { color: #52514e; border-left: 3px solid #e1e0d9; margin: 6pt 0; padding: 0 8pt; }
.title-block { text-align: center; margin-bottom: 14pt; }
.caption, em.caption { color: #52514e; font-size: 9pt; }
hr { border: 0; border-top: 1px solid #e1e0d9; margin: 12pt 0; }
"""


LIST_ITEM = re.compile(r"^( *)([-*]|\d+\.) ")


def normalise_lists(text):
    """Python-Markdown needs a blank line before a list and 4-space nesting;
    our documents use the GitHub style (no blank line, 2-space nesting)."""
```

```
    out = []
    in_code = False
    for line in text.split("\n"):
        if line.startswith("```"):
            in_code = not in_code
        m = None if in_code else LIST_ITEM.match(line)
        if m:
            indent = len(m.group(1))
            line = " " * (indent * 2) + line[indent:]
            prev = out[-1] if out else ""
            if prev.strip() and not LIST_ITEM.match(prev) and not prev.startswith("|"):
                out.append("")
        out.append(line)
    return "\n".join(out)


def find_chrome():
    for c in CHROME_CANDIDATES:
        if c and Path(c).exists():
            return c
    sys.exit("Chrome/Chromium not found: install Google Chrome to build the PDFs")


def to_pdf(chrome, html_path, pdf_path):
    subprocess.run([chrome, "--headless=new", "--disable-gpu", "--no-pdf-header-footer",
                    "--run-all-compositor-stages-before-draw", "--virtual-time-budget=10000",
                    f"--print-to-pdf={pdf_path}", html_path.as_uri()],
                   check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)


def build_doc(chrome, src, out_name, lang):
    src = ROOT / src
    body = markdown.markdown(normalise_lists(src.read_text()), extensions=["tables", "fenced_code",
"toc",
                                                           "attr_list", "md_in_html"])
    page = (f'<!doctype html><html lang="{lang}"><head><meta charset="utf-8">'
            f'<base href="{src.parent.as_uri()}/"><style>{CSS}</style></head>'
            f"<body>{body}</body></html>")
    html_path = BUILD / (out_name.replace(".pdf", ".html"))
    html_path.write_text(page)
    to_pdf(chrome, html_path, OUT / out_name)
    print("wrote", OUT / out_name)


def build_code_appendix(chrome):
    files = sorted((ROOT / "agentroute").rglob("*.py")) + sorted((ROOT / "experiments").glob("*.py")) \
        + sorted((ROOT / "viz").glob("*.py")) + sorted((ROOT / "tests").glob("*.py")) \
        + [ROOT / "docs" / "build_pdfs.py", ROOT / "configs" / "default.yaml", ROOT / "configs" /
"demo.yaml"] \
        + sorted((ROOT / "viz" / "web" / "src").rglob("*.js*")) \
        + sorted((ROOT / "viz" / "web" / "src").rglob("*.css"))
    files = [f for f in files if f.name != "__init__.py"]
    fmt = HtmlFormatter(style="friendly", nowrap=False)
    parts = ["<h1>AgentRoute: Code Appendix</h1>",
             "<p>Complete source code of the project (simulator, protocols, DQN, experiments, "
             "visualisation). Each module header names the responsible team member; files written "
             "with AI assistance say so in their header.</p><ol>"]
    parts += [f"<li><code>{html.escape(str(f.relative_to(ROOT)))}</code></li>" for f in files]
    parts.append("</ol>")
    for f in files:
        rel = f.relative_to(ROOT)
        code = f.read_text()
        parts.append(f'<h2 style="page-break-before: always">{html.escape(str(rel))}</h2>')
        parts.append(highlight(code, get_lexer_for_filename(f.name), fmt))
    page = (f'<!doctype html><html lang="en"><head><meta charset="utf-8"><style>{CSS}'
            f'{fmt.get_style_defs(".highlight")} .highlight pre {{ font-size: 7.6pt; '
            f'page-break-inside: auto; }}</style></head><body>{"".join(parts)}</body></html>')
    html_path = BUILD / "AgentRoute_Code_Appendix.html"
    html_path.write_text(page)
    to_pdf(chrome, html_path, OUT / "AgentRoute_Code_Appendix.pdf")
    print("wrote", OUT / "AgentRoute_Code_Appendix.pdf")


def main():
    chrome = find_chrome()
    BUILD.mkdir(parents=True, exist_ok=True)
    for src, out, lang in DOCS:
```

```
        if (ROOT / src).exists():
            build_doc(chrome, src, out, lang)
        else:
            print("skip (missing)", src)
    build_code_appendix(chrome)
    shutil.rmtree(BUILD)


if __name__ == "__main__":
    main()
```

configs/default.yaml

```yaml
# Default AgentRoute scenario. Experiment configs override only what they change.
seed: 1

topology:
  agents_per_cell: 20       # number of cells grows with the agent count
  min_cells: 9
  max_cells: 400
  cell_spacing_m: 500       # distance between neighbouring gNBs
  cells_per_core: 8         # one core router per 8 cells (at least 4)
  delay_ms:                 # one-way propagation delay per link type
    radio: 5.0
    xn: 2.0
    backhaul: 5.0
    core: 3.0
    cloud: 20.0
  loss:
    radio: 0.002
    wired: 0.0005
    bad: 0.03               # loss on "bad" wired links
  bad_link_fraction: 0.1

agents:
  n_agents: 100
  mobile_fraction: 0.5      # random-waypoint UEs
  cloud_fraction: 0.1       # agents hosted in the cloud node
  class_zipf: 1.0           # capability supply ~ Zipf(s); 0 = uniform
  card_alpha: 2.0           # each agent mixes its class's sentences ~ Dirichlet(alpha)
  speed_mps: 5.0
  pause_s: 2.0
  hysteresis: 0.15          # switch cell only if the new gNB is 15% closer
  handover_interrupt_s: 0.05
  mobility_dt_s: 1.0

routers:
  service_ms: 0.2           # per-packet processing time
  queue_limit: 100
  hotspot_fraction: 0.15    # routers with bursty background traffic
  rho_base: 0.2             # background utilisation of a normal router
  rho_busy: 0.95            # background utilisation of a busy hotspot
  busy_mean_s: 5.0          # hotspots switch busy/idle with this mean period
  bg_service_ms: 1.0
  bg_queue_limit: 50
  hello_s: 0.1              # neighbour state (queue, handover) is refreshed this often

sfib:
  radius: 2
  summary: kmeans           # kmeans | medoid | mean | exact
  k: 8
  exact_hops: 1             # agents this close keep their own row (0 = summaries only)
  encoding: int8            # float32 | int8, representatives in advertisements
  change_threshold: 0.01    # a handover message travels on only if a representative moves more
  refresh_s: 10.0
  failure_detect_s: 1.0
  handover_window_s: 2.0    # a gNB is flagged "in handover" for this long

forwarding:
  ttl: 16
  tau: null                 # null -> read results/tau.json (experiments/tune_tau.py)
  header: int8              # float32 | int8 | binary
  accept: best_known        # first (v1) | best_known
  omega: 0.08               # attenuation per hop of a remote candidate (SPFR's value)
  load_band: 0.05           # greedy_load: neighbours this close to the best count as equal

sim:
  duration_s: 30.0
  warmup_s: 2.0
  query_rate: 10.0          # task requests per second, whole network
  load: 1.0                 # scales query_rate and background utilisation
  failure_fraction: 0.0
  failure_time_frac: 0.33

baselines:
```

```yaml
  flood_ttl: 6
  flood_window_s: 0.2       # how long the requester collects answers
  registry_service_ms: 1.0  # one lookup (a vector search)
  registry_update_ms: 0.05  # one registration or heartbeat (a table write)
  registry_queue_limit: 200
  registry3_replicas: 3
  chord_bits: 32
  sdht_buckets: 64          # semantic DHT: size of the public capability catalogue
  sdht_nprobe: 2            # how many catalogue entries a request looks up
  spfr_radius: 2            # SPFR's control horizon

dqn:
  model_path: results/models/dqn.pt
  gnn_model_path: results/models/gnn.pt
  hidden: 64
  gamma: 0.98
  lr: 0.001
  batch_size: 128
  buffer_size: 50000
  train_every: 4
  target_update: 500
  episodes: 300
  eps_start: 1.0
  eps_end: 0.05
  eps_decay_frac: 0.6
  eval_every: 10
  reward:
    hop_cost: 0.05
    delay_scale_s: 0.1
    deliver_bonus: 1.0
    drop_penalty: 1.0
```

configs/demo.yaml

```yaml
# Small demo scenario for the animation, the Streamlit app and the web player.
# 100 agents -> 9 cells, 4 core routers, 1 cloud node. Every event is logged.
seed: 7
agents:
  n_agents: 100
  speed_mps: 8.0
routers:
  hotspot_fraction: 0.25
sim:
  duration_s: 30.0
  load: 2.0
```

viz/web/src/App.jsx

```jsx
import { useCallback, useEffect, useRef, useState } from "react";
import EventFeed from "./components/EventFeed.jsx";
import RunPicker from "./components/RunPicker.jsx";
import StatsPanel from "./components/StatsPanel.jsx";
import Timeline from "./components/Timeline.jsx";
import TopologyView from "./components/TopologyView.jsx";
import Waterfall from "./components/Waterfall.jsx";
import { METHOD_LABEL, buildRun, loadIndex, loadRun, parseJsonl } from "./data.js";

const METHOD_COLOR = {
  agentroute: "#2a78d6",
  agentroute_gnn: "#eb6834",
  greedy: "#1baf7a",
  greedy_load: "#0f8a8a",
  spfr: "#6250d6",
  flooding: "#eda100",
  registry: "#e87ba4",
  registry3: "#a8437a",
  chord: "#008300",
  sdht: "#7a6a5a",
};

export default function App() {
  const [runs, setRuns] = useState([]);
  const [runName, setRunName] = useState(null);
  const [run, setRun] = useState(null);
  const [error, setError] = useState(null);
  const [t, setT] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(0.05);
  const [direction, setDirection] = useState(1);
  const [selected, setSelected] = useState(null);
  const [onlyDelivered, setOnlyDelivered] = useState(false);
  const last = useRef(null);

  useEffect(() => {
    loadIndex().then((idx) => {
      setRuns(idx.runs || []);
      const first = (idx.runs || []).find((r) => r.method === "agentroute") || (idx.runs || [])[0];
      if (first) setRunName(first.name);
    });
  }, []);

  useEffect(() => {
    if (!runName) return;
    setError(null);
    loadRun(runName)
      .then((r) => { setRun(r); setT(0); setSelected(null); })
      .catch((e) => setError(`Could not load ${runName}: ${e.message}`));
  }, [runName]);

  // playback loop: simulated time advances by real time x speed
  useEffect(() => {
    if (!playing || !run) return undefined;
    let id;
    const step = (now) => {
      if (last.current != null) {
        const dt = (now - last.current) / 1000;
        setT((old) => {
          const next = old + direction * dt * speed;
          if (next <= 0 || next >= run.duration) {
            setPlaying(false);
            return Math.min(Math.max(next, 0), run.duration);
          }
          return next;
        });
      }
      last.current = now;
      id = requestAnimationFrame(step);
    };
    id = requestAnimationFrame(step);
    return () => { cancelAnimationFrame(id); last.current = null; };
```

```
  }, [playing, speed, direction, run]);

  const onFiles = useCallback(async (files) => {
    const byName = Object.fromEntries(files.map((f) => [f.name, f]));
    if (!byName["topology.json"] || !byName["events.jsonl"]) {
      setError("Please drop both topology.json and events.jsonl.");
      return;
    }
    const topo = JSON.parse(await byName["topology.json"].text());
    const events = parseJsonl(await byName["events.jsonl"].text());
    const summary = byName["summary.json"] ? JSON.parse(await byName["summary.json"].text()) : null;
    setRun(buildRun(topo, events, summary, "dropped files"));
    setRunName(null);
    setT(0);
    setSelected(null);
    setError(null);
  }, []);

  const followPacket = (pid) => {
    const p = run.packets.find((x) => x.pid === pid);
    setSelected(pid);
    if (p) {
      setT(Math.max(p.t0 - 0.002, 0));
      setSpeed(0.01);
      setDirection(1);
      setPlaying(true);
    }
  };

  const color = run ? METHOD_COLOR[run.method] || "#2a78d6" : "#2a78d6";
  const packetList = run ? run.packets.filter((p) => !onlyDelivered || p.ok) : [];

  return (
    <div className="app">
      <header>
        <div>
          <h1>AgentRoute log player</h1>
          <p className="muted">Replays a simulation log: packets addressed to capabilities, hop by hop.
</p>
        </div>
        <RunPicker runs={runs} current={runName} onPick={setRunName} onFiles={onFiles} />
      </header>
      {error && <p className="error">{error}</p>}
      {!run && !error && (
        <p className="muted">Export a run first: <code>python viz/export_web_data.py</code></p>
      )}
      {run && (
        <main>
          <section className="left">
            <div className="panel-title">
              <span className="swatch" style={{ background: color }} />
              {METHOD_LABEL[run.method] || run.method} &middot; {run.topo.meta.n_agents} agents
              &middot; R = {run.topo.meta.radius}
            </div>
            <TopologyView run={run} t={t} color={color} selected={selected} onSelect={followPacket} />
            <Timeline t={t} duration={run.duration} playing={playing} speed={speed}
              direction={direction} onTime={(v) => { setT(v); setPlaying(false); }}
              onPlay={setPlaying} onSpeed={setSpeed} onDirection={setDirection} />
            <p className="legend muted small">
              <span className="key gnb" /> gNB + edge router <span className="key core" /> core router
              <span className="key agent" /> agent <span className="key busy" /> busy router
              <span className="key ring-g" /> announce / serving agent <span className="key ring-r" />
withdraw
               &middot; scroll to zoom, drag to pan
            </p>
          </section>
          <section className="right">
            <div className="card">
              <h2>Run summary</h2>
              <StatsPanel run={run} />
            </div>
            <div className="card">
              <h2>Request</h2>
              <div className="row">
                <select value={selected ?? ""} onChange={(e) => followPacket(Number(e.target.value))}>
                  <option value="">choose a request...</option>
                  {packetList.map((p) => (
```

```
                    <option key={p.pid} value={p.pid}>
                      #{p.pid} {p.ok ? "✓" : "✗"} {p.q.text.slice(0, 48)}
                    </option>
                  ))}
                </select>
                <label className="small">
                  <input type="checkbox" checked={onlyDelivered}
                    onChange={(e) => setOnlyDelivered(e.target.checked)} /> delivered only
                </label>
              </div>
              <Waterfall run={run} pid={selected} color={color} />
            </div>
            <div className="card">
              <h2>Events</h2>
              <EventFeed run={run} t={t} onSelect={followPacket} />
            </div>
          </section>
        </main>
      )}
    </div>
  );
}
```

viz/web/src/components/EventFeed.jsx

```
import { eventIndexAt } from "../data.js";

const SHOWN = new Set(["query", "hop", "deliver", "drop", "handover", "fail", "withdraw"]);

function describe(e) {
  switch (e.type) {
    case "query": return `request ${e.pkt} from agent ${e.agent}: "${e.text}"`;
    case "hop": return `request ${e.pkt}: ${e.from} → ${e.to}`;
    case "deliver": return `request ${e.pkt} delivered to agent ${e.agent} (${e.hops} hops)`;
    case "drop": return `request ${e.pkt} dropped at ${e.at ?? "–"} (${e.reason})`;
    case "handover": return `agent ${e.agent}: cell ${e.from_cell} → ${e.to_cell}`;
    case "fail": return `router ${e.node} failed`;
    case "withdraw": return `withdraw ${e.from} → ${e.to}`;
    default: return e.type;
  }
}

// The last events before the current time; click a request to follow it.
export default function EventFeed({ run, t, onSelect }) {
  const end = eventIndexAt(run.events, t);
  const rows = [];
  for (let i = end - 1; i >= 0 && rows.length < 40; i--) {
    const e = run.events[i];
    if (SHOWN.has(e.type)) rows.push(e);
  }
  return (
    <div className="feed">
      {rows.map((e, i) => (
        <button key={`${e.t}-${i}`} className={`feed-row ${e.type}`}
          onClick={() => e.pkt != null && onSelect(e.pkt)} disabled={e.pkt == null}>
          <span className="feed-t">{e.t.toFixed(3)}</span>
          <span className={`tag ${e.type}`}>{e.type}</span>
          <span className="feed-text">{describe(e)}</span>
        </button>
      ))}
      {!rows.length && <p className="muted">No events yet – press play.</p>}
    </div>
  );
}
```

viz/web/src/components/RunPicker.jsx

```
import { useState } from "react";
import { METHOD_LABEL } from "../data.js";

// Choose one of the exported runs, or drop topology.json + events.jsonl files.
export default function RunPicker({ runs, current, onPick, onFiles }) {
  const [over, setOver] = useState(false);
  return (
    <div className="picker">
      <label>
        Run
        <select value={current || ""} onChange={(e) => onPick(e.target.value)}>
          {!runs.length && <option value="">no exported runs</option>}
          {runs.map((r) => (
            <option key={r.name} value={r.name}>
              {r.name} — {METHOD_LABEL[r.method] || r.method}
              {r.success_rate != null ? ` (${(r.success_rate * 100).toFixed(0)}%)` : ""}
            </option>
          ))}
        </select>
      </label>
      <div className={`drop ${over ? "over" : ""}`}
        onDragOver={(e) => { e.preventDefault(); setOver(true); }}
        onDragLeave={() => setOver(false)}
        onDrop={(e) => { e.preventDefault(); setOver(false); onFiles([...e.dataTransfer.files]); }}>
        or drop <code>topology.json</code> + <code>events.jsonl</code> here
      </div>
    </div>
  );
}
```

viz/web/src/components/StatsPanel.jsx

```
import { useState } from "react";
import * as d3 from "d3";
import { REASON_LABEL } from "../data.js";

// Headline numbers of the run + a D3 bar chart of why requests failed.
export default function StatsPanel({ run }) {
  const s = run.summary;
  const [hover, setHover] = useState(null);
  const reasons = Object.keys(REASON_LABEL)
    .map((k) => ({ key: k, label: REASON_LABEL[k], v: s[`drop_${k}`] || 0 }))
    .filter((d) => d.v > 0.001)
    .sort((a, b) => b.v - a.v);
  const w = 340;
  const rowH = 26;
  const h = reasons.length * rowH + 10;
  const x = d3.scaleLinear().domain([0, d3.max(reasons, (d) => d.v) || 1]).range([0, w - 190]);

  return (
    <div className="stats">
      <div className="tiles">
        <div className="tile"><span className="tile-label">Success</span>
          <span className="tile-value">{(s.success_rate * 100).toFixed(1)}%</span></div>
        <div className="tile"><span className="tile-label">Mean time</span>
          <span className="tile-value">{s.time_ms != null && !Number.isNaN(s.time_ms)
            ? `${s.time_ms.toFixed(1)} ms` : "—"}</span></div>
        <div className="tile"><span className="tile-label">Hops</span>
          <span className="tile-value">{s.hops != null && !Number.isNaN(s.hops) ? s.hops.toFixed(2) :
"—"}</span></div>
        <div className="tile"><span className="tile-label">Requests</span>
          <span className="tile-value">{s.n_queries}</span></div>
      </div>
      <h3>Why requests fail</h3>
      <svg viewBox={`0 0 ${w} ${h}`} className="bars" role="img" aria-label="Failure reasons">
        {reasons.map((d, i) => (
          <g key={d.key} transform={`translate(0, ${i * rowH})`}
            onMouseEnter={() => setHover(d.key)} onMouseLeave={() => setHover(null)}>
            <rect x={0} y={0} width={w} height={rowH} fill="transparent" />
            <text x={0} y={rowH / 2 + 4} className="bar-label">{d.label}</text>
            <path d={roundedBar(150, 6, Math.max(x(d.v), 2), rowH - 12)}
              className={hover === d.key ? "bar hover" : "bar"} />
            <text x={150 + x(d.v) + 6} y={rowH / 2 + 4} className="bar-value">
              {(d.v * 100).toFixed(1)}%</text>
          </g>
        ))}
      </svg>
      {!reasons.length && <p className="muted">No failed requests.</p>}
    </div>
  );
}

// bar with a 4px rounded data end, square at the baseline
function roundedBar(x0, y0, w, h) {
  const r = Math.min(4, w / 2, h / 2);
  return `M${x0},${y0} H${x0 + w - r} Q${x0 + w},${y0} ${x0 + w},${y0 + r} V${y0 + h - r}
    Q${x0 + w},${y0 + h} ${x0 + w - r},${y0 + h} H${x0} Z`;
}
```

viz/web/src/components/Timeline.jsx

```jsx
// Playback controls: play / pause / rewind, speed, scrubber (like NetAnim / nam).
const SPEEDS = [0.01, 0.05, 0.2, 1];

export default function Timeline({ t, duration, playing, speed, direction, onTime, onPlay,
  onSpeed, onDirection }) {
  return (
    <div className="timeline">
      <button className="btn" onClick={() => { onDirection(-1); onPlay(true); }}
        aria-pressed={playing && direction < 0} title="Play backwards">{"◀◀"}</button>
      <button className="btn primary" onClick={() => { if (!playing) onDirection(1); onPlay(!playing);
}}
        title={playing ? "Pause" : "Play"}>{playing ? "❚❚" : "▶"}</button>
      <button className="btn" onClick={() => onTime(0)} title="Back to start">{"⏮"}</button>
      <input type="range" min={0} max={duration} step={0.001} value={t}
        onChange={(e) => onTime(Number(e.target.value))} aria-label="Simulation time" />
      <span className="time">{t.toFixed(3)} s</span>
      <label className="speed">
        speed
        <select value={speed} onChange={(e) => onSpeed(Number(e.target.value))}>
          {SPEEDS.map((s) => <option key={s} value={s}>{s}x</option>)}
        </select>
      </label>
    </div>
  );
}
```

viz/web/src/components/TopologyView.jsx

```
import { useEffect, useMemo, useRef, useState } from "react";
import * as d3 from "d3";
import { agentPos, busyAt, packetPos } from "../data.js";

const LINK_STYLE = {
  xn: { w: 1, dash: null, cls: "link" },
  backhaul: { w: 0.8, dash: null, cls: "link" },
  core: { w: 1.8, dash: null, cls: "link-core" },
  cloud: { w: 1.8, dash: "5 4", cls: "link-core" },
};

// Network view: routers, links, agents, packets in flight, handovers.
// D3 provides the scales and the zoom behaviour; React renders the SVG.
export default function TopologyView({ run, t, color, selected, onSelect }) {
  const svgRef = useRef(null);
  const [transform, setTransform] = useState(d3.zoomIdentity);
  const width = 900;
  const height = 640;

  const { sx, sy, r } = useMemo(() => {
    const xs = run.topo.nodes.map((n) => n.x);
    const ys = run.topo.nodes.map((n) => n.y);
    const pad = 260;
    const dx = d3.max(xs) - d3.min(xs) + 2 * pad;
    const dy = d3.max(ys) - d3.min(ys) + 2 * pad;
    const k = Math.min(width / dx, height / dy);
    const ox = (width - dx * k) / 2;
    const oy = (height - dy * k) / 2;
    return {
      sx: (x) => ox + (x - d3.min(xs) + pad) * k,
      sy: (y) => height - (oy + (y - d3.min(ys) + pad) * k), // y axis up, like the plots
      r: k,
    };
  }, [run]);

  useEffect(() => {
    const zoom = d3.zoom().scaleExtent([0.5, 12]).on("zoom", (e) => setTransform(e.transform));
    const svg = d3.select(svgRef.current);
    svg.call(zoom);
    return () => svg.on(".zoom", null);
  }, []);

  const spacing = 500;
  const nodes = run.topo.nodes;
  const active = run.packets.filter((p) => t >= p.t0 && t <= p.t1 + 0.05);
  const sel = selected != null ? run.packets.find((p) => p.pid === selected) : null;
  const flashes = run.handovers.filter((h) => t >= h.t && t - h.t < 0.6);
  const agentList = [...run.agents.values()];

  return (
    <svg ref={svgRef} viewBox={`0 0 ${width} ${height}`} className="topology" role="img"
      aria-label="Network topology with packets in flight">
      <g transform={transform.toString()}>
        {nodes.filter((n) => n.kind === "gnb").map((n) => (
          <circle key={`cell${n.id}`} cx={sx(n.x)} cy={sy(n.y)} r={spacing * 0.56 * r} className="cell"
/>
        ))}
        {run.topo.links.map((l, i) => {
          const s = LINK_STYLE[l.kind];
          const [x1, y1] = run.pos.get(l.u);
          const [x2, y2] = run.pos.get(l.v);
          return (
            <line key={i} x1={sx(x1)} y1={sy(y1)} x2={sx(x2)} y2={sy(y2)} className={s.cls}
              strokeWidth={s.w} strokeDasharray={s.dash} />
          );
        })}
        {nodes.filter((n) => busyAt(run, n.id, t)).map((n) => (
          <circle key={`busy${n.id}`} cx={sx(n.x)} cy={sy(n.y)} r={22} className="busy" />
        ))}
        {sel && sel.segs.slice(1).map((s, i) => (
          <line key={`sel${i}`} x1={sx(s.a[0])} y1={sy(s.a[1])} x2={sx(s.b[0])} y2={sy(s.b[1])}
            stroke={color} strokeWidth={4} strokeLinecap="round" opacity={0.85} />
```

```
        ))}
        {nodes.map((n) => {
          const x = sx(n.x);
          const y = sy(n.y);
          if (n.kind === "gnb") {
            return <rect key={n.id} x={x - 7} y={y - 7} width={14} height={14} rx={2} className="gnb">
              <title>{`gNB + edge router ${n.id}`}</title></rect>;
          }
          if (n.kind === "core") {
            return <circle key={n.id} cx={x} cy={y} r={9} className="core">
              <title>{`core router ${n.id}`}</title></circle>;
          }
          const pts = d3.range(6).map((i) => {
            const a = (Math.PI / 3) * i + Math.PI / 6;
            return `${x + 13 * Math.cos(a)},${y + 13 * Math.sin(a)}`;
          }).join(" ");
          return <g key={n.id}><polygon points={pts} className="core"><title>cloud</title></polygon>
            <text x={x} y={y + 28} className="node-label" textAnchor="middle">Cloud</text></g>;
        })}
        {agentList.map((a) => {
          const [x, y] = agentPos(run, a.id, t);
          return <circle key={`a${a.id}`} cx={sx(x)} cy={sy(y)} r={2.6} className="agent">
            <title>{`agent ${a.id}: ${a.class_name} (${a.kind})`}</title></circle>;
        })}
        {flashes.map((h, i) => (
          <g key={`ho${i}`}>
            <circle cx={sx(run.pos.get(h.from_cell)[0])} cy={sy(run.pos.get(h.from_cell)[1])} r={20}
              className="ring-withdraw" />
            <circle cx={sx(run.pos.get(h.to_cell)[0])} cy={sy(run.pos.get(h.to_cell)[1])} r={20}
              className="ring-announce" />
          </g>
        ))}
        {active.map((p) => {
          const xy = packetPos(p, t);
          if (!xy) return null;
          return (
            <circle key={`p${p.pid}`} cx={sx(xy[0])} cy={sy(xy[1])} r={p.pid === selected ? 7 : 5}
              fill={color} className="packet" onClick={() => onSelect(p.pid)}>
              <title>{`request ${p.pid}: ${p.q.text}`}</title>
            </circle>
          );
        })}
        {sel && sel.ok && (() => {
          const [x, y] = agentPos(run, sel.out.agent, sel.out.t);
          return <circle cx={sx(x)} cy={sy(y)} r={11} className="ring-announce" />;
        })()}
      </g>
    </svg>
  );
}
```

viz/web/src/components/Waterfall.jsx

```
import { useState } from "react";
import * as d3 from "d3";

// Per-request delay waterfall: one row per hop, bar = time spent on that hop
// (link delay + queueing at the next router).
export default function Waterfall({ run, pid, color }) {
  const [hover, setHover] = useState(null);
  const p = run.packets.find((x) => x.pid === pid);
  if (!p) return <p className="muted">Select a request (click a packet or an event).</p>;
  const t0 = p.q.t;
  const rows = p.segs.map((s, i) => ({ i, label: s.label, start: (s.t0 - t0) * 1000,
    dur: Math.max((s.t1 - s.t0) * 1000, 0.05) }));
  const total = (p.t1 - t0) * 1000;
  const w = 380;
  const rowH = 22;
  const left = 96;
  const h = rows.length * rowH + 28;
  const x = d3.scaleLinear().domain([0, Math.max(total, 1)]).range([left, w - 12]);
  const ticks = x.ticks(4);
  const agent = p.ok ? run.agents.get(p.out.agent) : null;

  return (
    <div className="waterfall">
      <p className="intent">&ldquo;{p.q.text}&rdquo;</p>
      <p className="muted small">
        {p.ok ? `Delivered to agent ${agent.id} (${agent.class_name})`
          : `Dropped: ${p.out.reason}`} &middot; {total.toFixed(1)} ms &middot; {rows.length - 1} hops
      </p>
      <svg viewBox={`0 0 ${w} ${h}`} role="img" aria-label="Delay per hop">
        {ticks.map((tk) => (
          <g key={tk}>
            <line x1={x(tk)} x2={x(tk)} y1={0} y2={h - 20} className="grid" />
            <text x={x(tk)} y={h - 6} className="axis-label" textAnchor="middle">{tk} ms</text>
          </g>
        ))}
        {rows.map((r) => (
          <g key={r.i} transform={`translate(0, ${r.i * rowH})`}
            onMouseEnter={() => setHover(r.i)} onMouseLeave={() => setHover(null)}>
            <rect x={0} y={0} width={w} height={rowH} fill="transparent" />
            <text x={0} y={rowH / 2 + 4} className="bar-label">{r.label}</text>
            <rect x={x(r.start)} y={5} width={Math.max(x(r.start + r.dur) - x(r.start), 2)}
              height={rowH - 10} rx={3} fill={color} opacity={hover === r.i ? 1 : 0.8} />
            {hover === r.i && (
              <text x={Math.min(x(r.start + r.dur) + 4, w - 60)} y={rowH / 2 + 4} className="bar-
value">
                {r.dur.toFixed(2)} ms</text>
            )}
          </g>
        ))}
      </svg>
    </div>
  );
}
```

viz/web/src/data.js

```
// Loading and indexing of simulation logs (topology.json + events.jsonl).
// The player never talks to the simulator; it only reads these files.

export const METHOD_LABEL = {
  agentroute: "AgentRoute (DQN)",
  agentroute_gnn: "AgentRoute (GNN)",
  greedy: "Greedy semantic",
  greedy_load: "Load-aware greedy",
  spfr: "SPFR-style",
  flooding: "Flooding",
  registry: "Central registry",
  registry3: "Registry (3 replicas)",
  chord: "Chord (exact)",
  sdht: "Semantic DHT",
};

export const REASON_LABEL = {
  wrong_agent: "Wrong agent accepted",
  not_found: "No match found",
  ttl: "TTL expired",
  queue: "Queue overflow",
  link: "Link loss",
  node_down: "Node down",
  handover: "Agent in handover",
  stale_location: "Stale location",
  registry_queue: "Registry overloaded",
  no_route: "No route",
};

export async function loadIndex() {
  try {
    const r = await fetch("data/index.json");
    if (!r.ok) return { runs: [] };
    return await r.json();
  } catch {
    return { runs: [] };
  }
}

export async function loadRun(name) {
  const base = `data/${name}`;
  const [topo, text, summary] = await Promise.all([
    fetch(`${base}/topology.json`).then((r) => r.json()),
    fetch(`${base}/events.jsonl`).then((r) => r.text()),
    fetch(`${base}/summary.json`)
      .then((r) => (r.ok ? r.json() : null))
      .catch(() => null),
  ]);
  return buildRun(topo, parseJsonl(text), summary, name);
}

export function parseJsonl(text) {
  return text
    .split("\n")
    .filter((line) => line.trim())
    .map((line) => JSON.parse(line));
}

// Turns the flat event list into lookup tables used by the views.
export function buildRun(topo, events, summary, name) {
  const pos = new Map(topo.nodes.map((n) => [n.id, [n.x, n.y]]));
  const kind = new Map(topo.nodes.map((n) => [n.id, n.kind]));
  const agents = new Map(topo.agents.map((a) => [a.id, a]));
  const queries = new Map();
  const hops = new Map();
  const outcome = new Map();
  const moves = new Map();
  const load = new Map();
  const handovers = [];
  const adverts = [];
  events.sort((a, b) => a.t - b.t);
  for (const e of events) {
```

```
    switch (e.type) {
      case "query":
        queries.set(e.pkt, e);
        break;
      case "hop":
        if (!hops.has(e.pkt)) hops.set(e.pkt, []);
        hops.get(e.pkt).push(e);
        break;
      case "deliver":
      case "drop":
        outcome.set(e.pkt, e);
        break;
      case "move":
        if (!moves.has(e.agent)) moves.set(e.agent, []);
        moves.get(e.agent).push([e.t, e.x, e.y]);
        break;
      case "load":
        if (!load.has(e.node)) load.set(e.node, []);
        load.get(e.node).push([e.t, e.busy]);
        break;
      case "handover":
        handovers.push(e);
        break;
      case "adv":
      case "withdraw":
        adverts.push(e);
        break;
      default:
        break;
    }
  }
  const run = { name, topo, events, pos, kind, agents, queries, hops, outcome, moves, load,
    handovers, adverts, method: topo.meta.method, duration: topo.meta.duration + 3 };
  run.packets = buildPackets(run);
  run.summary = summary || summarise(run);
  return run;
}

// Every packet becomes a list of timed segments (uplink, hops, delivery).
function buildPackets(run) {
  const packets = [];
  for (const [pid, q] of run.queries) {
    const out = run.outcome.get(pid);
    if (!out) continue;
    const hs = run.hops.get(pid) || [];
    const segs = [];
    const start = agentPos(run, q.agent, q.t);
    const firstT = hs.length ? hs[0].t : out.t;
    segs.push({ t0: q.t, t1: firstT, a: start, b: run.pos.get(q.router), label: "uplink" });
    hs.forEach((h, i) => {
      const t1 = i + 1 < hs.length ? hs[i + 1].t : out.t;
      segs.push({ t0: h.t, t1, a: run.pos.get(h.from), b: run.pos.get(h.to),
        label: `${h.from} → ${h.to}`, from: h.from, to: h.to });
    });
    if (out.type === "deliver") {
      const last = hs.length ? run.pos.get(hs[hs.length - 1].to) : run.pos.get(q.router);
      const dst = agentPos(run, out.agent, out.t);
      const tEnd = segs[segs.length - 1].t1;
      segs.push({ t0: tEnd, t1: Math.max(out.t, tEnd + 1e-4), a: last, b: dst, label: "to agent" });
    }
    packets.push({ pid, q, out, segs, t0: q.t, t1: out.t, ok: out.type === "deliver" });
  }
  return packets;
}

function summarise(run) {
  const all = [...run.outcome.values()];
  const ok = all.filter((o) => o.type === "deliver");
  const s = { method: run.method, n_queries: all.length,
    success_rate: all.length ? ok.length / all.length : 0 };
  for (const o of all) {
    if (o.type === "drop") s[`drop_${o.reason}`] = (s[`drop_${o.reason}`] || 0) + 1 / all.length;
  }
  return s;
}

function bisect(arr, t) {
```

```
  let lo = 0;
  let hi = arr.length;
  while (lo < hi) {
    const mid = (lo + hi) >> 1;
    if (arr[mid][0] <= t) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}

export function agentPos(run, id, t) {
  const tr = run.moves.get(id);
  const a = run.agents.get(id);
  if (!tr || !tr.length) return [a.x, a.y];
  const i = bisect(tr, t);
  if (i === 0) return [tr[0][1], tr[0][2]];
  if (i >= tr.length) return [tr[tr.length - 1][1], tr[tr.length - 1][2]];
  const [t0, x0, y0] = tr[i - 1];
  const [t1, x1, y1] = tr[i];
  const f = (t - t0) / Math.max(t1 - t0, 1e-9);
  return [x0 + f * (x1 - x0), y0 + f * (y1 - y0)];
}

export function busyAt(run, node, t) {
  const tr = run.load.get(node);
  if (!tr) return false;
  const i = bisect(tr, t);
  return i > 0 ? tr[i - 1][1] : false;
}

export function packetPos(p, t) {
  for (const s of p.segs) {
    if (t >= s.t0 && t <= s.t1) {
      const f = (t - s.t0) / Math.max(s.t1 - s.t0, 1e-9);
      return [s.a[0] + f * (s.b[0] - s.a[0]), s.a[1] + f * (s.b[1] - s.a[1])];
    }
  }
  return null;
}

// index of the first event with time > t (events are sorted)
export function eventIndexAt(events, t) {
  let lo = 0;
  let hi = events.length;
  while (lo < hi) {
    const mid = (lo + hi) >> 1;
    if (events[mid].t <= t) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}
```

viz/web/src/main.jsx

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./styles.css";

createRoot(document.getElementById("root")).render(<App />);
```

viz/web/src/styles.css

```
:root {
  color-scheme: light;
  --page: #f9f9f7;
  --surface: #fcfcfb;
  --ink: #0b0b0b;
  --ink2: #52514e;
  --muted: #898781;
  --grid: #e1e0d9;
  --axis: #c3c2b7;
  --border: rgba(11, 11, 11, 0.1);
  --cell: #eef3fb;
  --node: #52514e;
  --node-fill: #ffffff;
  --agent: #898781;
  --accent: #2a78d6;
  --good: #0ca30c;
  --critical: #d03b3b;
  --serious: #ec835a;
}
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    color-scheme: dark;
    --page: #0d0d0d;
    --surface: #1a1a19;
    --ink: #f0efec;
    --ink2: #c3c2b7;
    --muted: #898781;
    --grid: #2c2c2a;
    --axis: #383835;
    --border: rgba(255, 255, 255, 0.1);
    --cell: #1f2733;
    --node: #c3c2b7;
    --node-fill: #1a1a19;
    --agent: #898781;
    --accent: #3987e5;
  }
}
:root[data-theme="dark"] {
  color-scheme: dark;
  --page: #0d0d0d;
  --surface: #1a1a19;
  --ink: #f0efec;
  --ink2: #c3c2b7;
  --muted: #898781;
  --grid: #2c2c2a;
  --axis: #383835;
  --border: rgba(255, 255, 255, 0.1);
  --cell: #1f2733;
  --node: #c3c2b7;
  --node-fill: #1a1a19;
  --agent: #898781;
  --accent: #3987e5;
}

* { box-sizing: border-box; }
body {
  margin: 0;
  background: var(--page);
  color: var(--ink);
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
  font-size: 14px;
}
.app { max-width: 1400px; margin: 0 auto; padding: 16px; }
header { display: flex; justify-content: space-between; align-items: flex-end; gap: 16px; flex-wrap:
wrap; }
h1 { font-size: 22px; margin: 0 0 4px; }
h2 { font-size: 15px; margin: 0 0 10px; }
h3 { font-size: 13px; margin: 14px 0 6px; color: var(--ink2); }
.muted { color: var(--muted); }
.small { font-size: 12px; }
.error { color: var(--critical); }
code { font-size: 12px; background: var(--grid); padding: 1px 4px; border-radius: 3px; }
```

```
main { display: grid; grid-template-columns: minmax(0, 1.6fr) minmax(0, 1fr); gap: 16px; margin-top:
16px; }
@media (max-width: 900px) { main { grid-template-columns: 1fr; } }
.left, .right { min-width: 0; }
.card, .left {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 14px;
}
.right { display: flex; flex-direction: column; gap: 16px; }
.panel-title { display: flex; align-items: center; gap: 8px; font-weight: 600; margin-bottom: 8px; }
.swatch { width: 12px; height: 12px; border-radius: 3px; display: inline-block; }

.picker { display: flex; gap: 12px; align-items: center; flex-wrap: wrap; }
.picker label { display: flex; gap: 6px; align-items: center; color: var(--ink2); }
select, .btn {
  font: inherit;
  color: var(--ink);
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 6px;
  padding: 5px 8px;
}
select { max-width: 100%; }
.btn { cursor: pointer; min-width: 38px; }
.btn.primary { background: var(--accent); color: #ffffff; border-color: var(--accent); }
.btn[aria-pressed="true"] { outline: 2px solid var(--accent); }
.drop {
  border: 1.5px dashed var(--axis);
  border-radius: 8px;
  padding: 8px 12px;
  color: var(--ink2);
}
.drop.over { border-color: var(--accent); background: var(--cell); }

.topology { width: 100%; height: auto; display: block; background: var(--surface); cursor: grab; touch-
action: none; }
.cell { fill: var(--cell); }
.link { stroke: var(--axis); }
.link-core { stroke: var(--muted); }
.gnb { fill: var(--node-fill); stroke: var(--ink2); stroke-width: 1.5; }
.core { fill: var(--node); stroke: var(--surface); stroke-width: 2; }
.node-label { fill: var(--ink2); font-size: 11px; }
.agent { fill: var(--agent); opacity: 0.7; }
.busy { fill: var(--serious); opacity: 0.35; }
.packet { stroke: var(--surface); stroke-width: 2; cursor: pointer; }
.ring-withdraw { fill: none; stroke: var(--critical); stroke-width: 2.5; }
.ring-announce { fill: none; stroke: var(--good); stroke-width: 2.5; }

.timeline { display: flex; align-items: center; gap: 8px; margin-top: 10px; flex-wrap: wrap; }
.timeline input[type="range"] { flex: 1; min-width: 160px; accent-color: var(--accent); }
.time { font-variant-numeric: tabular-nums; min-width: 72px; color: var(--ink2); }
.speed { display: flex; gap: 4px; align-items: center; color: var(--ink2); }

.legend { margin: 10px 0 0; line-height: 1.8; }
.key { display: inline-block; width: 10px; height: 10px; margin: 0 4px 0 10px; vertical-align: -1px; }
.key.gnb { border: 1.5px solid var(--ink2); background: var(--node-fill); border-radius: 2px; }
.key.core { background: var(--node); border-radius: 50%; }
.key.agent { background: var(--agent); border-radius: 50%; width: 6px; height: 6px; }
.key.busy { background: var(--serious); opacity: 0.5; border-radius: 50%; }
.key.ring-g { border: 2px solid var(--good); border-radius: 50%; }
.key.ring-r { border: 2px solid var(--critical); border-radius: 50%; }

.tiles { display: grid; grid-template-columns: repeat(4, 1fr); gap: 8px; }
.tile { display: flex; flex-direction: column; gap: 2px; }
.tile-label { font-size: 12px; color: var(--ink2); }
.tile-value { font-size: 20px; font-weight: 600; }
.bars, .waterfall svg { width: 100%; height: auto; }
.bar { fill: var(--accent); }
.bar.hover { opacity: 0.8; }
.bar-label { fill: var(--ink2); font-size: 11px; }
.bar-value { fill: var(--ink); font-size: 11px; font-variant-numeric: tabular-nums; }
.grid { stroke: var(--grid); stroke-width: 1; }
.axis-label { fill: var(--muted); font-size: 10px; }
.intent { font-weight: 600; margin: 8px 0 2px; }
```

```css
.row { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; }
.row select { flex: 1; min-width: 0; }

.feed { max-height: 260px; overflow-y: auto; display: flex; flex-direction: column; gap: 2px; }
.feed-row {
  display: grid;
  grid-template-columns: 58px 66px 1fr;
  gap: 6px;
  text-align: left;
  font: inherit;
  font-size: 12px;
  color: var(--ink);
  background: none;
  border: 0;
  border-radius: 4px;
  padding: 3px 4px;
  cursor: pointer;
}
.feed-row:hover:not(:disabled) { background: var(--cell); }
.feed-row:disabled { cursor: default; }
.feed-t { color: var(--muted); font-variant-numeric: tabular-nums; }
.feed-text { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
.tag { font-size: 11px; color: var(--ink2); }
.tag.deliver { color: var(--good); font-weight: 600; }
.tag.drop, .tag.fail { color: var(--critical); font-weight: 600; }

@media (max-width: 600px) {
  .picker, .picker label { width: 100%; }
  .picker select { flex: 1; min-width: 0; }
}
```